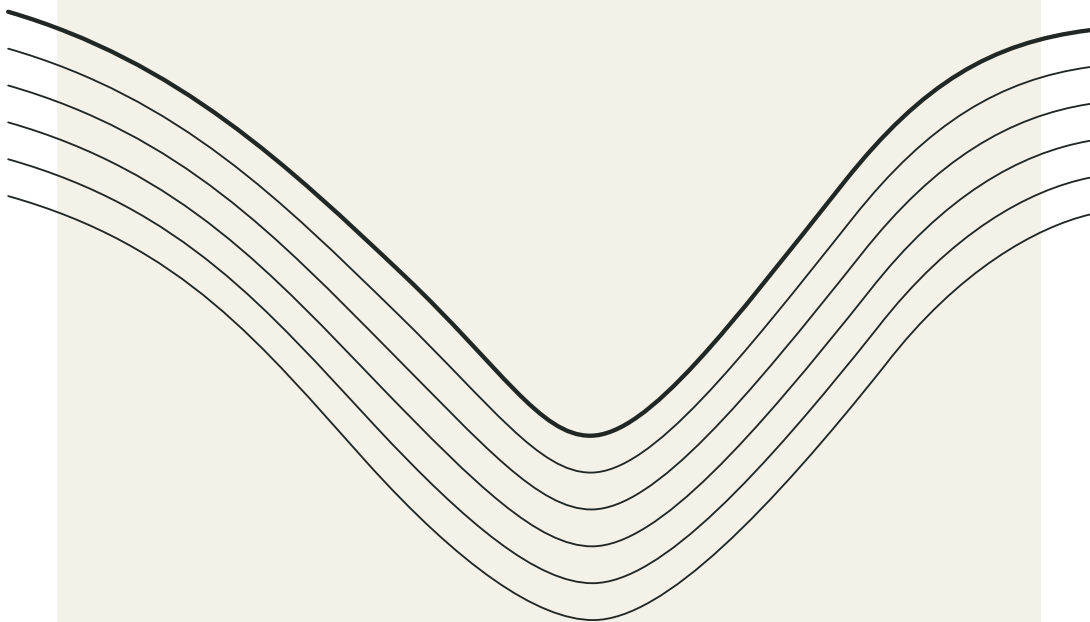


THE VALLEY RESEARCH

the-valley

Foundations & Research Opportunities

A critical literature and systems review



September 2026

READER EDITION

Contents

About this reading edition.....	6
Overview and principal findings.....	7
Principal findings	8
The report.....	9
Scope and method	12
How to interpret claims	15
1. What the-valley is trying to make possible	16
1.1 Three connected capabilities.....	17
1.2 Requirements as research questions.....	18
1.3 What exists at the snapshot	24
1.4 Six distinctions that organize the whole study	27
1.5 The strongest case for the project.....	30
1.6 The strongest case against it	31
2. Existing systems and the alternatives they make credible	33
2.1 A map of the alternatives	34
2.2 Sovereignty has several implementations.....	36
2.3 A pull request is one possible interface to a change	39
2.4 Concurrent integration is an established field	41
2.5 Portable execution does not settle evidence authority.....	43
2.6 Changes, operations, and content are different identities.....	44
2.7 Human review does more than discover defects.....	46
2.8 Recent work that narrows the agent-specific questions	47
2.9 What modularity should mean here	48

2.10 Credible architectural choices	49
3. Verification that can travel with a change	52
3.1. What exists, and what remains a claim	53
3.2. When equal inputs justify reusing a result.....	55
3.3. The closest intellectual relatives are build systems and selective testing	58
3.4. A signed executable hash is not evidence that the executable ran	61
3.5. What the supply-chain standards already solve	63
3.6. Integration is concurrency control over evidence, with qualifications	66
3.7. Effectful checks are observations with assumptions	70
3.8. Transparency records claims; revocation changes what may rely on them.....	72
3.9. What random witness sampling can and cannot promise	74
3.10. A research program that can discriminate between designs.....	76
3.11. The promising solution space	80
4. Events, trust, and durability.....	82
4.1. What exists, and what remains an ambition.....	83
4.2. A Git repository contains several different histories	85
4.3. Causality is not the order of entries in a log.....	89
4.4. Controllers and workflows solve different recovery problems	91
4.5. Reliable messages do not make external effects atomic.....	94
4.6. Durability needs a recoverable set of objects	97
4.7. Identity, authority, and evidence should stay distinct.....	99
4.8. Recursive governance needs a declared trust boundary.....	104

4.9. Federation should exchange evidence while preserving local authority	107
4.10. Experiments that would resolve the important uncertainties....	109
5. Knowledge, agents, and outcomes	112
5.1. The actual starting point	113
5.2. Knowledge preservation is a theory-reconstruction problem	114
5.3. Existing tools clarify the storage choices	118
5.4. Agent memory is several different systems	121
5.5. An outcome graph is a plan representation, not a complete planner	125
5.6. Human attention is a resource with special constraints	130
5.7. Evaluation should measure useful work and preserved understanding	132
5.8. The most promising research direction	138
6. Synthesis and a research agenda	141
6.1 What the review supports.....	141
6.2 Claims that should be narrowed before relying on them.....	143
6.3 A useful boundary for the minimal core	147
6.4 The most promising solution spaces	150
6.5 Experiments that would change the design	153
6.6 Measurement discipline.....	161
6.7 An opinionated end-to-end system.....	163
7. Reading guide and glossary	168
7.1 The shortest route through the foundations.....	168
7.2 Eight deeper study sessions.....	170
7.3 Questions for judging a new paper or tool.....	176

7.4 Working glossary	176
Sources S: architecture and existing systems.....	184
Sources V: verification and integration	195
Sources D: events, trust, and durability.....	209
Sources A: knowledge, agents, and outcomes.....	223
Repository source notes.....	239

About this reading edition

This edition contains the complete research report and its annotated sources. It is prepared for a Kindle Scribe, with pages sized for its screen, embedded fonts, and a linked table of contents. The research is dated September 20, 2026.

Start with the overview and chapter 6 for the main judgments. Read the chapters in order for the full argument. Chapter 7 offers a route through the original literature.

Citations bearing an S, V, D, or A source number link to the annotation included in this book. The annotation retains the link to the original publication, which needs an internet connection. Use the reader's Back command to return after following a link.

Compact comparison tables remain tables. Dense tables are presented as a sequence of labeled entries, and the architecture diagram is rendered as a list of connections. Repository references appear as notes containing the original file paths. Those source files are not included; they refer to the report's inspected repository snapshot, 5bb3a7736fe49b0cc4914299b142c0c9bafb6870.

Overview and principal findings

A critical literature and systems review, 2026-09-20

the-valley investigates how software development could work when authors are often agents, project state belongs to its operator, and acceptance decisions carry evidence that can outlive a hosting service. This report studies those goals against existing systems and academic literature. It identifies the distinctions needed to reason about the design, the strongest alternatives, and experiments that could establish whether its more ambitious ideas work.

The central question is whether **clear requirements and an opinionated set of end-to-end constraints can make established tools work better together**. A good existing solution to a requirement is a reason to adopt it. Innovation in an individual subsystem is optional. The project succeeds when the complete workflow meets its requirements with acceptable cost and complexity.

Portable acceptance evidence and project knowledge are especially important connections between the parts. They let work retain its justification as it moves between authors, tools, and hosts. Reliable effect recovery, precise authority boundaries, and honest completion semantics determine whether that continuity survives actual use. Their value should be measured in the whole workflow.

This report is independent research material. It lives outside `.the-valley/`, has no knowledge-node status, and does not amend the design or roadmap. Its judgments are proposals to consider.

Principal findings

1. **Evidence reuse is a strong idea with exact preconditions.** Equal relevant inputs can justify reuse of a deterministic check. The check definition, dependency model, execution assumptions, and current authority must all be explicit. A familiar check name is insufficient.
2. **Signing and verification establish different facts.** A signature authenticates a claim. It does not prove that a named program ran, that the test was adequate, or that an outcome was attained. The inspected design makes several stronger claims than its mechanisms support.
3. **Replay has three different jobs.** Rebuilding current state, recovering historical decisions, and finishing interrupted effects require different records. Current Git ref enumeration handles only part of the first job.
4. **Existing solutions are candidates to adopt as well as baselines.** Zuul, Gerrit, distributed collaboration systems, and portable build runtimes make important parts of the problem familiar. Each deserves a fit assessment against the requirements before custom work replaces it.
5. **The knowledge graph should be tested through unfamiliar changes.** Recall and retrieval are

intermediate measures. The decisive question is whether a fresh contributor can apply the design with fewer conceptual mistakes and less later correction.

6. **A work graph does not define success.** Readiness, accepted attainment, cancellation, abandonment, and continuing obligations need distinct meanings. Scheduling more attempts cannot repair an inadequate acceptance criterion.
7. **The complete system is the unit of success.** The important test is whether shared constraints preserve intent, evidence, authority, and recovery across a useful workflow. Each custom component needs a demonstrated reason to exist. None needs to be novel for the system to be worthwhile.

These findings are developed and qualified in the chapters below. Static implementation findings are not reported as demonstrated exploits. Proposed experiments are not reported as results.

The report

1. Project and problem map

Subject: Goals, implemented state, and research questions

What it should make clear: The difference between the immediate infrastructure project and the larger outcome engine

2. Existing systems

Subject: Forges, portable collaboration, review, queues, and execution

What it should make clear: Which alternatives are credible and what each already solves

3. Verification and integration

Subject: Nix, incremental computation, evidence, concurrency, supply-chain trust

What it should make clear: When evidence transfers, what signatures mean, and what witnesses can establish

4. Events, trust, and durability

Subject: Causality, replay, effects, recovery, delegation, and governance

What it should make clear: Which facts and obligations must survive failure

5. Knowledge, agents, and outcomes

Subject: Design rationale, memory, planning, scheduling, attention, and evaluation

What it should make clear: How to distinguish useful understanding and attained outcomes from stored context and activity

6. Synthesis and research agenda

Subject: Judgments, architectural choices, and nine experiments

What it should make clear: What to investigate next and which results would change the design

7. Reading guide and glossary

Subject: A structured route through the literature

What it should make clear: How to deepen understanding and keep related concepts distinct

The annotated bibliography is divided into systems and architecture, verification, distributed systems and trust, and knowledge and agents. Entries identify contribution, relevance, limitations, and the material actually read. Source IDs S, V, D, and A are local to those registers. When two chapters use the same work, its appearance in two registers does not make it independent evidence.

For a first pass, read chapters 1 and 6, then the chapter nearest the next engineering decision. For a sustained study, follow chapter 7 and return to the experiments after reading the primary sources. For critical design review, start with the guarantee table in chapter 6 and follow each row to its supporting analysis.

Scope and method

The repository snapshot is 5bb3a7736fe49b0cc4914299b142c0c9bafb6870, whose latest commit is dated 2026-09-01. The working tree was clean before this report was added. The review examined the premise, scenario ladder, requirements, architecture, detailed designs, selected knowledge nodes, schemas, and relevant implementation paths. It treated proposed nodes as proposals and checked code where a claim concerned implemented behavior. It did not inspect the deployed host or external private systems referenced by the project.

The external review was organized around the project's questions rather than one discipline's terminology. Searches followed these connections:

Checks should run once and travel

Search and reading families: Functional deployment, incremental builds, safe test selection, attestation, builder trust

Integration should remain fast

Search and reading families: Optimistic concurrency, speculative gating, merge trains, collaboration conflicts

One history should explain effects

Search and reading families: Logical clocks, provenance, event sourcing, reconciliation, outboxes, sagas

Authority should be bounded and revocable

Search and reading families: Complete mediation, capabilities, workload identity, secure metadata updates, human-presence assertions

Agents need durable understanding

Search and reading families: Theory building, design rationale, truth maintenance, retrieval, memory, software-agent evaluation

Outcomes should drive useful work

Search and reading families: Goal refinement, hierarchical planning, DAG scheduling, stopping criteria, human attention

Sources were selected for a direct connection to a requirement, a mechanism the project could adopt, or a result that challenges an architectural assumption. The review favored original papers, author-hosted texts, standards, and official documentation. It used references within discovered papers to follow important intellectual predecessors. Product claims are identified as documented behavior, not independent evaluations. Recent preprints are labeled and treated cautiously.

This is a **critical narrative review with a scoping survey of systems**, not an exhaustive systematic review or meta-analysis. There was no preregistered database protocol, exhaustive screening count, independent reproduction of published experiments, or attempt to pool effect sizes across unlike studies. Search visibility, available full text, and the repository's own framing influence coverage. The source registers make those limits inspectable rather than hiding them behind a citation count.

The study deliberately gives less attention to public-forge abuse, commercial procurement, general distributed consensus engineering, and broad philosophical claims about model cognition. These are either deferred by the project or unnecessary to decide the near-term questions. It

also avoids claiming that a sampled literature search establishes novelty or that any existing system implements the entire proposed combination.

How to interpret claims

Repository observation describes the inspected snapshot.

Established result describes a theoretical construction or finding with the cited work's assumptions. **Documented behavior** is an implementation's published contract.

Inference applies those ideas to the-valley.

Recommendation expresses a design judgment.

Hypothesis requires measurement. The prose uses these distinctions where confusion would materially affect a decision.

An abstract-only source is useful for locating a result and reporting what its authors summarize. It does not support detailed claims about methods that were not inspected. Living documentation can change after the access date. Versioned papers and documentation are used where available; this report does not archive full copies of the external corpus.

The report's validation concerns its own prose, local references, citation coverage, and consistency. Its experimental agenda remains future work. Findings can be adopted into project decisions later, with their scope and supporting evidence preserved.

1. What the-valley is trying to make possible

the-valley is an experiment in making software work portable, verifiable, and understandable across changes of host and author. Its immediate application is a developer working with coding agents. Its larger ambition is a system that turns requested outcomes into completed work while preserving the reasons, authority, and evidence behind each consequential action.

This chapter reconstructs that ambition from the repository. The later chapters ask which parts have established solutions, which are engineering integration problems, and which remain research questions. The reconstruction describes `commit 5bb3a7736fe49b0cc4914299b142c0c9bafb6870`. It is an interpretation for this report, not a replacement for the project's requirements.

The engineering aim is to satisfy clear requirements using good existing solutions wherever they fit. The project's constraints determine which combinations are acceptable. Research helps clarify those requirements, discover reusable solutions, and test the complete system. A novel mechanism is one possible result of that work, never a prerequisite for success.

1.1 Three connected capabilities

The first capability is **durable, independently operated development infrastructure**. Code and project knowledge should survive the disappearance of a hosting provider. Normal development should remain recognizable. The scenario ladder therefore starts with hosting, replication, and demonstrated recovery, rather than with a new collaboration interface.

The second is **integration that accepts evidence produced before submission**. A contributor should be able to establish that the required checks passed where the work was authored. The integrator should decide whether that evidence still applies to the state it will publish. This is more specific than making CI faster: it changes the relationship between execution, evidence, and admission.

The third is **a persistent account of intent that can drive further work**. Outcomes, dependencies, decisions, and explanations become inputs to agents. The graph should help a fresh agent understand the project and choose useful work. Completion must eventually mean that the requested result exists, rather than that a worker has stopped or a status field has changed.

These capabilities reinforce one another, and each can also be useful independently. Portable evidence could be useful with an ordinary forge. A well-maintained knowledge graph could help agents without an autonomous scheduler. Reliable self-hosting could be valuable even if the outcome engine never materializes. Those independent uses help

diagnose and compare components. The primary evaluation still concerns the complete workflow: whether the same intent, evidence, and authority remain meaningful as work passes between them.

The local basis is the premise¹, requirements², scenario ladder³, and outcome sketch⁴.

1.2 Requirements as research questions

The requirements mix desirable experiences with technical guarantees. Each needs an observable test. The following translation preserves their intent while making the questions more precise. These questions guide both the selection of existing tools and the evaluation of their composition.

S1: repositories cannot be lost

Question that can be investigated: Which acknowledged states remain recoverable after which failures, and within what time?

Principal literatures: Replication, backup consistency, local-first systems

Report treatment: Chapter 4

S2: integration in seconds

Question that can be investigated: Under which workload and trust assumptions can already-verified work be integrated in seconds?

Principal literatures: Incremental computation, build systems, concurrency control

Report treatment: Chapter 3

S3: attributable autonomous authors

Question that can be investigated: What can the system establish about a run's origin, authority, actions, and checks?

Principal literatures: Workload identity, capabilities, provenance, software agents

Report treatment: Chapters 4 and 5

S4: one history explains effects

Question that can be investigated: What facts must be retained to reconstruct an action and distinguish observation from explanation?

Principal literatures: Distributed causality, event sourcing, workflow recovery, provenance

Report treatment: Chapter 4

S5: bounded and revocable trust

Question that can be investigated: Which permissions are checked at each boundary, and when does withdrawal take effect?

Principal literatures: Authorization, delegation, secure update systems

Report treatment: Chapters 3 and 4

S6: incidents are attributed and remembered

Question that can be investigated: What evidence supports attribution, and how is uncertainty represented?

Principal literatures: Causal reasoning, operational learning, knowledge maintenance

Report treatment: Chapters 4 and 5

Demand-shaped work

Question that can be investigated: How are goals refined, prioritized, attempted, evaluated, and abandoned without losing the original intent?

Principal literatures: Goal-oriented requirements, planning, scheduling, human automation

Report treatment: Chapter 5

Minimal, portable composition

Question that can be investigated: Can an implementation be replaced without rewriting the project's authoritative state?

Principal literatures: Modular design, protocols, interoperability

Report treatment: Chapter 2 and Chapter 6

This translation exposes a useful distinction. "Runs in seconds" is a performance target. "Cannot accept the wrong check" is a safety property. "Eventually processes every eligible request" is a liveness property. "Still works after the primary host is destroyed" is a recovery property. One successful demonstration cannot establish all four.

Keep requirements, constraints, and mechanisms separate

A requirement states the behavior that must hold for an actor in a specified situation. A constraint limits the acceptable solutions. An architectural bet selects a way to meet the requirement within those constraints. An implementation supplies that mechanism. Keeping these statements separate makes it possible to adopt an existing tool without accidentally changing the intended result.

For example, preserving the explanation for an accepted change is a requirement. Operator ownership of the retained record can be a constraint. Storing that record in Git is an architectural choice. The same requirement might be met by another store with suitable recovery and export behavior. A preference for Git should therefore be argued as a means of satisfying the chosen constraints.

The premise already imposes open source, minimalism, Nix in the reference implementation, and decentralization where possible. Their exact consequences need interpretation. Minimalism could mean few components, little custom code,

or low operating burden; those are different objectives. The report recommends judging it by the understanding and work needed to operate the complete system.

Questions that must be answered before choosing the composition

The existing requirements leave several boundaries unsettled. These are proposed clarification questions, not silently adopted answers:

- **Acknowledged durability.** If the primary host disappears immediately after a successful integration, may that change be lost? "Integrated means replicated" and replication "within minutes" describe different acknowledgment policies. Define the allowed loss window, independent failure domains, and which record the durability promise covers.
- **Latency and capacity.** What workload, percentile, resource budget, and evidence prerequisites does "seconds" assume? Report author-side checking as well as submission-to-acceptance time. State what happens when changes invalidate one another or arrive faster than they can be processed.
- **Trust and attribution.** Is an authorized but dishonest author in scope? Which observer is trusted to report execution and identify an agent run? Equal acceptance guarantees for humans and agents need not imply equal permissions. These answers determine which existing verification and identity mechanisms are sufficient.

- **History and explanation.** Which accepted decisions, failed attempts, authority changes, and external actions must remain explainable after losing the controller and bus? Distinguish observed events from an incident's inferred cause. An uncertain external result needs an explicit state.
- **The recoverable project.** Does recovery preserve only code and knowledge, or also the evidence, policy snapshots, trust roots, artifact references, and outstanding obligations needed to interpret and continue its work? Identify what can be discarded or reconstructed.
- **Completion.** What observation establishes that the requested result exists, and who may accept it or revise the criterion? A landed change, a cancelled prerequisite, and an attained outcome need separate meanings. State how a continuing obligation is reassessed.
- **Operating burden.** What does adding a project or collaborator require? Which credentials, policy surfaces, upgrades, and recovery procedures must the operator understand? Test whether the operator can resume safely after an absence and replace a component without redesigning its neighbors.
- **Execution equivalence.** Must local and remote checks use the same identified definition and environment, or literally the same backend? Which results must be reproducible, and which are observations with a freshness limit? State what a non-Nix consumer must understand or reproduce.

The scenario ladder deliberately postpones detailed acceptance criteria for later rungs. Preserve that discipline. Clarify a boundary when the active workflow depends on it, while recording later questions as unresolved. The proposed constraints in [section 6.7](#) help organize those decisions without prematurely selecting their implementation.

The report treats S7, public contribution by strangers, as a boundary condition. It does not recommend building a public federation or an abuse-management system before the solo and small-team cases are useful. However, mechanisms introduced at S2 and S3 should state their trust assumptions so that their limits remain visible at S5.

1.3 What exists at the snapshot

The repository contains considerably more implementation than its opening status paragraph alone suggests. It also contains proposed behavior that implementation comments sometimes describe in the present tense. A research report needs a separate inventory.

Hosting and replication

Evidence in the repository: Host module⁵, host schema⁶, scenario acceptance checkboxes

What this report can conclude: Configuration and test artifacts exist. The scenario document records a restore exercise. This study did not independently inspect the deployed host or repeat that restore.

Knowledge convention

Evidence in the repository: Graph documentation⁷, node schema⁸, lint⁹

What this report can conclude: Typed Markdown nodes and validation are implemented. Semantic adequacy of their contents is a separate question.

Evidence production

Evidence in the repository: Attestation implementation¹⁰, schema¹¹, conformance vectors¹²

What this report can conclude: Tree identification, statements, signed notes, and verification tooling exist. The existence of a signature does not establish the truth of every field.

Integration

Evidence in the repository: Controller¹³, verdict¹⁴, detailed design¹⁵

What this report can conclude: A polling controller evaluates requests and transfers evidence. It is not yet the event-subscribing controller suggested by the architecture overview.

Identity

Evidence in the repository: Registry schema¹⁶, compiler¹⁷, governance decision¹⁸

What this report can conclude: Registry compilation exists. This does not establish enforcement of every proposed delegation or human-presence rule.

Events

Evidence in the repository: Event schema¹⁹, CLI²⁰, integrator publication²¹

What this report can conclude: Ref projections and integration notifications exist. Current-state reconstruction and complete historical recovery are different properties.

Witnesses and transparency

Evidence in the repository: Verification design²², roadmap²³

What this report can conclude: The full independent re-verification and transparency backstop remains a proposed layer in the inspected material.

Outcome production

Evidence in the repository: Demand-pressure idea²⁴, production-DAG idea²⁵

What this report can conclude: The conceptual model is developed. This study found no basis to call the complete general outcome engine implemented or evaluated.

Recursive transparency

Evidence in the repository: Draft invariant²⁶, architecture bootstrap discussion

What this report can conclude: Governance induction has concrete design, but the claimed closure over every durable effect remains unresolved.

These are static observations. The report does not certify a deployment, claim a security audit, or claim benchmark results. Its experimental designs are proposals, clearly separated from findings.

1.4 Six distinctions that organize the whole study

Identity, authority, and truth

A signature can establish that a key signed some bytes. A policy can establish that the key was authorized to make a particular decision. Neither alone establishes that an asserted computation occurred, that the computation tested a useful property, or that the requested outcome was achieved.

Consider a statement saying that a migration test passed. The signature might be valid. The signer might be an approved development host. The test might nevertheless contain no assertions. These are three separate questions, requiring three separate kinds of evidence.

This distinction should govern both local checks and agent provenance. A host can attest that it launched a particular harness with particular inputs. That provides an accountable observation. It does not identify an independently acting model principal or expose all the causes of the model's output. Chapters 3 and 4 develop the relevant trust boundaries.

State, history, and explanation

A checkout answers what the project currently contains. A durable operation record answers what happened to it. An explanation also needs the intent, policy, observations, and assumptions that made an action reasonable at the time.

Suppose a branch is created, rejected twice, rewritten, integrated, and deleted. The final source tree may retain the result while losing most of that sequence. Conversely, retaining every event still does not explain why a requirement mattered. The report therefore evaluates state recovery, historical recovery, and explanation quality separately.

Valid computation and adequate judgment

Reproducing a result establishes agreement about a computation under some assumptions. It does not establish that the computation was the right way to decide whether to act. The same separation applies to an LLM judge: a consistently repeated assessment can still use an inadequate rubric.

This is especially important for the proposed progression from judgment by agents to deterministic checks. The useful question is which parts of a decision have become stable enough to encode. A mechanized proxy should retain a link to the requirement it approximates and to examples it gets wrong. Otherwise the system can improve the proxy while losing the purpose.

Availability and authority

Many replicas can make an object easy to retrieve. They do not decide which object is the accepted project state. A designated integrator can supply that decision without being the only holder of the data. Federation can also replicate a proposal without requiring the receiving valley to accept its signer or policy.

This makes decentralization a collection of choices: storage, discovery, identity, decision making, and recovery authority. Counting servers is an inadequate measure. Chapter 2 compares systems that make different choices along these dimensions.

Eligibility and attainment

A task is eligible when its prerequisites permit an attempt. An outcome is attained when the desired condition holds. Closing blockers may establish eligibility. It cannot generally establish attainment, because the decomposition may be incomplete or a prerequisite may have been abandoned.

The current graph's terminal-state convention is therefore an important boundary. It is useful for administrative traversal, but an autonomous scheduler needs to know whether "no longer blocking" means satisfied, waived, replaced, or impossible. Chapter 5 treats this as a central design problem.

Fast admission and safe consequences

A change can become part of source history before every possible consequence is authorized. A low-risk documentation edit and an irreversible external action need not share one evidence threshold. This is a research recommendation, not a statement of current policy.

Separating these decisions gives the latency ambition more room. Local evidence might justify integration while an independent check is still required for a particular deployment. The resulting system must make both decisions explicit so that "landed" never silently becomes "approved for every effect."

1.5 The strongest case for the project

An important engineering requirement is to make **the reason an action was accepted portable**. Source code already travels well. Approval context, check definitions, evidence, and authority often travel less well. the-valley can investigate a compact record that lets another implementation reconstruct a decision without consulting the original service.

An important empirical question is how to make **project understanding testable across fresh authors**. The theory-revival idea²⁷ asks whether a well-maintained graph helps a new agent preserve a project's design. The evaluation sketch²⁸ suggests why-questions, proposed changes, and violations as probes. Those are substantially better targets than measuring the amount of stored context.

Their connection is central to the end-to-end system. An agent could use project knowledge to propose a change, attach evidence to it, and leave a durable record of what the system accepted. Subsequent agents could then distinguish accepted decisions from speculative notes and failed attempts. The interesting object is that entire learning cycle. Each component has extensive prior art; whether their composition improves sustained development remains an empirical question.

1.6 The strongest case against it

The project could recreate a forge, a workflow engine, an authorization service, a build coordinator, and an agent memory system under separate names. Small executables do not ensure a small operating burden. If every component needs custom recovery rules and tightly coordinated upgrades, the overall system may become harder to replace than a conventional platform.

It could also obtain speed mainly by weakening the acceptance criterion. Accepting a trusted host's claim immediately is a legitimate policy for a personal project. It

should be compared with another system given the same trust policy. Comparing that path only with a remote build that independently executes everything would confound trust and performance.

Finally, it could confuse richer records with better judgment. More provenance, dependencies, and agent transcripts can increase both storage and attention cost without reducing mistakes. The right tests ask whether those records change decisions, shorten recovery, or improve subsequent work.

These objections should shape a small end-to-end reference workflow assembled from the strongest available components. Tests of evidence transfer, recovery, and knowledge quality can then explain where that workflow succeeds or fails. The project need not demonstrate a separate innovation in each subsystem before evaluating their joint value. Chapter 6 sets out the composition constraints, adoption rule, and experiments for that evaluation.

2. Existing systems and the alternatives they make credible

Most individual mechanisms proposed by the-valley have substantial prior art. These systems are candidates to adopt or configure. The selection question is whether they satisfy the requirements and preserve the chosen guarantees when combined. The relevant comparison concerns complete workflows, including the work needed to connect, operate, and replace their components.

This chapter compares documented mechanisms. It does not report hands-on product benchmarks. System documentation was consulted on 2026-09-20; the [source register](#) records versions and reading depth. Evaluations of suitability are this report's judgments.

2.1 A map of the alternatives

Self-operated forge

Systems worth understanding: Forgejo, SourceHut

Problem substantially addressed: Hosting control and established collaboration

Question still belonging to the-valley: Does replacing the collaboration model provide enough benefit to repay integration work?

Distributed collaboration

Systems worth understanding: Radicle, Fossil, Tangled

Problem substantially addressed: Keeping collaboration independent of a single conventional forge

Question still belonging to the-valley: Which state and authority must travel with a project?

Change-oriented review

Systems worth understanding: Gerrit; email with b4

Problem substantially addressed: Separating a proposed change from an ordinary branch

Question still belonging to the-valley: What is the stable identity of a change and its evidence?

Concurrent gating

Systems worth understanding: Zuul, GitHub merge queues, GitLab merge trains

Problem substantially addressed: Testing combined changes while maintaining merge order

Question still belonging to the-valley: Can evidence transfer reduce repeated work without weakening admission?

Portable execution

Systems worth understanding: Nix, Bazel, Dagger

Problem substantially addressed: Sharing execution definitions and reusing results

Question still belonging to the-valley: Who is authorized to assert that a particular result occurred?

Version-control experiments

Systems worth understanding: Jujutsu, Pijul

Problem substantially addressed: Recording operations and treating changes more explicitly

Question still belonging to the-valley: Which ideas can improve authoring without replacing the durable protocol?

Durable reactions

Systems worth understanding: Kubernetes controllers, Temporal

Problem substantially addressed: Reconciliation and recovery of long-running work

Question still belonging to the-valley: What remains the system of record for effects and decisions?

Project memory and agents

Systems worth understanding: git-bug, Beads, memory systems, agent frameworks

Problem substantially addressed: Persistent coordination and context

Question still belonging to the-valley: Which knowledge is authoritative, and how does it improve attained outcomes?

The last two rows receive detailed treatment in chapters 4 and 5. Nix's foundations and evidence systems receive detailed treatment in [chapter 3](#).

2.2 Sovereignty has several implementations

Forgejo is the practical baseline for operating an ordinary forge. Its documented administration includes exporting files and databases and restoring repository units such as issues, pull requests, comments, and releases. This

shows that owning the hosting and recovering collaboration data need not require a new source-management architecture. It also shows why a Git clone alone is a narrower recovery artifact than the complete service. S05: Forgejo CLI

The implication is a burden of proof for the-valley. If the immediate problem is control of hosting, an established forge plus tested backups deserves an actual place in the decision set. A new substrate becomes compelling when portability of decisions or changes to authoring and integration matter enough to justify its additional responsibilities.

SourceHut demonstrates that unbundling can retain familiar collaboration. Its project hub was designed to organize independently composed repositories, lists, and trackers, with no fixed one-repository-to-one-tracker relationship. That is direct prior art for separating concerns while still offering a coherent project surface. S06: SourceHut project hub

The lesson is architectural rather than aesthetic. A common interface does not require one authoritative object model, and separate components do not require an incoherent experience. the-valley's derived views could follow this principle. Their replacement cost should be low even when the daily interface is cohesive.

Fossil provides the strongest compact alternative to the assumption that minimalism means unbundling. It combines version control with project facilities in a self-

contained executable. Its own comparison describes cloning the project's public history and associated material together. This is a project-authored comparison, so its favorable usability judgments are not independent measurements.

S04: Fossil versus Git

For the-valley, Fossil asks an uncomfortable but productive question: is the unit to minimize a component's code, the number of deployment units, or the operator's total burden? Those objectives can conflict. A distributed set of small tools may need more backup procedures and upgrade coordination than an integrated program. The report recommends measuring operating burden instead of treating composition itself as evidence of simplicity.

Radicle is the closest prior art for portable, authenticated collaboration over Git. Its protocol stores issues, patches, and identity changes as collaborative objects. Peers replicate signed data, and repository delegates determine canonical state. Its object graphs also represent concurrent operations. S03: Radicle protocol

The important distinction is that replication and acceptance are separate. A valley could receive a proposal through such a transport while retaining its own evidence policy. Adopting a transport need not entail adopting every review convention. Conversely, designing another Git namespace for social objects deserves justification against this existing model. Peer availability, retention, and the meaning of canonical state still need operational evaluation; the protocol description is not a restore test.

Tangled is a particularly relevant adjacent design. Its documentation separates Git hosting servers, called knots, from a network view, using AT Protocol for the wider collaboration system. [S08: Tangled overview](#) Its spindle component adds workflow execution with Nixery and microVM engines, while retaining a YAML pipeline format. [S09: Tangled spindles](#)

This combination makes Tangled useful for comparison even though its current workflow choices differ from the-valley's. Decentralized hosting, derived views, and Nix-related execution can coexist without requiring local evidence to authorize integration. The comparison isolates the-valley's additional bet: whether an evidence protocol is a better boundary than a service reporting pipeline status. No interoperability between these systems has been tested in this study.

2.3 A pull request is one possible interface to a change

Gerrit already distinguishes a change from the commits that revise it. Within a repository and target branch, a stable Change-Id groups patch sets across revisions. The review record can therefore follow a logical proposal through amended commits. Gerrit also distinguishes the author, uploader, reviewers, and submit strategy. [S10: Gerrit changes](#)

This matters because the-valley's tree identity and a user's concept of "this change" have different jobs. A tree digest identifies content. A proposal identity preserves a conversation across content changes. An integration attempt identifies a decision about particular content under particular policy. Eliminating the PR object does not eliminate these three relationships.

Gerrit's submit requirements are direct precedent for policy stored with a project. Requirements are expressed in project configuration and can apply to changes to that configuration. The docs explicitly discuss locking out future policy changes through misconfiguration and the recovery paths available to administrators. [S11: Submit requirements](#)

The research implication is that recursive governance is a familiar systems problem with practical failure modes. the-valley adds stronger ambitions for signed decisions and portable history, but it should learn from systems that already have to recover from rules that prevent their own amendment.

Email-based contribution separates submission from repository write permission. b4 retrieves patch series by message identity, checks attestations, and assembles patches and review trailers for maintainers. This is a working precedent for proposals arriving through a transport the maintainer does not equate with authority to update the accepted branch. [S07: b4 patch retrieval](#)

The relevant lesson is not that email should be the new interface. It is that transport independence, attribution, review history, and a maintainer's final decision have long been separable. A valley can borrow that separation while providing a more structured machine interface.

The difficult design work moves to continuity. After a proposal is rewritten, which feedback remains relevant? Which approvals must be renewed? Can a reader distinguish approval of an intent from approval of one exact patch? A derived review page still needs explicit answers in the underlying records. These are semantic requirements, regardless of whether the interface resembles a PR.

2.4 Concurrent integration is an established field

The simple model "remote CI runs serially before every merge" is an inadequate baseline.

Zuul tests speculative combined states concurrently. If changes A, B, and C are queued, it can test A, A+B, and A+B+C in parallel. Failure invalidates affected speculative successors. It also supports dependencies and shared queues across projects. [S12: Zuul gating](#)

the-valley instead asks when a check result obtained on one state can apply to another without re-execution. These approaches are compatible: speculation chooses future states to examine, while evidence transfer decides which computation is already valid for a chosen state. A

comparison should measure both strategies on independent edits, shared dependencies, frequent failures, and highly contended files. A simple serialized gate is still useful as a baseline, but it is not the strongest existing alternative.

GitHub's merge queue verifies temporary combined states and exposes build concurrency. Its documentation explains how merge-group events drive checks and how queue reordering can trigger new builds. [S13: GitHub merge queues](#) **GitLab's merge trains likewise run pipelines for queued combinations and remove failed changes from the train.** [S14: GitLab merge trains](#)

Those mechanisms are existing solutions to concurrent checking with ordered integration. The next question is whether an available configuration meets the project's evidence-reuse and portability requirements. A custom transfer rule needs a demonstrated gap in those solutions. A second executor and integrator agreeing on the same record would test the portability requirement, regardless of which project supplied the rule.

There is also earlier research on detecting conflicts before submission. Brun, Holmes, Ernst, and Notkin's Crystal work distinguishes textual conflicts from build and test conflicts. It explores speculative version-control operations to warn collaborators early. [S20: Proactive Detection of Collaboration Conflicts](#)

The implication for agents is to investigate coordination before the integration boundary as well as at it. A warning that another worker is changing the same abstraction can

prevent wasted work even when a merge queue would eventually catch an interaction covered by its required checks. Conversely, a warning based only on common file paths can be noisy. The useful signal is a predicted interaction with consequences for the ongoing task.

2.5 Portable execution does not settle evidence authority

Dagger addresses the local/CI runtime split directly. Its documented model uses code and a portable execution DAG with caching, usable before or after pushing a change. [S15: Dagger introduction](#)

This makes it a meaningful baseline for the complaint about a separate verification platform. Its execution model alone does not settle whether a different party should accept a result as sufficient evidence for integration. That authorization question remains available for a portable protocol, regardless of the executor.

Bazel makes two useful structures explicit: an action cache and a store addressed by output content. The action cache associates an action identity with result metadata; the content store holds the bytes. [S16: Bazel remote caching](#) Its hermeticity guidance also identifies host tools, timestamps, source-tree mutation, and other ways an ostensibly repeatable build can depend on undeclared conditions. [S17: Bazel hermeticity](#)

For the-valley, an evidence store needs more than artifact identity. It needs the claim, the computation or observation supporting it, and the policy that accepts the claimant. A useful cache hit and an authorized integration decision can share hashes without being the same fact. Chapter 3 examines how far Nix's dependency model can support that distinction.

A backend portability experiment should therefore implement one meaningful check with two executors. The point is not to compare their syntax. It is to discover whether the supposed common contract contains assumptions that only one backend can satisfy. Identical filenames in two schemas are weak evidence of portability; independently accepted conformance records are stronger evidence.

The place where a result becomes available also differs from the place where it was executed. A local Nix invocation may obtain an output from a configured binary cache. The evidence needs to distinguish executing, importing, observing, and endorsing a result. [Chapter 3](#) examines these trust assumptions and the independence required for a witness rebuild.

2.6 Changes, operations, and content are different identities

Jujutsu records repository operations separately from source history. An operation records a view of refs, heads, and workspaces, with parent operations. Its documented

recovery commands can inspect or restore earlier repository views, including situations involving concurrent operations.

S18: Jujutsu operation log

That is valuable conceptual precedent for the-valley's distinction between current refs and the history of what happened to them. An authoring tool's operation log does not automatically become an authoritative shared audit log. Retention, replication, signatures, and acceptance of events still need to be specified. It does show that source commits need not carry every kind of history.

Pijul explores version control in terms of changes and their dependencies. Its manual describes commutation of independent changes and a graph representation that retains conflict information. S19: Pijul theory

The narrow lesson is to keep change identity conceptually separate from a particular base and snapshot. The stronger lesson must be resisted: algebraic properties of text changes do not establish that their combined program satisfies a requirement. Two textually independent changes can alter a shared behavioral assumption. the-valley still needs semantic checks and a policy for their reuse.

These tools are worth studying without making replacement of Git a prerequisite. The current project deliberately preserves native Git workflows. A better authoring client can remain optional while the evidence protocol defines the acceptance boundary.

2.7 Human review does more than discover defects

The empirical case for review is broader than a pass/fail gate. Bacchelli and Bird's study of modern code review describes knowledge transfer, awareness, alternative solutions, and understanding of changes alongside defect finding. It is evidence about the studied development setting, not a proof that all projects need the same review practice. S22: Expectations, Outcomes, and Challenges of Modern Code Review

The implication is that removing routine human review creates obligations beyond replacing its correctness checks. A system must explain how important design changes become shared understanding. For one operator and many short-lived agents, the knowledge graph may perform part of that function. For a small team, selected discussions and summaries may still matter even when no human signature is required before integration.

The report's recommendation is to distinguish approval, learning, and attention. Approval authorizes an action. Learning updates project understanding. Attention directs a limited human resource. A single review interaction can serve all three, but evaluating only approval latency would hide losses in the other two.

2.8 Recent work that narrows the agent-specific questions

Two recent preprints are particularly close to the-valley's problem. They merit attention, with less evidentiary weight than mature mechanisms or replicated empirical findings.

BulkPR-Bench studies selection and ordering of interacting proposed changes. It distinguishes partial delivery scores from safe completion of a whole queue. Its constructed candidate pools and fixed harness limit generalization to naturally occurring work. This report treats it as a 2026 preprint; a future conference label in its manuscript is not evidence of an already completed peer-reviewed publication. S21: Xiong et al., version 1

AI Agent Pull Requests on GitHub studies temporal overlap and replayed textual merges in an agent PR dataset. Its limitations explicitly distinguish textual conflict from semantic conflict and observed conflicts from unmeasured costs such as maintainer time. Its observation period predates the publication and should not be read as a measurement of every current agent workflow. S23: Xu, Subramanian, and Karthik, version 2

Together they suggest a useful experimental split. One experiment should measure contention and evidence invalidation given known changes. Another should measure whether agents discover the relationships that make those changes safe to combine. Better scheduling cannot

compensate for missing relationships, while a perfect relationship graph cannot by itself implement safe admission.

2.9 What modularity should mean here

Parnas's argument for information hiding concerns the decisions a module keeps from other modules. It does not equate modularity with dividing a process into many sequential steps. S01: On the Criteria To Be Used in Decomposing Systems into Modules

Applied to the-valley, a useful boundary lets the system change its build executor, bus, or review interface without changing what a historical acceptance record means. A poor boundary merely moves the same shared assumptions into several processes. This is the report's application of Parnas's criterion, not a claim that the paper evaluated developer platforms.

Saltzer, Reed, and Clark's end-to-end argument supplies a second discipline. A lower layer's success cannot necessarily establish the application's desired outcome. Their examples include delivery and duplicate suppression. S02: End-to-End Arguments in System Design

For this project, the inference is straightforward. Delivery to the event bus does not establish that a deployment occurred. A verified signature does not establish that a requested capability works. The layer that can observe the required

result must contribute to the evidence. Lower-level guarantees remain useful, but their scope needs to remain visible.

2.10 Credible architectural choices

The following are comparison configurations, not recommendations to migrate immediately.

Conventional forge, Nix checks, tested backups

Why it is credible: Addresses hosting and execution with few new semantics

Principal sacrifice: Accepted decisions remain tied to forge records unless exported

What an experiment should compare: Operator time, recovery completeness, contribution latency

Gerrit or ordinary review plus Zuul

Why it is credible: Strong comparison for policy and concurrent integration

Principal sacrifice: More infrastructure and established review conventions

What an experiment should compare: Throughput, retesting cost, failures under contention

Portable collaboration plus valley evidence

Why it is credible: Reuses transport and social records while testing the evidence boundary

Principal sacrifice: Interoperability and dual-model complexity

What an experiment should compare: Can another host independently reconstruct an acceptance?

Git plus the current narrow valley components

Why it is credible: Preserves present workflow and concentrates new code on evidence

Principal sacrifice: Valley must own recovery and evidence semantics

What an experiment should compare: Equal-trust comparison with the first two configurations

Outcome engine layered over any of the above

Why it is credible: Separates knowledge and planning research from hosting choices

Principal sacrifice: Requires a stable boundary between proposals and authorized actions

What an experiment should compare: Verified outcomes, revision cost, understanding, human attention

The report favors the fourth configuration as the immediate experiment, with the first two as serious baselines. The fifth should be evaluated as an additional layer. That position preserves the unusual parts of the-valley's ambition while giving each of them a chance to fail independently.

3. Verification that can travel with a change

The-valley's strongest verification idea is that a check result should remain useful when work moves between machines, authors, and integration attempts. A developer should not have to repeat an expensive computation merely because another service wants to see it happen. That idea has strong foundations in build systems, incremental computation, and database concurrency control. Its difficult part is establishing what the computation actually means, who can be trusted to report it, and when its result still applies.

This chapter's research conclusion is favorable to portable evidence and selective reuse. It is considerably less favorable to treating signed local execution plus eventual auditing as a substitute for a protected verification boundary in adversarial settings. Those are separate architectural choices. The first does not require the second.

This is a research assessment of repository commit 5bb3a7736fe49b0cc4914299b142c0c9bafb6870, with external sources consulted on 2026-09-20. It is not a change to project policy. **Established result** means a result or construction in the cited literature; **documented behavior** means a system's published contract; **inference** means this report's reasoning about the-valley; **recommendation**

means a proposed direction; **hypothesis** means something requiring measurement. The annotated source register records reading depth and limitations.

3.1. What exists, and what remains a claim

The repository contains substantial verification machinery. The attestation helper²⁹ exports a committed tree, runs selected checks, records their subjects and results, signs statements, and stores them in Git refs. Verification³⁰ checks statement syntax, schema, signatures, and optionally the subject tree. It does not independently execute the claimed check. The integrator³¹ computes a candidate landing, derives requirements from its own policy sources, gathers evidence, and asks the verdict library³² whether that evidence transfers. Its protected-ref update uses compare-and-swap.

The distinction between implementation and aspiration matters. The roadmap³³ places witness rebuilding and the transparency log in a later phase. A signed statement about execution is implemented. The inspected implementation does not yet provide independent re-verification of those local execution claims. Nor is a quantitative assurance level established merely by describing a future trust score.

The current design separates two claims. A pure check claims that a fixed computation over fixed inputs produced a result. An effectful check claims that an environment

observed a result at a particular time. That distinction is valuable. It should be supplemented by several independent acceptance questions:

Question	What can answer it?	What does not answer it?
Are these the bytes the signer signed?	Signature verification and unambiguous encoding	Whether the signer told the truth
Does the evidence describe the required computation?	Binding to the approved definition, inputs, platform, and policy	A familiar check name
Does the required computation support the claimed result?	A trusted execution boundary, independent execution, or suitable proof	A self-reported executable hash
Is that computation an adequate test of the desired property?	Specification, test design, review, and empirical validation	Reproducibility alone
Is accepting it authorized now?	Current authorization and revocation policy	An old valid signature alone

These are not five competing definitions of trust. They are five obligations that an acceptance decision can accidentally collapse into one green result.

A successful independent rerun supports the claimed result under the rerun's assumptions. It does not prove that the signer's particular historical execution occurred. A policy concerned only with the result may find the rerun sufficient.

A policy requiring evidence of who executed what at a particular time needs a trustworthy contemporaneous observation as well.

The current implementation is a useful base because it already exposes the subject, predicate, signer, and transfer decision separately. The research priority is to make each obligation explicit at those existing boundaries, before adding a reputation system whose score might conceal which obligation remains unsatisfied.

3.2. When equal inputs justify reusing a result

For a deterministic function, equal inputs imply equal outputs. The engineering work is deciding what counts as the function and all its inputs. Consider a check that compiles a package and runs its tests. Its inputs include the package source, tests, compiler, libraries, flags, generated data, and any runtime behavior on which the answer depends. If the check consults a remote feature flag or the host clock, those observations also matter.

Dolstra's Nix thesis gives the foundational approach: describe builds as functions of explicit dependencies and identify derivations so their results can be reused and composed. Its discussion of purity also recognizes that hashing cannot itself prevent undeclared environmental influence. The lesson is an explicit model with stated assumptions, not a proof that every command launched by Nix is pure. V01 — Dolstra, 2006

Three terms need separate meanings. **Hermeticity** restricts what a computation can observe. **Determinism** says repeated execution under the specified conditions has the same result. **Reproducibility** says another party can recreate the specified artifacts. The Reproducible Builds definition deliberately includes source, environment, and instructions, and compares selected artifacts rather than every incidental log. [V09 — Reproducible Builds definitions](#)

A sandbox can improve hermeticity while leaving scheduling nondeterminism. A deterministic program can reproducibly calculate the wrong answer. A complete source archive can become impossible to rebuild when an essential external input disappears. These are different failure modes and should produce different diagnoses.

The empirical evidence supports Nix's practical value without proving universal purity. Malka, Zacchiroli, and Zimmermann recreated historical build environments and rebuilt 14,452 of 14,461 previously successful jobs from one old revision. The experiment reused cached dependencies; it did not rebuild the complete historical dependency closure from source. It also left bit-for-bit artifact reproducibility outside scope. Its preliminary failure analysis identifies sandbox leakage and flaky tests. A rebuild-success percentage must not be presented as a byte-reproducibility percentage. [V10 — Reproducibility of Build Environments through Space and Time](#)

A concrete limitation of the current digest

The helper's `drvInputs`³⁴ queries direct references of a derivation. `inputsDigest`³⁵ hashes their sorted path strings. Separately, `evalClosure` hashes the derivation file. This is a commitment to Nix's symbolic dependency identity. It is not an independent observation and hash of every byte actually read during execution.

That distinction does not make the design automatically unsound. A dependency graph can be represented transitively by hashes that name other hashed objects. However, correctness then depends on what those names guarantee. Nix distinguishes content-addressed store objects from input-addressed objects; one must not silently equate a recipe-derived path with a measured output-content hash.

V06 — Nix content-addressing documentation

Inference: the reportable claim should be narrower than “the complete observed input closure has this digest.” It is “this runner resolved this derivation and these dependency identities.” An implementation that wants the stronger claim must establish the store's integrity, the addressing modes it permits, the execution restrictions, and the role of substituted dependencies. The output digest helps compare outputs. It does not retroactively establish how an untrusted machine produced them.

The Nix manual documents configurable sandbox paths, exceptions for fixed-output derivation networking, and signature requirements for importing non-content-addressed paths. Those controls demonstrate that the daemon

configuration and cache trust are part of the execution boundary. They cannot be inferred from the word `nix` in a statement. V04 — Nix configuration reference

3.3. The closest intellectual relatives are build systems and selective testing

Build Systems à la Carte separates scheduling from deciding whether existing results remain valid. It also distinguishes dependencies known before execution from dependencies discovered during execution, and traces that validate a result from traces that recover a cached result. This vocabulary is directly useful: the-valley's integrator combines an integration scheduler with a validator of previously recorded computation. Those components can evolve independently. V02 — Mokhov, Mitchell, and Peyton Jones, 2018

The lesson from self-adjusting computation is that incrementality requires dependency information whose collection is itself sound. Acar and colleagues combine recorded dynamic dependencies with memoization to update computations after changes. Their results show substantial speedups on suitable workloads, alongside tracing and storage costs. They do not establish that an arbitrary shell process's reads can be represented cheaply or completely. V03 — Acar et al., 2009

Safe regression-test selection asks a closely related question: which old tests must execute again after a program changes? Rothermel and Harrold prove safety

under explicit conditions, where safety means retaining every previously available test capable of exposing a fault in the modified program. This does not imply that the available test suite detects every fault. That boundary is especially useful for interpreting the-valley's claims about semantic conflicts. V27 — Rothermel and Harrold, 1997

Inference: the-valley can aim for two progressively stronger statements:

1. Every reused result equals the result that its approved check would produce on the candidate landing.
2. The approved collection of checks is adequate for the project's acceptance requirements.

The first is an evidence-reuse property. The second is a software-assurance property. A perfect implementation of the first leaves the second open. Documentation saying semantic interference is “caught by construction” should specify that construction's test coverage and dependency assumptions.

The economic tradeoff is the size of the dependency set

Suppose a spelling check reads all of a repository because its derivation takes the repository root as its source. A change to a Go package now changes the spelling check's input identity, even if the check examines only Markdown. Reverification is conservative and potentially wasteful.

Filtering the source set to Markdown can improve reuse, but only if configuration, dictionaries, includes, and file discovery remain accounted for.

Conversely, suppose a test enumerates a directory and reads whichever configuration files exist. A trace that records only the files opened on the previous run may miss a newly added file. The directory listing, including the prior absence of a name, was an input too. “Track files read” is not yet a complete dependency model.

Bazel's documentation gives practical examples of hermeticity failures, including host tools, timestamps, and writing into the source tree. Its remote cache distinguishes action-result metadata from content-addressed output storage. This is relevant prior art for both disciplined dependency declarations and the separate trust required in the mapping from an action to its claimed output. V07 — Bazel hermeticity, V08 — Bazel remote caching

Recommendation: keep conservative whole-check invalidation as the baseline. Introduce finer source sets only with tests that attempt to change omitted inputs. Compare saved execution against the cost of maintaining those declarations. Precise reuse is valuable when it is demonstrably cheaper than rerunning the check; it is not an end in itself.

3.4. A signed executable hash is not evidence that the executable ran

The verification mechanism stack³⁶ overstates several guarantees. These overstatements matter because they concern the adversarial case that local verification must eventually address.

Consider this failure trace:

1. An attester has an authorized signing key and control of its own machine.
2. It computes the expected tree and derivation identifiers without executing the check.
3. It writes the identifier of the approved attestation tool into a statement.
4. It invents a passing result and signs the statement directly.

No modified-tool hash must appear. The attacker is choosing the statement's bytes. A verifier learns that the key endorsed those bytes, not that the named tool produced them. Local root can likewise suppress local observations. A sandbox controlled by the same adversary is not an independent observer of that adversary.

This is a protocol-level counterexample, not a claim that the existing operator is malicious. The current trust model can be entirely appropriate for an operator's own devices and

agents. The inaccurate part is claiming that these mechanisms force a malicious signer to leave detectable evidence of how it lied.

Hardware-backed remote attestation offers a different trust boundary. The RATS architecture separates an attester's evidence, a verifier's appraisal, reference values, and the relying party's decision. A protected measurement can support a claim about an execution environment when its binding and freshness are established. It still needs an appraisal policy and a trusted measurement chain. V23 — RFC 9334

Recommendation: keep three deployment profiles conceptually distinct: trusted personal attesters; delegated builders whose signing authority is protected from submitted code; and independent witnesses that check selected claims. A project may combine them. A numeric score should never make the first profile appear cryptographically equivalent to the second.

Independent execution also needs an independence definition. Two witnesses using the same compromised compiler or binary cache can agree on a malicious artifact. Thompson's compiler attack is the classic illustration of a toolchain reproducing a compromise even when inspected source no longer contains it. V24 — Reflections on Trusting Trust

The practical requirement is not unlimited diversity at any cost. It is identifying which common failures a second witness is intended to exclude: a dishonest author, a

corrupted machine, a poisoned cache, a compromised compiler, or a flawed test. Each requires a different kind of independence.

3.5. What the supply-chain standards already solve

The supply-chain ecosystem contains several distinct layers. Treating them as interchangeable makes both adoption and criticism less precise.

Prior art	Its useful contribution	Its boundary
in-toto	Authorized steps and continuity between their inputs and outputs	Correctness still depends on the authorized process and trust assumptions
in-toto Statement	A subject digest and a typed claim about it	The predicate defines the claim's meaning
DSSE	A signed encoding that binds payload bytes and payload type	An envelope does not evaluate the payload's truth
SLSA Build track	Requirements for provenance generation and builder isolation	A level is about meeting requirements, not counting security mechanisms
Sigstore	Identity-linked signing and transparent signature history	Consumers still decide acceptable identities and claims

Prior art	Its useful contribution	Its boundary
TUF	Secure distribution of current trusted metadata despite several compromise and replay threats	It does not execute a build or determine whether a test is adequate

The in-toto paper is particularly relevant because it treats the connections between steps as security-critical. A project layout defines authorized functionaries and artifact rules; signed links record their work. This is close to the-valley's desire to compose evidence from multiple tools without giving a single forge ownership of the workflow. [V15 — Torres-Arias et al., 2019](#)

The Statement specification already separates subject from predicate type. Pure verification and effectful observation can therefore be different predicates without abandoning interoperability. The current local statement schema³⁷ has a similar conceptual separation, though its serialized protocol is its own. [V16 — in-toto Statement v1](#)

DSSE authenticates the payload's exact serialized bytes together with its type, using an unambiguous encoding. It does not require all producers to canonicalize an application document into a universal byte representation before verifying the supplied bytes. The-valley's text format can still have independent benefits for readability and deterministic production; those benefits should justify its maintenance cost. [V18 — DSSE protocol](#)

Recommendation: preserve a precise internal claim model and implement an explicit interoperable representation at the boundary. Re-encoding a statement changes its signed bytes. An adapter should either preserve the original signed object or produce a new, clearly attributed statement; it should not pretend signatures survive arbitrary translation.

The SLSA comparison needs correction

The claim of being “roughly SLSA Level 3” is not supported by the mechanism stack. SLSA v1.2 Build L3 requires provenance strongly resistant to forgery by tenants, a trusted control plane, protected signing material, and build isolation. Its hosted requirement excludes builds on an individual's workstation. Reproducibility and periodic sampling do not, by themselves, satisfy those requirements. V17 — SLSA v1.2 build requirements

This is not a reason to imitate a hosted forge. It is a reason to describe the actual trust boundary. The-valley could integrate evidence from a conforming builder, and the specification allows systems involving independent rebuilders as build platforms. Any level claim would still require a concrete mapping of the entire deployment to the requirements. A per-check attestation is also a different object from complete provenance for a released artifact.

Sigstore's use of short-lived certificates and transparency improves identity binding and auditability. Its documentation explicitly assigns monitoring responsibilities to users. It does not turn a signature into proof of successful computation. Whether to use its public services, privately operate

components, or retain local keys is a deployment decision with operational and trust consequences. [V19 — Sigstore security model](#)

3.6. Integration is concurrency control over evidence, with qualifications

Kung and Robinson's optimistic concurrency control separates speculative work from validation and publication. Transactions proceed on local state; validation checks the conditions required for a serial order. Their argument assumes individual transactions preserve the relevant integrity conditions. Mapping this to code integration is fruitful only if the-valley states what its checks establish and what their dependency sets include. [V11 — Kung and Robinson, 1981](#)

The project already has the essential outline. Prepare a candidate against a target snapshot. Determine which results remain applicable. Publish only if the target still has the expected value. A lost compare-and-swap race causes reconsideration. This avoids silently landing a candidate validated against the wrong target.

However, the analogy does not make software meaning transactional. A check may omit an important behavior. Two changes can preserve every tested property while jointly violating an untested one. Crystal's empirical work established that collaboration conflicts include build and test failures beyond overlapping textual edits, and explored speculative analysis for earlier detection. It supports testing

combinations, not the inference that clean textual merges or finite tests establish semantic compatibility. V12 — Brun et al., 2011

Reuse needs a computation identity and an acceptance identity

A useful proposed model has two independent questions:

Computation identity:

approved check definition + resolved inputs + runner semantics + platform

Acceptance decision:

computation result + applicable policy + authorized signer(s)
+ freshness/revocation state + candidate landing

This is a design proposal, not a new wire format. A policy change need not invalidate a genuinely identical computation. It may change whether that result is sufficient. Conversely, preserving a check's display name must not conceal a change to its executable definition.

There are three distinct failure traces worth testing against the implementation:

Candidate-controlled no-op. Policy requires unit-tests; the candidate changes that derivation to create an output immediately. The attestation truthfully reports success. A witness faithfully rebuilds the no-op and agrees. The repository already records this exposure in bd-eaefe82³⁸. Its suggestion that witness re-derivation would eventually

detect a check that computes nothing does not follow: detecting the wrong check requires comparing against an approved definition or semantic expectation.

Same tree, different named computation. The helper accepts a check name mapped to an attribute. The evidence collector extracts the name and result but does not retain the attribute in the verdict's evidence structure. The pure unmoved rule accepts an otherwise admissible passing statement when the subject tree is unchanged, without recomputing the expected derivation. Thus a same-tree shortcut has a stronger precondition than the implementation visibly establishes: the statement must name the computation policy required. This is a static code-review finding, not a reported end-to-end exploit.

Effectful label substitution. The verdict retains an effectful observation's result and timestamp, but not the command or environment identity recorded in its statement. It can therefore decide freshness without itself comparing those fields to the required command. Again, upstream guarantees could change the assessment; the inspected verdict path does not establish that binding. A trustworthy signer can still make a configuration mistake, so this issue is not limited to malicious keys.

Recommendation: establish computation identity before applying any transfer shortcut. Keep it separate from the policy digest in the transfer record. Resolve instance-mandated definitions from an independently authorized source. For changes to checks, run both the incumbent

acceptance check and the proposed check where feasible; authorize the transition explicitly when compatibility is impossible. This permits improvements to tests without letting a proposal silently lower its own bar.

Cheap validation is a hypothesis about the runner

`attest inputs` calls `nix eval` and is described as building nothing. Nix's import-from-derivation feature can pause evaluation to realize a derivation. Its manual explicitly documents disabling this behavior. Therefore evaluation-only latency needs an enforced restriction, rather than a comment in the integrator. V05 — Nix import from derivation

Even with that restriction, evaluation, source materialization, and hashing have costs. A serial controller can bottleneck while doing those operations before its small atomic ref update. The size of the atomic operation and the end-to-end service rate are different measurements. The current closure collector³⁹ runs an `attest inputs` subprocess per pure check, so repeated exports and evaluation are concrete costs to measure.

Merge queues are a baseline and a possible complement

Zuul tests speculative combinations in parallel. Later queued changes include earlier ones; a failed predecessor causes affected work to restart. Its adjustable window limits wasted speculation. GitHub's merge queue likewise validates temporary combinations incorporating the target and

preceding queued work. These are substantial existing solutions to integration throughput, not merely serialized CI gates. [V13 — Zuul project gating](#), [V14 — GitHub merge queues](#)

Hypothesis: a hybrid may improve some mixed workloads. Reuse unchanged per-check evidence first. Speculatively execute only invalidated checks on likely future landings. Reduce speculation when failures or conflicts rise. Reserve an integration opportunity for changes that repeatedly lose races. The hybrid adds scheduling and bookkeeping costs, and failed speculation can waste execution. It should be compared experimentally; there is no basis yet for claiming one queue policy works best for every repository.

3.7. Effectful checks are observations with assumptions

A check against a live API may be useful even though it cannot be replayed exactly. Its result means the observed interaction succeeded under the conditions recorded. A ten-minute acceptance window means the project accepts an observation up to ten minutes old. It does not mean the service remained healthy during those ten minutes.

The repository's path-class rule adds a separate assumption: intervening changes outside the class do not affect the observation's applicability. That assumption can be wrong if a shared configuration file, service dependency, or credential policy sits outside the class. The time bound and the dependency scope should be reviewed independently.

The RATS freshness discussion similarly recognizes that state can change immediately after evidence is produced; freshness is a policy choice about acceptable recentness. V23 — RFC 9334, section 10 That is a useful interpretation for ordinary test observations too, without claiming they are hardware attestations.

The current verdict⁴⁰ rejects observations whose calculated age reaches the validity window. Static inspection shows no rejection of negative age. A future timestamp is consequently a concrete test case. The larger issue is defining acceptable clock skew, which clock supplies time, and whether a trusted receipt or challenge is required. A sender's signature authenticates its stated time; it does not make the clock accurate.

Not every nondeterministic check belongs in the same category as a live API test. Luo and colleagues studied flaky-test fixes and identified multiple causes of unstable outcomes. A race or test-order dependence can exist inside an otherwise isolated environment. Marking the wrapper `nix` does not eliminate that behavior. V26 — Luo et al., 2014

Recommendation: define admissible result types. A deterministic check can have an equality-based result. A statistical benchmark may need a sample size, distribution summary, and acceptance rule. A live-service observation needs environment and time context. A flaky deterministic check should trigger investigation of its purity claim, rather than acquire an arbitrary expiration that conceals the defect.

3.8. Transparency records claims; revocation changes what may rely on them

A transparency log can make a submitted statement discoverable and make alterations detectable. It does not determine whether the statement is true. Certificate Transparency also distinguishes inclusion from consistent views of history; a dishonest log can present conflicting histories unless other participants compare or witness checkpoints. Monitors must look for the failures that matter. V21 — RFC 9162, security considerations

Tessera is a plausible implementation component, not a complete operational policy. Its documented lifecycle distinguishes sequencing an entry, integrating it into the tree, and publishing a signed checkpoint. Its publishing machinery supports witness policies. The-valley would still need to choose which stage permits integration and what happens when publication or witnesses are unavailable. V22 — Tessera documentation

There is a related crash boundary in the existing integrator. `land`⁴¹ moves the protected ref before storing the transfer evidence. It reports subsequent failures, but a crash can still leave a landed change without the intended durable record. Composing and signing before the ref move reduces one failure class; it does not atomically publish evidence alongside the landing.

Recommendation: investigate a Git ref transaction covering the target and evidence refs, or a durable intent record with recovery. For an external log, specify a two-stage protocol with a recoverable intermediate state. The experiment is a process kill at every persistence boundary, followed by a new controller reconstructing what happened. “Every landing has evidence” should be tested as a storage invariant.

Revocation presents another distinction. An old statement can remain authentic while becoming unacceptable. A signing key may be revoked; a check definition may be disallowed; a vulnerability database may advance; or a policy may require a stronger builder. TUF's role separation, version checks, expiration, and key-update procedures are important prior art for distributing current trust information and resisting stale metadata. V20 — TUF specification

Inference: a useful revocation record names the affected authority or statement, the reason, the effective interval, and the dependent decisions requiring reconsideration. A reverse index from attestations to transfers, releases, and deployments becomes operationally important. Pure computation results need not expire merely because time passes, but permission to rely on them can change.

Revocation also does not undo consequences. An already deployed secret leak cannot be reversed by withdrawing an attestation. The architecture's tradeoff therefore needs an explicit exposure budget: which actions may proceed before independent verification, how far their outputs may propagate, and which actions require verification first.

3.9. What random witness sampling can and cannot promise

Sampling can make repeated dishonesty increasingly likely to be discovered. It cannot guarantee discovery of a particular unsampled statement. The following calculation is this report's elementary probability model, not an empirical estimate of the-valley's security.

Suppose each false statement is independently sampled with probability p . Suppose a sampled false statement is always detected by an honest witness, and the attacker cannot choose its behavior after learning the sample decision. After m false statements:

$$P(\text{no detection}) = (1 - p)^m$$

$$P(\text{at least one detection}) = 1 - (1 - p)^m$$

At $p = 0.01$, one false statement has a 99% chance of escaping this sampling scheme. One hundred false statements have about a 36.6% chance of all escaping. Reaching a 95% probability of detecting at least one requires 299 false statements. The expected number of false statements up to and including first detection is 100; that expectation is not a maximum.

If an audit detects a sampled false statement with probability q , independent of selection, replace p by $p \cdot q$. A witness that reruns a candidate-controlled no-op has $q = 0$ for the defect “this check does not test what policy intended.” Increasing sampling does not fix that defect.

These equations also exclude correlated witness failures, compromised samplers, adaptive attacks, and delayed audit execution. A high historical success score gives no mathematical assurance against a signer that behaves honestly until one valuable attack. Multiple new identities can undermine reputation unless identity creation itself has a controlled cost or authorization boundary.

Recommendation: select audits after the statement is irrevocably committed, using randomness the producer cannot predict or steer cheaply. Separate two aims: estimating routine error rates and deterring targeted abuse. Random sampling serves the first well under suitable assumptions. The second may require mandatory independent verification for high-impact changes, protected builder keys, or limits on what unverified outputs can authorize.

A confidence statement also needs a denominator. If no failures appear in n independent, representative audits, a one-sided 95% upper bound for a stable defect rate is $1 - 0.05^{(1/n)}$, approximately $3/n$ for large n . With 300 clean audits, this is about 1%. It says nothing comparable about an adaptive adversary's next submission. Calling it an attester's universal trust probability would be misleading.

Finally, sampling without a response mechanism only produces observations. Useful operation requires a disagreement state, preservation of both runs, diagnosis of nondeterminism, and a rule for pausing dependent

decisions. Automatic punishment on first mismatch risks treating an honest witness that exposed a flaky check as the attacker.

3.10. A research program that can discriminate between designs

The most useful next work is a small set of falsifiable experiments. Each should compare a concrete baseline, measure costs, and exercise a failure mode that normal successful builds would miss.

Reuse on real history

Baseline and variation: Recheck everything; whole-tree cache; current per-check transfer; narrower source sets

What would change the design decision?: Saved compute and landing latency justify declaration complexity without changed results

Controlled interference

Baseline and variation: Changes to imports, generated files, directory membership, lockfiles, platform, and check definitions

What would change the design decision?: Any false reuse requires repairing the dependency or identity model before optimizing further

Same-tree evidence substitution

Baseline and variation: Keep subject and check name; alter attribute, command, environment, or approved definition

What would change the design decision?: Every mismatch is rejected before the unchanged-tree shortcut

Local forgery

Baseline and variation: An authorized key signs a fabricated passing statement and the approved tool identifier

What would change the design decision?: Demonstrates exactly which deployment profiles need a separate execution authority

Genuine witness independence

Baseline and variation: Rebuild with a fresh store; shared cache; separate cache; altered environment

What would change the design decision?: Distinguishes actual independent execution from repeated trust in the same cached artifact

Purity stress

Baseline and variation: Vary clock, CPU count, scheduling, locale, kernel, and allowed sandbox settings

What would change the design decision?: Determines which checks qualify for deterministic reuse and what platform identity must include

Evaluation cost

Baseline and variation: Warm/cold caches, import-from-derivation, large flakes, many required checks

What would change the design decision?: Establishes whether the integrator must cache evaluation, parallelize preparation, or restrict runner features

Queue contention

Baseline and variation: Replay identical arrivals through sequential, speculative, reuse-only, and hybrid queues

What would change the design decision?: Compare completed landings, wasted computation, tail latency, and starvation rather than only average throughput

Evidence durability

Baseline and variation: Crash before and after each ref, evidence, memo, and log publication operation

What would change the design decision?: Recovery restores a complete account without silently reclassifying an incomplete landing

Revocation response

Baseline and variation: Revoke a signer or definition after several dependent integrations and releases

What would change the design decision?: Every affected dependency is found, and consumers converge within a measured interval

Hypothesis: dependency breadth and evaluation overhead will decide the value of selective reuse more often than signature-verification cost. A source filter that inadvertently includes the entire repository may erase most of the intended advantage. This should be established by historical replay before designing elaborate attester economics.

Hypothesis: checks that use many global fixtures will benefit more from speculation and batching than from finer evidence transfer. Conversely, isolated documentation and component checks should transfer frequently. A single aggregate cache-hit rate could hide this split; results should be reported per check family and by computation cost.

One experiment must address the meaning of “passed.” The oracle literature studies how tests obtain an expected answer, including specifications, contracts, derived oracles, and metamorphic relations. It shows why automating execution does not finish automating judgment. V25 — Barr et al., 2015

For the-valley, mutate both code and its candidate-supplied tests. Compare incumbent tests, candidate tests, external invariants, and independent review. The aim is not to assign one universal quality score. It is to find changes for which execution evidence looks impeccable while the intended requirement is no longer being tested. This is especially relevant when the same agent can write the implementation, test, and account of success.

3.11. The promising solution space

The strongest version of this architecture makes evidence portable while keeping acceptance contextual. Execution produces a claim with a precise computation identity. Integration checks whether that computation still applies and whether its evidence is currently acceptable. Release or deployment can impose stronger requirements than integration without discarding the earlier evidence.

Several design choices remain legitimate:

- A personal installation can trust local machine keys and use occasional independent checks to catch mistakes.
- A collaborative project can accept local results for fast feedback while requiring a protected builder for selected acceptance predicates.
- A high-assurance release can require several independent rebuilds, durable transparency evidence, and explicit authorization of check-definition changes.

- A low-conflict project can use a simple queue and rely on build-cache reuse, avoiding a more sophisticated integrator until measurements justify one.

The strongest objection is that existing build caches and merge queues may already capture most of the performance benefit. The-valley must demonstrate an additional benefit in portable authority, partial re-attestation, host independence, or explanatory records. Moving familiar mechanisms into new formats without improving one of those outcomes would not justify their operational cost.

The corresponding opportunity is substantial. A transfer statement can explain exactly why earlier evidence still supports a new landing. That is more informative than a historical green badge detached from its assumptions. The research recommendation is to pursue that explanation as the core contribution, with conservative identity binding and explicit trust profiles. Reputation, universal purity claims, and mandatory transparency infrastructure should follow only where their measured benefit and threat model warrant them.

4. Events, trust, and durability

the-valley is trying to make a project's history sufficient to explain and govern its operation. That is a stronger ambition than keeping source code in Git. It reaches into distributed systems, workflow recovery, authorization, and the conditions under which a record deserves to be called evidence. The most useful distinction is between **remembering a decision, making its consequences happen, and knowing what those consequences were.** Each needs a separate contract, even when all three use the same infrastructure.

This chapter's central judgment is that the project's direction is sound, but its strongest language about replay and transparency currently exceeds the guarantees visible in the implementation. The next step should be to specify exactly which facts survive failure and which actions can be retried. That work would make the existing small tools more dependable without requiring a general workflow platform.

The analysis uses repository revision 5bb3a7736fe49b0cc4914299b142c0c9bafb6870 and sources checked on 2026-09-20. “Implemented” below means visible in this checkout, not independently verified on the deployed host. Code inspection supports the implementation findings;

the proposed failure experiments have not been executed. The annotated source register records reading depth and limitations for D01-D28.

4.1. What exists, and what remains an ambition

The requirements⁴² ask for durable project state, agent attribution, bounded and revocable trust, and a single history from which consequences can be explained. The architecture⁴³ chooses Git, a replaceable event bus, independent controllers, and a governed identity registry. These choices already produce useful machinery, but they do not yet form the complete system described by the later scenarios.

Event publication

Evidence in this checkout: A detached post-receive hook publishes ref changes; the integrator publishes outcomes.

Larger claim still requiring work: Every historically important transition can be recovered after bus loss.

Integration

Evidence in this checkout: A polling controller reads durable request refs and compares evidence before updating a protected ref.

Larger claim still requiring work: A landing and its durable decision record survive every interruption as one complete result.

Replay

Evidence in this checkout: valley replay enumerates current refs and publishes synthetic creation events.

Larger claim still requiring work: Replay reproduces the original sequence of updates, deletions, judgments, and external effects.

Identity

Evidence in this checkout: A CUE registry compiles into SSH authorized keys and attestation verifier keys.

Larger claim still requiring work: Every capability, delegation, and recovery action is checked against current governed authority.

Governance

Evidence in this checkout: Policy and registry changes are described as amendments judged under earlier rules.

Larger claim still requiring work: Runtime behavior, privileged recovery, and external outputs are covered by the proposed self-transparency invariant.

The relevant implementation is in the host module⁴⁴, the CLI⁴⁵, the integrator⁴⁶, and the identity compiler⁴⁷. Some prose lags the code: the integrator's help still says no registry compiler exists, although the compiler is present. Conversely, a decision document's settled status does not demonstrate that every enforcement boundary it describes has shipped.

This distinction matters because architectural simplification can hide obligations. Saying that the bus is replaceable is safe only when every irreplaceable fact has another durable home. Saying that identity is declared in a repository is safe only when running gates check the declaration, or a clearly identified compilation of it. The design should be judged at these boundaries.

4.2. A Git repository contains several different histories

State reconstruction is weaker than event reconstruction

An event-sourced system retains the events needed to reconstruct application state. Fowler's account also identifies two replay hazards: repeating external updates and substituting present-day external query results for historical ones. Event sourcing therefore requires a replay policy for effects and observations, not just a stored sequence. [D02: Event Sourcing](#)

Git can carry an event store, but its ordinary commit graph does not automatically record every action taken against a repository. A commit says what tree it names and which commits precede it. A branch ref says which object it currently names. Those are different facts from “this ref moved from A to B in this accepted push.” The same final refs can result from several different sequences of pushes.

The current event schema⁴⁸ acknowledges this directly: replay sets old to the all-zero object ID because a ref’s previous value is not derivable from the current repository. The replay command⁴⁹ runs `git for-each-ref` and publishes one ref-updated event per surviving ref. A deleted ref has no row to enumerate. A branch that moved A→B→C is presented as a creation at C.

This gives a concrete counterexample to the strongest replay claim. Create a temporary branch, update it twice, and delete it. Erase the bus. The final ref inventory cannot tell a fresh subscriber that any of those operations occurred. A clone containing the same current objects and refs cannot recover the missing transitions merely by inspecting commit ancestry. Some objects may eventually disappear if nothing retains them.

Reflogs are useful recovery aids, but their standard contract is different from a permanent domain ledger. Git documents ref-update logs with expiration controls, including default expiration periods for ordinary and unreachable entries.

Making a reflog the authoritative event history would therefore require explicit retention, replication, and restore rules. [D03: git-reflog](#)

There are three separate promises the project could make:

1. **Reconstruct present state.** Rebuild indexes and let controllers converge from durable current facts.
2. **Reconstruct decision history.** Recover accepted and rejected decisions with the evidence and policy used at the time.
3. **Recover unfinished effects.** Determine which authorized actions remain incomplete or uncertain after failure.

The existing ref replay primarily serves the first promise. The S4 “why did X occur” requirement needs the second. Reliable deployment and notification need the third. None follows from the others.

The landing has a recording gap

The integrator already treats the target-ref update as a commit point. In `land`⁵⁰, it prepares signed evidence, moves the target ref using the expected previous object ID, stores evidence refs, deletes the request, and publishes the outcome. It attempts the later steps even when an earlier post-commit step fails. That is sensible error handling within a running process.

A process can also disappear between those steps. countersign⁵¹ writes evidence blobs before the target moves, but the refs that make those blobs discoverable are installed afterwards. A crash immediately after the target update can therefore leave the accepted commit without its ordinary evidence references. The objects may still exist, but an ordinary consumer has no complete durable landing record to follow. This is a crash-consistency concern from inspection, not a claim of an observed production incident.

Git offers transactions over multiple ref updates through `update-ref --stdin`. Its documentation also warns that a concurrent reader may observe only some modifications even when the transaction's updates were accepted together. That makes multi-ref transactions a candidate tool, with reader and crash behavior to test against the selected ref backend. They are not a blanket claim of database-style snapshot isolation. [D04: git-update-ref](#)

Recommendation. Make one immutable landing record the durable explanation of a transition. It should name the previous and accepted heads, request identity, evidence, policy snapshot, and intended downstream work. Protect the relationship between that record and the target update. A transaction, a recovery journal, or one authoritative record from which the target is derived are alternatives to evaluate. Bus publication then becomes a retryable projection of an already recorded fact.

The objection is extra machinery around a small tool. The answer is to keep the record narrow. The system need not remember every internal computation. It must remember enough that a restarted controller does not have to guess whether an accepted change was authorized or which consequences remain due.

4.3. Causality is not the order of entries in a log

Lamport's happened-before relation orders events connected by process order and message transmission. It is a partial order: independent events may be concurrent. Logical timestamps can extend that relation into a total order, but an earlier logical timestamp does not establish that one event caused another. A total ordering chooses a sequence compatible with known dependencies; it does not discover the dependencies. D01: Time, Clocks, and the Ordering of Events in a Distributed System

For the-valley, an integration event followed by a deployment event in a bus is therefore insufficient explanation. The deployment might concern a different commit. It might have been manually requested. A restarted subscriber might publish old work after newer work. The record needs explicit links: which accepted artifact the controller observed, which policy authorized the action, and which request identified the intended effect.

“One history” can mean one queryable body of linked evidence without demanding one global sequence across all valleys. Per-project ordering remains useful at the protected ref. Cross-project causality can be represented by references to particular decisions or artifact versions. A single cross-valley total order would add coordination without answering the semantic question of why a change mattered.

W3C PROV distinguishes production relationships from attribution and delegation. Its delegation relation describes responsibility; it does not enforce a permission. D14: PROV-DM The knowledge chapter develops the vocabulary. Here the practical lesson is to record authorization separately from a claim about who acted.

OpenTelemetry addresses a neighboring problem. Its tracing API supplies trace and span identifiers, parent relationships, and links across traces; links are especially useful when one operation has several predecessors. The API also exposes recording and sampling distinctions. These are good correlation mechanisms, but the tracing specification is not a promise to retain every domain decision permanently or authenticate its truth. D15: OpenTelemetry Tracing API

A practical division follows. Domain records should carry stable causal references even if telemetry expires. Traces should point to those records and supply operational detail while retained. The existing three signal contracts⁵² already separate durable events, lossy metrics, and logs retained under policy. The difficult part is deciding which observation must be promoted before the raw data disappears.

Provenance does not establish incident blame

A deployment record may prove which artifact ran and who authorized it. It does not prove that this artifact caused an outage. A database failure, traffic change, or independent configuration update may have contributed. Halpern and Pearl formalize actual causation using structural models and counterfactual interventions. Their analysis makes the chosen variables and model assumptions part of the answer. A dependency graph alone is not that model. D28: Causes and Explanations, Part I

The project's incident memory should preserve this distinction. A useful report would separate observed facts, candidate explanations, discriminating tests, and the current assessment. “Rollback restored service” is evidence. It can remain inconclusive if rollback also restarted a dependency or changed traffic. Automated attribution should preserve the observations supporting a conclusion and the alternatives that remain plausible. A confidence number without that structure would make the record look more scientific than it is.

4.4. Controllers and workflows solve different recovery problems

Kubernetes controllers repeatedly compare desired and observed state and request changes that reduce the difference. They can act through the API server or on external systems, and they report observed state for other controllers to use. This is a strong precedent for durable

request refs plus repeated reconciliation: a missed notification should delay inspection rather than erase an obligation. D08: Kubernetes Controllers

The implemented integrator takes this approach. Its polling loop⁵³ works without a bus subscription. The production-DAG idea⁵⁴ makes the same move for future work: events invalidate observations, while durable demand determines what should progress. This is a better foundation for the outcome engine than encoding obligations only in chains of callbacks.

Temporal retains an event history for each workflow execution so execution can recover after a crash. The history records commands and externally reported results, and it has explicit size and event-count limits. This is a mature example of preserving the progress of a particular operation, rather than inferring all progress from current desired state. D09: Temporal Events and Event History

Temporal's activity lifecycle also makes an essential limit visible. A worker can disappear after starting an activity; a timeout can trigger a retry. Cancellation is cooperative and may be ignored by activity code. A workflow engine consequently does not remove the need to reason about actions that may already have occurred outside the engine. D10: Temporal Activity Execution

For this project, the useful comparison is by task:

Task	State that must survive	Natural starting point
Keep a service on an accepted artifact	Desired artifact and current observation	Reconciliation
Land a pending change	Request, evidence, judgment, target state	Reconciliation with a durable commit record
Execute a deployment with an approval and timeout	Progress, approval scope, deadline, effect results	Explicit operation state or durable workflow
Run an agent until an outcome is satisfied	Demand, attempts, budget, reservations, evaluations	Reconciliation around recorded attempts
Send a notification once for a decision	Effect identity and delivery result	Durable effect record plus recipient-aware retry

These are recommendations, not requirements to adopt Kubernetes or Temporal. A small Git-backed state machine may be sufficient at the project's intended scale. However, once it implements durable timers, retries, cancellation, compensation, operation versioning, and recovery, the project is implementing workflow machinery even if there is no workflow configuration file.

Opinion. Keep the bus a transport and keep standing obligations in durable state. Introduce a workflow engine only when a real operation requires enough sequential recovery logic to justify it. Compare a small custom controller and Temporal on one deployment experiment.

Count operational dependencies, understandable failure states, and recovery behavior as well as lines of application code.

The outcome engine needs an additional distinction: the dependency graph says what could be attempted; it does not say which attempt owns a budget or may publish a result. Attempts need durable ownership, expiry, and acceptance rules. Otherwise two controllers can satisfy the same apparent demand twice while both behave correctly against stale observations.

4.5. Reliable messages do not make external effects atomic

JetStream streams define retention through limits and policies. They may discard older messages or refuse new ones when limits are reached. The official documentation distinguishes ordinary unacknowledged publication from JetStream publication acknowledged after storage.

“Persistent stream” is therefore an incomplete durability specification without the publication mode, retention configuration, and recovery source. [D05: NATS Streams](#)

JetStream's documented exactly-once mechanisms combine publication deduplication with acknowledgement confirmation. The standard duplicate window is finite, and deduplication uses the message ID rather than payload equality. A successful double acknowledgement prevents redelivery caused by a lost acknowledgement. These are

useful transport guarantees; they do not put an arbitrary deployment API in the same transaction as the consumer acknowledgement. D06: JetStream Model Deep Dive

Consumers separately track delivery and acknowledgements and can redeliver unacknowledged messages. JetStream exposes pull and push consumers for different processing patterns. Consumer state should consequently be treated as part of an operational recovery plan, even when the underlying domain facts can be re-created elsewhere. D07: NATS Consumers

The current host module sets a bounded file store and creates a stream for valley.> using CLI defaults. Its publishers call ordinary nats pub; the inspected calls do not request a JetStream storage acknowledgement. Publication errors are logged and nonfatal. These are understandable choices for observational notifications. They do not yet establish durable delivery of an irreplaceable effect request. The broad subject wildcard also needs review when the planned lossy metrics namespace is introduced.

The classical publication gap occurs when one operation changes durable state and a separate operation sends a message. A transactional outbox records the state change and message together, then lets a relay publish the message repeatedly until acknowledged. Microsoft's implementation example makes that local atomic boundary explicit and uses idempotent consumers for duplicate delivery. D11: Transactional Outbox

The Git equivalent need not be a database table. A durable landing record can contain the fact that needs publishing. The relay's cursor is disposable only if the retained records support re-enumeration and consumers tolerate repetition. The word “outbox” names the atomicity requirement, not a mandatory storage technology.

The next gap is harder. Consider this sequence:

1. An actuator reads an authorized deployment request.
2. The remote platform accepts the deployment.
3. The actuator crashes before recording the response.
4. Recovery sees a request without a completion record.

Retrying can create a duplicate. Declining to retry can abandon a request that never reached the remote service. A local log cannot tell these cases apart. The external system must support a stable operation key, a way to query the original operation, or a compensation whose meaning is acceptable to the project. Otherwise the honest state is “outcome unknown,” with a defined resolution procedure.

Helland's treatment of independent transaction scopes puts duplicate handling in application semantics: an operation is idempotent when repetitions do not repeat the substantive change. The definition depends on what the application regards as substantive. Repeated observation and repeated charging are plainly different. D13: Life beyond Distributed Transactions

Sagas supply another option: divide a long operation into committed steps with compensating actions. Compensation restores an acceptable business condition rather than erasing history. For the-valley, reverting a release can compensate a deployment, but it cannot make an already delivered notification unseen or undo every external consequence of bad code. D12: Sagas

Recommendation. The proposed actuator⁵⁵ should own authorization and an effect ledger. Each effect should have a stable identity, target, approved arguments, deduplication contract, result, and recovery policy. Separate a logical effect from its attempts. Rebuilding a view must never implicitly issue historical effects again.

4.6. Durability needs a recoverable set of objects

The durability requirement⁵⁶ is unusually concrete: integrated work should reach independent copies, including an offsite copy, within minutes, and a copy counts only after a verified restore. That is a stronger starting point than merely configuring backup software. The remaining ambiguity is what must be restored together.

A useful recovery inventory includes source commits, knowledge nodes, policy history, registry history, attestation refs, requests, decision records, and any effect results that cannot be recovered from the target system. It also includes

the material necessary to interpret and verify those records. A clean source checkout is insufficient if it omits the evidence namespace that justified its accepted state.

Git's ordinary clone and mirror modes have different ref coverage. `--mirror` maps all refs and configures later updates to overwrite those mirrored refs. This matters for custom namespaces such as attestations and integration requests. An export or backup test must specify which namespaces it includes rather than assuming that “clone” means all project state. [D26: git-clone](#)

There are two recovery problems: losing bytes and losing authority. A backup may retain all commits but restore an obsolete registry that re-enables a revoked key. A correct current registry may be useless if the sole authorized recovery credential was lost with the host. These cases call for separate restore checks: content completeness, trustworthy policy continuity, and a safe way to establish the recovered instance's current authority.

Replication also has a different failure model from backup. Raft maintains an agreed replicated log through leader election and majority-based commitment under its stated assumptions. Its safety argument is about agreement despite failures, not protection from every authorized destructive command or compromised administrator. [D27: In Search of an Understandable Consensus Algorithm](#)

For the-valley, several copies on one host do little for host loss. Several continuously synchronized copies can faithfully propagate an unwanted deletion. A recovery copy that

retains earlier state serves a different purpose from a live replica. These are consequences of the project's failure scenarios, not reasons to operate a consensus cluster immediately.

Recommendation. Name three observable states for an accepted change: locally committed, independently replicated, and included in a verified recoverable snapshot. This need not put a remote round trip on the integration path. It makes the allowed exposure window visible. The requirement's phrase “integrated means replicated ... within minutes” otherwise mixes an immediate status with an eventual obligation.

A restore exercise should recover one complete accepted change, its author evidence, the integrator's decision, the policy used, and one downstream effect record. It should demonstrate that an old revoked credential cannot regain access silently. That experiment would validate much more than a successful `git fsck`.

4.7. Identity, authority, and evidence should stay distinct

The identity registry decision⁵⁷ contains a valuable rule: a permission belongs in the schema only when an enforcement boundary checks it. This resists a common failure in authorization design, where richly named roles express intentions that no running component actually enforces.

Still, four questions must remain separate. Which key authenticated the request? Which principal does current policy associate with that key? What may that principal do here? What evidence supports the claims it makes? A machine can authenticate correctly and still supply a false check result. An honest machine can produce valid evidence while lacking permission to modify a protected policy.

Saltzer and Schroeder's complete-mediation principle requires authority checks across access paths, including initialization, recovery, and maintenance. Their least-privilege and separation-of-privilege principles also explain why a signing service and an unrestricted agent process should not automatically share authority. The relevance is structural: a policy file cannot constrain an operation that bypasses its enforcement point. D20: The Protection of Information in Computer Systems

Delegation must narrow what can be done

The proposed delegation model⁵⁸ lets agent runs act under a principal without receiving persistent registry entries. Each delegation narrows authority, and human approval is non-delegable. That avoids treating an ephemeral run as a durable account. It leaves the enforcement question open: who verifies the claimed chain, and who prevents a run from using credentials outside it?

Macaroons are important prior art for attenuating authority by adding caveats. Their chained authentication construction supports restrictions and third-party discharge conditions. They also discuss expiry and state-based revocation. Their

ordinary bearer-credential model is a poor direct match for the registry's rejection of long-lived bearer secrets, although the paper discusses holder-of-key restrictions. Study the attenuation semantics before choosing the credential format.

D21: Macaroons

A minimal dispatch authorization could name the project, allowed verbs, input or target scope, expiry, resource budget, parent authorization, and recipient enforcement boundary. A child authorization would be accepted only if its rights are a subset of the parent's rights. The receiver would check the entire relevant chain and current revocation policy. These fields are a proposed experiment, not a settled format.

The most significant choice is whether the run ever receives reusable credentials. If every privileged action passes through an actuator, the actuator can verify scope and sign receipts. If the run has a general shell on a host with ambient credentials, a carefully signed delegation chain describes intended authority while the actual process can do more. The boundary determines the guarantee.

Workload identity is a useful implementation option

SPIFFE defines workload identities and short-lived identity documents, with a Workload API for obtaining them across heterogeneous environments. It provides a basis for mutual authentication. It does not decide that an authenticated

workload may approve a registry amendment or deploy a particular artifact. Those remain authorization decisions.

D22: SPIFFE Overview

SPIRE implements this model with a server, node agents, registration entries, and node/workload attestation. It can issue identities based on configured selectors and conditions. This offers concrete machinery for replacing repeatedly provisioned raw machine keys, at the cost of another service and another set of attestation assumptions.

D23: SPIRE Concepts

There is no necessary conflict with a repository as the governance source. In a possible deployment, the registry would determine permitted workload identities and compile registration policy; SPIRE would perform issuance. The registry would still govern authority. At the current scale, that may be more machinery than the problem warrants. The decision should turn on measured credential churn and revocation needs, not the appeal of a standardized name.

Expiry is only as strong as its enforcement

The actual identity renderer⁵⁹ omits expired principals when it successfully compiles. The host deliberately keeps manually declared SSH keys as a recovery path. A failed compilation preserves the last good files; it does not cause already installed raw keys to expire at the SSH server. Thus “expires on this date” currently means removal after a successful convergence, subject to separate recovery access.

That tradeoff favors avoiding accidental lockout. It should be stated as such. If convergence stops, the effective revocation delay can exceed the timer interval indefinitely. A stronger policy could use short-lived certificates, enforce expiry at the receiving boundary, or reject sensitive operations when the compiled policy is too old. Those alternatives have different availability consequences.

The claim that access is purely a function of integrated repository bytes also needs a time input once expiry exists. The more precise model is that access depends on the selected governance state, the evaluation time, and the enforcement boundary's installed state. Recording those values would make stale authorization diagnosable.

Human presence and agent attribution need precise claims

WebAuthn distinguishes user presence from user verification and distinguishes backup-eligible multi-device credentials from single-device credentials. A generic passkey is therefore not evidence that its private key can never leave one hardware device. An approval protocol must select and check the properties it requires and bind the assertion to the intended statement and relying party. Presence or verification alone does not establish that the person understood and approved that statement. The approval interface and transaction binding remain part of the trust boundary. D25: Web Authentication Level 3

Similarly, the agent-provenance proposal⁶⁰ is a useful record of inputs, but identical prompts and model identifiers can produce different executions. A run needs an occurrence identity as well as input provenance. Two identical experiments are still two observations. A host signature establishes who asserted the provenance; it cannot by itself establish that an untrusted harness recorded everything accurately.

Opinion. Keep persistent principals few, but give every run and every privileged effect a distinct durable identity. Keep the signer, the claimed producer, the delegating authority, and the verifier as separate fields. Collapsing them makes records simpler to print and harder to trust.

4.8. Recursive governance needs a declared trust boundary

The self-transparency draft⁶¹ proposes that no actor durably changes the system or its outputs without transparency. The architecture has since added a partial answer to the bootstrap question: rules at tip N judge the transition to tip N+1. That temporal induction is coherent. It prevents a proposed amendment from authorizing itself merely by declaring more permissive rules in its own output.

The Update Framework supplies a concrete succession precedent: a root update requires both old and new signing thresholds. The client starts with trusted root metadata; it does not manufacture initial trust. D24: The Update Framework Specification, §5

The unresolved part is broader than succession. A root user can replace an executable, edit a bare repository, change authorized keys, or restore an earlier disk image. An external service can accept a credential outside the integrator's path. Versioning the intended configuration does not make these paths disappear. The architecture already recognizes recovery below the machinery; the invariant must explain how that recovery relates to transparency.

Three interpretations are available:

Interpretation	Meaning	What would establish it
Governed normal operation	Supported changes go through recorded policy decisions.	Enforced gates and a documented privileged recovery path.
Detectable deviation	Out-of-band durable changes can be discovered after they occur.	Independent observation, retained baselines, and evidence of comparison.
Prevented deviation within a boundary	No actor inside a specified authority boundary can bypass recording.	A complete mediator with a stated trusted computing base.

These interpretations have different costs. The first fits the solo-operator scenario. The second is valuable when a compromised controller might hide a change. The third requires a much sharper account of which host, kernel, keys, and external services are trusted. A universal statement

covering the operator's root access and all external outputs would be difficult to defend within the project's explicit non-goals.

Recommendation. State the initial invariant over named enforcement boundaries. Record exceptional recovery as a separate signed transition when normal operation resumes. Preserve evidence that an exception occurred, even when it was necessary. Distinguish an authorized exceptional action from an unobserved action; neither should silently become an ordinary integration.

This still leaves a research question: what evidence can survive the compromise of the system that ordinarily records evidence? An independent witness or retained checkpoint can constrain history rewriting. It cannot demonstrate every unobserved host action. The strongest useful guarantee may be that accepted project states have an externally checkable chain of authority, while runtime drift is separately monitored.

Self-modification also has an epistemic limit. An old policy can authorize a new policy correctly while the new policy is a poor decision. Governance proves the route by which rules changed. It does not prove that every permitted amendment preserves the project's goals. The knowledge layer's rationale and evaluations remain necessary.

4.9. Federation should exchange evidence while preserving local authority

Local-first software treats users' local data as primary and seeks useful operation without a provider's continued permission. The original paper connects offline work, collaboration, ownership, privacy, and long-term preservation, while treating CRDTs as one promising technical ingredient. This is a close philosophical fit for the valley's insistence that project knowledge travels with the repository. D16: Local-First Software

CRDTs offer convergence when replicas receive the same updates under specified assumptions. The foundational work gives sufficient conditions involving monotonic state merges or commuting operations. That is useful for some collaborative metadata. It does not by itself decide whether concurrent approvals satisfy a governance rule or whether two individually valid changes jointly violate an application invariant. D17: Conflict-free Replicated Data Types

For example, an append-only set of signed review comments is a reasonable candidate for independent replication. “The release is approved by the current governing set” is a derived judgment that depends on a particular policy state. Automatically merging both sides of a concurrent membership change may preserve bytes while producing an unauthorized interpretation. The project should identify which data can merge freely and which decisions require an owning authority.

Radicle provides an especially relevant detail beyond the systems comparison: a repository identifier anchors the history of its identity document. D18: Radicle Protocol Guide This deserves comparison with the-valley's genesis and subsequent policy amendments.

The research opportunity for the-valley is more specific: combine governed integration, transferable verification evidence, durable project knowledge, and recoverable effects. Radicle deserves an implementation-level comparison before inventing a federation format. Whether its identity and collaborative-object rules can carry the-valley's policies remains to be tested; similarity is not compatibility.

Fossil's integrated project facilities provide another comparison; their scope is covered in the systems chapter. Its own account favors compact deployment. D19: Fossil Versus Git

Fossil challenges the assumption that minimalism always means many independently replaceable tools. A single program can be easier to operate than several services whose failure contracts must be composed. the-valley's modularity can still be valuable, especially for verification portability. The comparison should count the total recovery and maintenance burden, not just the size of each component.

A useful first federation experiment would send a proposed change, its evidence, and its approval claims from valley A to valley B. B would verify the material under B's own policy

and return an explicit acceptance or refusal. A statement that A trusts a signer would not automatically become B's authorization. Imported history would retain its origin and causal references.

Cross-repository release atomicity should be deferred until its meaning is explicit. Two refs moving together is one possible requirement. Ensuring all deployed consumers remain compatible is another, harder requirement. A staged schema change with a compatibility interval may serve the scenario better than a distributed commit protocol. The experiment should begin with the user-visible invariant, then choose coordination machinery.

4.10. Experiments that would resolve the important uncertainties

The following experiments are proposed research work. They are deliberately small enough to produce decisive evidence before a broad architecture commitment.

History versus snapshot replay

Procedure: Create, update, force-move, and delete disposable refs; record notifications; erase only the bus; rebuild.

What would count as success: The report names exactly which facts return and which do not. Any promised historical facts have a demonstrated durable source.

Interrupted landing

Procedure: Kill a disposable integrator at each step around target update, evidence storage, request deletion, and publication.

What would count as success: Restart yields one accepted transition with discoverable evidence and no indefinitely stranded request.

Duplicate external effect

Procedure: Use a fake target that accepts an operation and then drops the response; restart the actuator repeatedly.

What would count as success: One logical effect occurs, or the operation becomes explicitly unknown with a safe recovery procedure.

Revocation during failed convergence

Procedure: Expire a disposable principal while registry compilation fails; test each receiving boundary.

What would count as success: Observed access matches a declared maximum staleness policy, including the recovery path.

Complete restore

Procedure: Restore onto a fresh disposable host from an offsite copy without the original machine.

What would count as success: One accepted change, its evidence, applicable policy, and downstream result remain verifiable; revoked authority is not silently revived.

Federation without shared trust

Procedure: Exchange one contribution between two isolated valleys with different signer policies.

What would count as success: Transport success never substitutes for local authorization, and refusal explains the unsatisfied policy.

Two further comparisons would sharpen the solution choice. Implement one interrupted deployment in both a small controller and Temporal, then compare the amount of durable state and recovery logic each requires. Export a representative project's knowledge and approval records through Radicle and Fossil-shaped models, then list what remains awkward or unrepresentable. These are evaluations of fit, not proposals to migrate the project immediately.

The most promising direction is a small number of durable, independently interpretable records: proposed change, accepted decision, delegated authorization, intended effect, observed result, and exceptional recovery. Transport, indexes, and most telemetry can then remain replaceable. The research burden shifts to proving the relationships among those records and the operations they govern. That is a tractable version of the project's ambition, and a useful basis for its more speculative outcome engine.

5. Knowledge, agents, and outcomes

The most consequential question for the-valley is whether a project can preserve enough understanding to let a succession of unfamiliar contributors change it well. An agent that can retrieve a decision is useful. An agent that understands when the decision applies, notices when its premises fail, and produces a change consistent with the project's purpose is substantially more useful. The knowledge graph and outcome engine should be judged against that second standard.

This chapter examines that ambition through design rationale, belief revision, planning, agent memory, human automation, and evaluation. The central conclusion is an inference from those fields: the graph can be a strong common substrate, but it does not eliminate the distinct problems of deciding what is true, deciding what to attempt, deciding what counts as success, or deciding when a human needs to intervene. Those distinctions should survive even if one interface presents them together.

Research was checked on 2026-09-20. Source identifiers A01-A28 refer to the [annotated source register](#). Literature findings are attributed below. Statements marked **assessment**, **recommendation**, or **hypothesis** are this report's judgments, rather than results established by the cited work. Proposed experiments have not been run.

5.1. The actual starting point

The repository already has a useful, modest implementation. Its knowledge convention⁶² defines Markdown files with typed frontmatter. The schema⁶³ and lint⁶⁴ check structure, identifiers, links, supersession, and graduation. The convention explicitly describes itself as a directory convention. This is materially different from an implemented autonomous planner.

The outcome-DAG sketch⁶⁵ proposes priority propagation and dispatch against unblocked outcomes. The demand-pressure experiment⁶⁶ adds continuous reconciliation, leases, and escalation. The later production-and-events node⁶⁷ describes events as reasons to re-examine state, with a reconciler deciding what work remains. These are design proposals with different maturity levels. The roadmap places agent dispatch and observable knowledge changes in later phases.

Three existing bug nodes already identify important limits: runaway dispatch⁶⁸, integration starvation⁶⁹, and task-state churn⁷⁰. Their mechanisms deserve tests. Their strongest claims about inevitable severity should not be treated as measurements. Small text objects in Git do not, by themselves, establish unacceptable clone cost; that depends on update frequency, packing, retention, and repository size.

One concrete semantic problem appears before any performance question. The current convention and `blocker_cleared` predicate treat an abandoned outcome as a

cleared blocker. That can reasonably mean “the parent may now be reconsidered.” It cannot mean “the abandoned requirement has been fulfilled.” The demand-pressure node itself recognizes abandonment as a possible stale dependency. An automated engine will need an explicit answer to this difference.

Assessment: the small file convention is a good experimental platform. Its main missing feature is not a vector database. It is a tested account of how a change in knowledge changes the validity of plans and completion claims.

5.2. Knowledge preservation is a theory-reconstruction problem

Naur supplies the challenge, not a guarantee

Peter Naur's *Programming as Theory Building* treats programming as the formation of understanding about how a program addresses the world. His account makes the continued presence of people holding that understanding central to a program's life. Familiarity with the program text and documentation alone is insufficient; exact revival after the theory holders disappear is impossible in his account. This is a conceptual argument developed through programming experience, not an experiment on modern language models. A01: Naur, 1985

The repository's theory-revival node⁷¹ recognizes that challenge and makes a wager against its strongest conclusion. Cheap capture and strong readers might preserve enough of the important structure to make repeated reconstruction effective. The evaluation node⁷² proposes why-questions, defended modifications, and detection of plausible violations as probes.

Recommendation: test a narrower, operational claim: access to curated project knowledge improves the quality of previously unseen modifications at an acceptable reconstruction cost. Success would be valuable without establishing that text contains the whole original theory. Nor should agreement with the current theory holder be the only criterion. A contributor may understand a design and find a valid reason to revise it.

For example, a contributor asked to speed integration might suggest running all missing checks on the host. Repeating “the host does not run checks” shows recall. Explaining how that choice changes the trust and execution boundary shows understanding. Recognizing a case that requires a separately authorized verification service shows the ability to extend the design. These are different levels of competence, and the evaluation should score them separately.

Design rationale has a substantial history

Issue-Based Information Systems, associated with Kunz and Rittel's 1970 work, precede contemporary agent memory by decades. Conklin and Begeman's gIBIS implemented

collaborative, typed hypertext for capturing early design deliberation. The paper's abstract describes a database-backed tool and reports encouraging but incomplete early experience. It establishes prior art for structured deliberation, not proof that graph-shaped documentation improves engineering outcomes at scale. A02: Conklin and Begeman, 1988

Assessment: the-valley's insistence on a clean statement of current design addresses a real retrieval problem: a reader should not have to reconstruct the conclusion from a discussion. However, clean current prose and recoverable justification serve different questions. “What should the system do?” needs a current answer. “Under which changed conditions should this decision be reopened?” needs premises and limiting conditions. Git history preserves previous bytes, but those conditions remain difficult to find if they were never stated clearly.

A useful decision record could therefore say: “This design assumes one integrator per protected ref. Reconsider the design if integration must remain available during loss of that integrator.” That sentence is current rationale, not an account of participants or discarded drafts. A report or research appendix can retain the wider alternatives without turning the normative graph into a transcript archive.

Formal argumentation addresses a related but different problem. Dung models arguments and attacks between them, then defines conditions under which sets of arguments are acceptable. Different acceptance semantics can yield different conclusions. The framework deliberately abstracts

away from the internal content of arguments; it does not determine whether a natural-language argument is sound. A28: Dung, 1995

Inference: disagreement belongs in the research surface even when the normative graph states one current policy. Two incompatible proposals can each deserve investigation. Recording the objection to a proposal is useful without granting it blocking authority over all project work. Moreover, counterarguments can form cycles even when execution prerequisites must remain acyclic. The whole knowledge graph should not inherit the outcome scheduler's topological restrictions. A typed graph earns its complexity by giving those relations different meanings.

Belief dependencies differ from work dependencies

Doyle's truth maintenance system records reasons for beliefs and revises accepted beliefs when their supporting assumptions change. Its important contribution here is the explicit relationship between a conclusion and the conditions that justify it. The paper addresses symbolic problem solvers; it does not solve the problem of extracting reliable justifications from arbitrary prose. A03: Doyle, 1979

Inference: a single untyped notion of “depends on” would obscure three relationships in the-valley. A deployment can wait for a build. A design decision can rely on an assumption about threat models. An incident conclusion can be supported by an observed log. Finishing the build releases

work; invalidating the assumption calls for reconsideration; withdrawing the log weakens an explanation. The appropriate response is different in each case.

W3C PROV distinguishes entities, activities, derivation, and responsibility. That vocabulary helps represent evidence relationships; it does not establish their truth. A04: W3C PROV-DM, 2013

Recommendation: keep authoritative status, supporting evidence, and observed provenance separately queryable. Initially this can be prose with a small number of explicit links. Add machine semantics only when a concrete question requires them. A useful first question is: “Which active conclusions need review because evidence X was invalidated?” A general-purpose ontology is unnecessary to ask it. When promoting a conclusion into durable knowledge, retain enough evidence to reassess it after raw logs expire. If that support becomes unavailable, record the loss explicitly rather than presenting the conclusion as equally supported.

5.3. Existing tools clarify the storage choices

git-bug embeds a distributed issue tracker in Git without adding ordinary project files. Its official documentation describes offline operation, synchronization through Git remotes, several interfaces, and bridges to hosted trackers. It is direct evidence that repository-associated project

knowledge does not require Markdown in the checked-out source tree. Its documented behavior is not an independent performance evaluation. [A05: git-bug documentation](#)

Beads is closer to the proposed dispatch surface. Its current README describes a dependency-aware issue graph, a ready-work query, atomic task claiming, persistent memory, and Dolt-backed storage. Embedded mode is described as single-writer; server mode supports concurrent writers. JSONL is an export rather than the authoritative database. These are claims about the documented implementation at the access date. They should not be generalized to every historical Beads version. [A06: Beads documentation](#)

The local Beads comparison⁷³ identifies the important requirement: knowledge changes should pass through the same governed integration boundary as other durable project changes. **Assessment:** the phrase “one history” needs a precise definition before it becomes a storage constraint. A repository can contain many Git refs with different histories. A system with several stores can present one causally linked audit record. Neither fact alone establishes atomic publication of code and knowledge.

Consider the specific failure to prevent. A change fixes a parser, but its outcome is marked done before the protected ref advances. A later integration rejection leaves the outcome falsely closed. Storing both records in one directory does not prevent this if they land separately. Conversely, two stores can enforce a publish protocol that never exposes

completion before the relevant integration receipt. That protocol may be too complex for v1, but the decisive property is the transition rule.

Three designs are worth comparing experimentally:

Design	Main advantage	Question that determines suitability
Markdown nodes in the code tree	Simple inspection and atomic code-plus-node commits	How much concurrent editing and status traffic remains comfortable?
Git object records outside the code tree	Independent collaboration without worktree noise	Which fetch, backup, and authorization rules cover the additional refs?
Transactional graph store with immutable publication receipts	Efficient claims, queries, and concurrent updates	How are graph publication and code integration coupled and recovered?

Recommendation: keep the current durable files until measured workloads justify another backend. Build derived indexes over commit-pinned snapshots. Give transient leases their own operational contract; do not silently turn a heartbeat into authoritative project knowledge. The existing signal contracts⁷⁴ already distinguish durable events, lossy metrics, and retained logs. That distinction is a better starting point than choosing one storage engine for everything.

5.4. Agent memory is several different systems

Retrieval improves access, not authority

Retrieval-augmented generation couples generation with retrieval from external documents. Lewis and colleagues demonstrated this approach on knowledge-intensive language tasks using a learned retriever and sequence generator. It provides a foundation for bringing external material into generation; it does not establish which project document is authoritative when several disagree. [A17: Lewis et al., 2020](#)

MemGPT makes memory management part of agent behavior. It gives the model operations for moving information between limited working context and external storage, and evaluates conversational memory and document analysis. Its experiments also expose dependence on tool-use capability and retrieval behavior. The operating-system analogy is a design aid, not a guarantee that information will be loaded when needed. [A11: Packer et al., 2023/2024](#)

Generative Agents combines remembered experiences, retrieval, reflection, and planning in a simulated community. Its evaluations emphasize believable behavior and emergent coordination. The work is useful for understanding how these components interact, but social believability in a small simulation is not a test of reliable software maintenance. [A12: Park et al., 2023](#)

Reflexion stores verbal feedback from attempts and uses it in later trials without updating model weights. It supplies a concrete model for learning from execution feedback across attempts. Its reported benchmark gains depend on the evaluator, feedback, task distribution, and retry setup. They do not imply that an agent's written explanation of its failure is itself reliable evidence. [A13: Shinn et al., 2023](#)

Assessment: these mechanisms are compatible with the-valley's single-home-for-project-knowledge principle⁷⁵. The durable graph can remain authoritative while each run uses a temporary summary, search index, or working-memory view. The important boundary is promotion. A remembered observation should not become project policy simply because a model summarized it confidently.

A useful distinction is:

Material	Example	Appropriate treatment
Project commitment	Integration reads policy from the target branch	Governed durable knowledge
Observation	A command failed against a named tree and tool version	Evidence with execution context
Tentative explanation	The failure may come from an environment mismatch	Hypothesis, open to correction

Material	Example	Appropriate treatment
Run working state	Files still to inspect during this attempt	Disposable or check-pointed execution state

These categories can share an interface. They should retain different authority and retention rules. For example, an index may retrieve a hypothesis as relevant context, but the briefing should not render it as a decided invariant.

A large context window is not a memory evaluation

Lost in the Middle found that the tested models often used relevant information less successfully when it appeared in the middle of long contexts. Its contribution is an experimental method: manipulate position while holding the information constant. The measured degradation belongs to the tested models and tasks; its magnitude should not be assumed for current models. [A14: Liu et al., 2023/2024](#)

LongMemEval broadens memory testing to extraction, cross-session reasoning, temporal reasoning, updates, and abstention. It uses curated questions within constructed conversation histories. These dimensions matter more to the-valley than a single retrieval score, although conversational memory is not project design understanding. [A15: Wu et al., 2024/2025](#)

The 2026 LongMemEval-V2 preprint moves toward environment-specific experience: state changes, procedures, recurring pitfalls, and questions whose premises may be false. Its evaluation asks memory systems to return bounded evidence for a fixed reader. That separation helps distinguish retrieval quality from answering ability. Its website environments and question-answering formulation still leave long-term engineering quality unmeasured. A16: Wu et al., 2026

Recommendation: evaluate retrieval over actual project histories containing deliberate changes of mind. Ask which decision is current, what was true at a past commit, whether an old assumption still holds, and whether the graph contains enough evidence to answer. “Unknown” should be correct when the record is insufficient. A system that answers every question fluently is a poor project memory.

The retrieval result should identify its source commit and preserve access to the full passage. Rank current authoritative material separately from historical material. Test lexical search before adding embeddings, and compare both with graph expansion from explicit references. The winning method may differ for exact identifiers, conceptual questions, and questions about change over time.

SWE-agent provides another relevant result: changing the agent's tools and feedback changes its ability to work in a repository. Its study evaluates specialized navigation, editing, and execution interfaces, including ablations against

a shell-based baseline. It supports treating interface design as part of the experimental system, rather than attributing every success or failure to the model. [A18: Yang et al., 2024](#)

Hypothesis: small project-specific verbs such as “show current decision,” “explain blockers,” and “show evidence for completion” will provide more reliable value initially than an elaborate autonomous memory manager. Test that hypothesis under equal tool-call and context budgets.

5.5. An outcome graph is a plan representation, not a complete planner

Refinement needs a meaning

Goal-oriented requirements engineering separates desired properties from operations that might achieve them. Van Lamsweerde's account distinguishes AND refinements, alternative OR refinements, conflicting goals, responsibility, and assumptions about the environment. It also distinguishes achieving a condition from maintaining one over time. This is directly relevant to a system that wants to represent both shipping an artifact and keeping a service healthy. [A07: van Lamsweerde, 2001](#)

Inference: the current `blocked_by` relation is sufficient for an initial worklist, but too weak to carry all the meanings implied by an outcome engine. “Release is waiting for a security review” is a precedence constraint. “A valid release consists of a tested artifact and a compatible migration” is a claim about sufficient conditions. “Restore service by

rollback or by repair” presents alternatives. Representing every edge as a mandatory blocker would launch unnecessary work in the last example.

Hierarchical task network planning studies how compound tasks can be reduced through methods into more concrete tasks. Erol, Hendler, and Nau show that expressivity and complexity depend strongly on the allowed task-network structure and restrictions. Their results concern formal planning models; they do not say that the-valley's current small graph is intractable. They do show why unrestricted recursive decomposition deserves its own design rather than being treated as free scheduling. A08: Erol et al., 1994

Recommendation: initially let agents propose decompositions while a deterministic scheduler executes an accepted graph. Each proposed decomposition should state its parent condition, its assumptions, why the children are sufficient, and what would invalidate the plan. A new child should consume a share of the parent's budget; adding an edge should never mint new spending authority. This is a recommendation for bounded automation, not a demand for approval of every low-impact task. Pin the accepted objective and acceptance procedure, including which authority may revise them. An agent's proposed amendment must remain distinguishable from the rule currently deciding success.

Readiness, priority, and criticality are different

A graph can identify nodes without open predecessors. Priority propagation can identify which root caused demand. Neither calculation identifies the longest remaining path without an estimate of duration. Resource limits, actor capabilities, communication, and uncertain work lengths further change which assignment produces the earliest completion.

HEFT is useful prior art because it explicitly schedules a known task graph on heterogeneous processors using estimated computation and communication costs. It ranks tasks and assigns them to processors using a heuristic. Its setting differs from agents discovering new tasks during execution, but it makes the missing quantities visible. [A09: Topcuoglu, Hariri, and Wu, 2002](#)

Suppose a release needs a two-hour verification run and a five-minute policy clarification. Starting the expensive run immediately may be best if the clarification cannot change its inputs. Resolving the clarification first may be better if it could invalidate the run. The second choice is about reducing uncertainty, not simply following the longest visible path. A scheduler needs to recognize that information-gathering work can prevent wasted production.

Recommendation: begin with eligibility, explicit root priority, age, estimated cost, and actor availability. Record the estimates and actual outcomes. Add a critical-path heuristic only when measurements support useful estimates. Preserve a bounded share of capacity for older or lower

priority work if starvation is unacceptable. A stream of high-priority roots otherwise gives ordinary work no completion guarantee.

Queueing theory offers a basic diagnostic. Little's relation connects average work in progress, arrival rate, and average time in the system under the theorem's conditions. It is not a universal causal promise that reducing any visible queue improves throughput. A10: Little, 1961

Inference: dispatching more agents can increase work awaiting integration or human clarification without increasing completed outcomes. Measure each queue separately: ready work, running work, changes awaiting integration, and requests awaiting human decisions. The bottleneck may move when one stage improves.

Stopping is part of the outcome contract

Consider an outcome “users can export all their data.” Its children implement an endpoint and update the documentation. Both changes land and pass their checks. The parent remains false if the endpoint omits archived records. A completion receipt demonstrates the checks that ran; it does not repair an incomplete definition of the outcome.

Recommendation: distinguish at least these conditions in the engine's internal model:

1. A prerequisite has reached a terminal administrative state.

2. Work is eligible to attempt.
3. An attempt produced evidence satisfying its declared acceptance procedure.
4. The desired condition is accepted as attained in a stated environment and scope.
5. The condition remains valid under current observations.

This need not mean five new frontmatter statuses. It means the scheduler and closure procedure must not substitute one statement for another. In particular, an abandoned prerequisite should cause parent reassessment, replacement of the plan, or abandonment of the parent. It should not count as evidence that a required condition holds.

The production sketch⁷⁶ already recognizes that service obligations do not terminate. Its claim that versioning turns feedback cycles into acyclic artifact histories is useful, but acyclicity does not guarantee progress. An infinite sequence of distinct revisions can remain stuck. Retries need stable logical identity across runs, otherwise renaming a task erases its failure count.

Recommendation: measure progress toward acceptance, not graph growth or number of closed child nodes. Bound total spend, attempt count, elapsed time, and decomposition. Cancellation should withdraw the cancelled root's demand through the descendant graph. A shared prerequisite can remain required by another live root. Lease expiry must not grant two workers simultaneous authority to perform the same external effect; that requires enforcement at the effect

boundary. Log enough of each failed attempt to distinguish a transient environment failure from repeated misunderstanding. Budget reservations must account atomically for concurrent descendants, retries, and restarted controllers. Independent workers checking the same remaining balance can otherwise each authorize spending that exceeds the shared limit.

5.6. Human attention is a resource with special constraints

The demand-pressure proposal treats a human-blocked node as work dispatched through notification. That is useful for representing responsibility. It is insufficient as a model of attention. A human cannot be cloned, paused, or restarted like a process. An interruption has costs beyond the seconds spent reading it, and multiple notifications can concern the same underlying decision.

Bainbridge's *Ironies of Automation* explains a central difficulty of supervision: automation may leave humans responsible for exceptional intervention while depriving them of the practice and ongoing involvement needed to intervene well. It is a conceptual analysis of industrial automation, not a quantified study of coding-agent review. Its relevance is the failure mode it makes visible. A25: Bainbridge, 1983

Parasuraman, Sheridan, and Wickens distinguish automation of information acquisition, analysis, decision selection, and action implementation. These functions can have different

automation levels. Their framework discourages treating autonomy as one switch that controls the whole system. A26: Parasuraman et al., 2000

Inference: the-valley can automate evidence collection and execution while retaining human judgment over the meaning of a release criterion. It can automatically summarize incidents while requiring a separate act to revise the project's threat model. These boundaries may vary by outcome class. “Human in the loop” is too vague to describe them.

Horvitz's mixed-initiative work explicitly considers uncertainty about user goals, the timing of interruptions, expected costs and benefits, clarification, and easy invocation or termination of automation. It offers a foundation for deciding when assistance should act or ask, illustrated with a scheduling interface rather than software production. A27: Horvitz, 1999

Recommendation: route a decision request with the exact choice, relevant evidence, consequence of waiting, and action that follows each answer. Combine requests that depend on the same decision. Separate urgent intervention from a digest of completed work. Measure whether the recipient can make the decision from the supplied context, not merely whether a notification was delivered.

For example, five agents blocked on a signing-policy change should generate one coherent policy decision with five affected outcomes. Five independent urgent messages would turn graph structure into interruption amplification.

Likewise, an agent's third failed integration should appear as one recurring failure with its history, not another apparently new proposal.

The architecture's review-as-feedback bet⁷⁷ becomes defensible when paired with evidence about detection delay and consequence. A formatting change and a modification to the authority boundary need different review timing. The useful comparison is between concrete policies: pre-integration review, sampled post-integration review, targeted review of uncertain changes, and automatic acceptance with later incident analysis. The report does not assume one policy wins for every change class.

Hypothesis: a small, calibrated stream of decision requests and sampled completed changes will preserve understanding better than either reviewing everything superficially or seeing only failures. A trial should measure missed defects, review time, interruption count, and the operator's ability to explain current system behavior after several weeks.

5.7. Evaluation should measure useful work and preserved understanding

What existing benchmarks establish

SWE-bench turns historical repository issues into executable patching tasks. It evaluates whether candidate changes satisfy tests associated with issue resolution while preserving specified existing behavior. This is substantially

closer to repository work than isolated function generation. It still does not measure all of specification discovery, architectural coherence, deployment, or maintenance over successive changes. [A19: Jimenez et al., 2023/2024](#)

SWE-bench-Live develops an updatable collection of issue-resolution tasks and automated environment construction. Fresh tasks and broader repositories help address staleness and some leakage risks. Recency does not establish absence of training exposure, nor does it make test-based acceptance equivalent to complete issue resolution. [A20: Zhang et al., 2025](#)

Oren and colleagues demonstrate a statistical contamination test based on the likelihood of canonical versus shuffled benchmark orderings. Its assumptions and access requirements matter. Evidence from a particular detector cannot be generalized into proof that every undetected benchmark is clean. [A21: Oren et al., 2023](#)

AI Agents That Matter argues for jointly evaluating cost and accuracy, adequate holdouts, simple baselines, and reproducible evaluation procedures. Its empirical examples show that apparent gains from elaborate agent designs can be misleading when spending and retries are not controlled. That critique is especially relevant to a scheduler whose normal response to unfinished work is another attempt. [A22: Kapoor et al., 2024](#)

The practical productivity evidence is also conditional. METR's early-2025 randomized study found a 19% increase in completion time when 16 experienced maintainers could

use the studied AI tools across 246 tasks. That result describes those tools, maintainers, and tasks; it is not a general estimate of present agent productivity. [A23: Becker et al., 2025](#)

METR's February 2026 update reports that selection into its later experiment and measurement of time under concurrent agent use made the later estimates unreliable. The organization argues that the study design needs revision. Reporting the earlier slowdown without this update would invite a misleading current conclusion. [A24: Becker et al., 2026](#)

Assessment: the-valley should learn experimental discipline from these sources without choosing one headline score as its objective. The system exists to produce outcomes while preserving control and understanding. Patch success, operator time, latency, expenditure, and later rework are separate measurements.

A theory-reconstruction experiment

The strongest near-term experiment uses a fixed repository snapshot and tasks that require applying its design to new situations. The task set should include valid modifications, plausible changes that violate a principle, and questions the existing record cannot answer. Avoid tasks copied from documents that already contain their solutions.

Compare four knowledge conditions while holding the model, harness, tools, and budget constant:

Condition	What the contributor receives	What the comparison tests
Code and ordinary documentation	The repository without the knowledge graph	Value added by curated project knowledge
Current graph	Code, documents, and the current typed nodes	Benefit of the existing convention
Graph with justification and evidence	The same material plus explicit premises and source links	Whether additional structure improves application rather than recall
Retrieved briefing	A bounded briefing derived from the same snapshot, with source access	Whether context selection preserves understanding cheaply

Add a matched-information comparison that presents identical facts and rationale as ordinary prose and as typed nodes. This separates the value of the information from the value of graph structure. Keep held-out tasks independent of wording and examples unique to the graph.

Score at least factual recall, validity of proposed modifications, identification of missing assumptions, justified abstention, and ability to explain tradeoffs. Ask blinded human reviewers to judge a sample using a rubric established before seeing outputs. A model judge can extend coverage after calibration, but agreement with one favored model is not a definition of design quality.

Measure tokens and time until a satisfactory solution, including retrieval and unsuccessful attempts. Count later corrections. Give multiple independent runs each task and report uncertainty around differences. Publish representative failure cases, not just a mean score. A knowledge condition that raises recall but produces more overconfident invalid changes should not win.

Then reverse the experiment: hold the model fixed and compare graph versions. Use held-out tasks so that editing the graph to fix observed failures does not simply teach it the test answers. Separate improvements in the graph from changes in prompts, tools, and model versions. This makes the repository's two-sided evaluation proposal concrete.

Completion claims need adversarial tests

A second evaluation should attack the outcome engine's semantics. Construct small graphs where a blocker is abandoned, evidence expires, an OR alternative succeeds, a parent is cancelled during a child run, or a worker loses its lease before returning. Include mutually dependent outcomes and repeated revisions that never make substantive progress.

The expected behavior should be stated independently of the implementation. For example, abandoned work must not count as fulfilled evidence; cancellation must stop work justified only by the cancelled root while preserving shared prerequisites still demanded elsewhere; stale workers must not duplicate external effects; and a cycle must produce a diagnostic rather than silent inactivity. Compare the

proposed automatic engine with a simple manual ready-work query. The engine must add useful progress, not merely activity.

For scheduling policies, replay representative arrivals and measured task durations in simulation before spending model calls. Compare priority-only dispatch, priority with aging, estimated critical-path dispatch, and a policy that resolves high-impact uncertainty first. Report root completion time, worst waiting time, total spend, discarded work, and human decision load. The best policy will depend on workload; finding those boundaries is a useful result.

Reproducibility has limits that should be explicit

The current agent-provenance proposal⁷⁸ already notes that a hosted model is an impure service boundary. The useful experiment record includes the repository tree, graph snapshot, prompt and retrieved context, tool versions, tool results, model identifier, budgets, and acceptance procedure. A digest of withheld material proves a commitment to bytes; it does not make those bytes available for independent replay.

Recommendation: distinguish rerunning a workflow, reproducing its environment, and reproducing its exact output. Even known model weights do not by themselves fix sampling, numerical execution, or changing external observations. Record what is controlled and what is merely identified.

An observed tool transcript is evidence of externally visible actions. An agent's explanatory prose is a claim about its process. Neither is direct access to all internal reasoning. Evaluation should prefer interventions: remove a premise, change a tool response, or vary the available evidence and observe the resulting behavior. This can test dependence on information without pretending to read the model's mind.

Finally, maintain a prospective task set whose solutions do not exist at task creation. Record when tasks become public and whether providers may receive them. Reserve some tasks and judging criteria from normal graph maintenance. Public task freshness, controlled access, and documented exposure reduce different forms of leakage; none should be described as perfect decontamination.

5.8. The most promising research direction

Assessment: the-valley's distinctive opportunity is to connect governed project knowledge with evidence about completed work. Repo-local notes, agent memory, task graphs, workflow execution, and provenance each have substantial prior art. The interesting question is whether their combination can make project understanding cheaper to reconstruct while making delegated work easier to inspect and correct.

The strongest claim to pursue is therefore conditional and measurable: a fresh contributor using the project's current record can make an unfamiliar change with fewer

conceptual mistakes, at lower total recovery cost, while preserving an inspectable chain from intent to accepted result. That claim would justify the substrate even if a fully general outcome-production engine remains distant.

Four investigations can discriminate among the plausible designs:

1. **Knowledge ablation:** determine whether the present graph improves unseen modifications over ordinary repository documentation. If it does not, investigate content and retrieval before adding more node types.
2. **Completion semantics:** implement an executable model of readiness, acceptance, invalidation, abandonment, and cancellation. Use counterexamples to determine which distinctions the public schema actually needs.
3. **Operational load:** measure realistic graph mutation and dispatch workloads. Compare files, derived indexes, and a transactional backend on conflict rate, recovery, and operator effort.
4. **Attention trial:** run bounded automation for several weeks and measure the work needed to maintain understanding. Compare review and notification policies by defects caught and human time, not volume of generated summaries.

Recommendation: prioritize the first two before a rich autonomous scheduler. Better scheduling cannot rescue an engine that mistakes an administrative transition for an attained outcome. More memory cannot rescue an

authoritative record that omits the assumptions needed to interpret it. The research should reward useful negative results: a simpler query, a shorter briefing, or a human decision at the right boundary may outperform a more elaborate agent loop.

6. Synthesis and a research agenda

the-valley's aim is a useful end-to-end system assembled around clear requirements and deliberate constraints. Existing solutions should satisfy individual requirements wherever they fit. The central hypothesis is that consistent rules across those solutions can make the complete workflow more capable and easier to operate. No individual subsystem needs to be an innovation.

The next step is a small reference workflow that makes those rules observable. Tests of portable evidence, project understanding, and recovery should explain its behavior. Their purpose is to locate the work needed to make the whole system succeed.

This chapter states the report's judgments. Its proposals are research recommendations, not adopted project requirements. The empirical experiments below have not been run. Their purpose is to make the most interesting claims falsifiable and to identify useful results even when a proposed approach fails.

6.1 What the review supports

Five conclusions survive comparison with the literature and existing systems.

Portable evidence is a sound organizing idea. Build systems already provide a basis for reusing computation under explicit dependency assumptions. Supply-chain systems already separate artifacts, claims, signers, and policy. the-valley can combine those ideas into a record explaining why one particular state was accepted. The opportunity is the precision and usefulness of that record, rather than the mere existence of another signature format. The evidence and limits are in [chapter 3](#).

The knowledge graph has a stronger purpose than agent memory. Its most interesting purpose is to help a fresh contributor apply the project's design to an unfamiliar change. That is a measurable claim about understanding. It can fail even when retrieval is excellent and every fact is recalled. Conversely, a relatively small graph may succeed if it preserves the right explanations and constraints. [Chapter 5](#) develops this distinction from Naur, design rationale, planning, and agent evaluation.

Durable state plus reconciliation is a good starting point. It gives the system a way to discover unfinished work after a notification is lost. However, accepted decisions and external effects need their own recovery semantics. The relevant contrast is between recoverable obligations and transient messages. [Chapter 4](#) explains why current ref replay is insufficient for complete historical recovery.

Ordinary systems are strong candidates for adoption. Concurrent merge validation, portable execution, distributed collaboration records, and policy stored in a repository all

have working precedents. The first question is which configurations meet the requirements. Their guarantees and operating costs determine where adaptation or custom work is justified. Those same configurations also supply credible baselines for evaluating the complete workflow. Chapter 2 supplies those baselines.

Autonomy needs separate boundaries for proposing, accepting, and acting. An agent can propose work without authority to integrate it. Integration can accept a source change without authorizing every external effect. A system can automate evidence gathering while preserving human judgment over what an outcome means. Treating these as distinct decisions creates more useful choices than one global autonomy level.

6.2 Claims that should be narrowed before relying on them

The table identifies research findings about the snapshot. It does not silently amend the project's design documents. Implementation concerns remain static findings until reproduced or ruled out by an executable experiment.

A malicious local attester must reveal a modified tool hash

More defensible statement: A signer can claim an approved hash without executing that tool

Evidence and next check: Protocol counterexample in chapter 3; test a directly fabricated signed statement

Pure re-verification leaves no residual attack

More defensible statement: Independent execution checks agreement under shared assumptions; an inadequate check can agree everywhere

Evidence and next check: Chapter 3; use a passing no-op replacement and separate definition authorization from execution

Witness sampling guarantees detection within a bounded window

More defensible statement: Random sampling provides a probability under stated assumptions; an isolated unsampled false claim may remain undetected

Evidence and next check: Chapter 3's sampling analysis; explicitly model witness capacity and correlated failures

The described stack is roughly SLSA Build L3

More defensible statement: No level follows without satisfying the specified builder and provenance requirements

Evidence and next check: Chapter 3 and its SLSA source; map one real deployment requirement by requirement

Replaying Git reconstructs the lived event history

More defensible statement: Current-ref enumeration reconstructs a view of surviving refs

Evidence and next check: Chapter 4; create, rewrite, and delete a ref before losing the bus

A landing and its evidence are one durable result

More defensible statement: The inspected sequence has multiple writes around the commit point

Evidence and next check: Chapter 4; interrupt the process between each durable write

Every declared expiry is an enforced deadline

More defensible statement: Raw compiled keys remain subject to successful policy convergence and recovery paths

Evidence and next check: Chapter 4; let a key expire while compilation repeatedly fails

A dependency DAG provides a critical-path scheduler

More defensible statement: Topology supplies precedence; useful criticality also needs duration and resource information

Evidence and next check: Chapter 5; compare priority-only and estimated-duration policies

Terminal blockers imply an outcome can be considered achieved

More defensible statement: Administrative termination, readiness, and accepted attainment are different facts

Evidence and next check: Chapter 5; abandon a required child while retaining the parent's acceptance condition

Recording agent inputs explains exactly who did everything

More defensible statement: A trusted observer can record a run and its external actions; the record has collection and coverage limits

Evidence and next check: Chapters 4 and 5; distinguish run occurrence, delegated principal, signer, and observed actions

Several of these limits are already recognized in the knowledge graph or open questions. The report does not claim to have discovered every issue first. Its contribution is

to connect them to broader literature and explain which claims would remain unjustified even after an obvious implementation bug was fixed.

6.3 A useful boundary for the minimal core

The following is a proposed division of responsibility. It is not a protocol specification.

Information flow

Each arrow means that the receiving stage needs information from the preceding stage. The connections are:

- Requested outcome and acceptance conditions → Candidate change
- Current design and rationale → Candidate change
- Candidate change → Checks and observations
- Checks and observations → Evidence about a precise subject
- Policy and current authority → Integration decision
- Evidence about a precise subject → Integration decision
- Candidate change → Integration decision
- Integration decision → Durable accepted state and decision record
- Durable accepted state and decision record → Indexes, review views, and notifications

- Durable accepted state and decision record → Separately authorized external action
- Separately authorized external action → Observed result or unresolved outcome
- Observed result or unresolved outcome → Current design and rationale
- Observed result or unresolved outcome → Requested outcome and acceptance conditions

The key property is that each arrow names information the receiver needs. A check executor does not decide the project's authority policy. An index does not become the only record of an approval. An agent's proposed explanation does not become accepted knowledge merely because it is fluent. An actuator does not infer unlimited authority from an integration event.

The minimal core could define identities, immutable records, policy evaluation inputs, and the conditions under which a transition becomes accepted. Execution backends, scheduling heuristics, retrieval systems, and user interfaces could remain replaceable. This is an architectural preference, not evidence that the resulting implementation will be small. A second independent consumer of the records is the practical test.

Existing contracts and components may supply these responsibilities. The proposed division does not require a new core implementation.

The report also recommends separating the following graphs, even if they share storage:

Graph	What an edge means	Mistake caused by treating it as another graph
Source history	A commit names parent commits	Assuming ancestry records every ref movement or operational event
Computation dependencies	A result depends on an identified input or action	Treating common file paths as a complete dependency model
Outcome dependencies	A requested condition requires, or is supported by, another condition	Treating closed child tasks as proof of the parent
Provenance	An activity used or produced an entity under stated attribution	Treating a recorded relationship as a causal explanation of an incident
Authority delegation	A principal grants bounded rights to another actor	Treating declared responsibility as enforced permission

An undifferentiated graph with an edge named `depends_on` would be easier to implement initially and harder to reason about later. The recommendation is semantic separation, not a demand for five databases or an elaborate ontology.

6.4 The most promising solution spaces

Evidence transfer as a conservative optimization

Start from a clearly authorized check over a clearly identified subject. Accept reuse only when the system can establish that the relevant computation and inputs remain the same. Fall back to new evidence when the dependency model is incomplete. This gives a soundness target independent of whether the check itself is a good test.

There are several implementation choices: whole-tree identities, filtered source sets, declared dependency closures, dynamic traces, or combinations. None is best in every project. Whole-tree identities are conservative but can waste computation. Finer declarations can save work while adding maintenance and omission risk. The research question is where the additional precision pays for itself on the pilot's real workload.

Trust should be explicit alongside this optimization. A personal host, an isolated builder, and an independent witness support different claims. A source change can be admitted under one policy while a higher-impact deployment requires additional evidence. This preserves a useful fast path without pretending that later revocation undoes every consequence.

Immutable decisions with disposable views

Store a narrow decision record that identifies what was accepted, under which policy, and using which evidence. Derive review pages, queues, indexes, and bus publications from retained facts. Keep effect attempts and unresolved external outcomes durable where they cannot be reconstructed.

Possible storage choices include Git objects and refs, an append-only journal, or a transactional store with exports. A single history should mean a coherent account of accepted project state and its interpretation. It need not mean forcing every high-frequency telemetry event into the main source branch.

The design should name its required atomic boundaries before selecting storage. If two records must be accepted together, that is a constraint on the transaction or recovery protocol. If one is merely a cache, it should be safely reconstructible. Describing both as files in the same repository does not settle the distinction.

Project knowledge as tested explanatory material

Preserve the current design in plain language, with enough premises and evidence to explain why it takes its present form. Evaluate it using new changes and plausible design violations. Improve retrieval only after establishing that the source material contains the necessary understanding.

This path permits multiple derived views: a short briefing for an agent, a design review for a human, or a query over unresolved decisions. Each should point back to an accepted snapshot. Generated summaries can be useful while remaining fallible views over the record.

The key risk is making the graph increasingly complete and increasingly expensive to maintain. Its value should be measured net of that maintenance. If a concise design document performs as well as a large typed graph, that is a useful result for a project committed to minimalism.

Bounded production with explicit acceptance

Begin outcome automation with a ready-work query, a bounded attempt, and a separately evaluated completion claim. Record why work is blocked, what evidence would clear the block, and when further attempts should stop. Add scheduling intelligence as measured contention justifies it.

The important distinction is between a plan the system currently believes and an obligation the project has accepted. Agents can revise plans as they learn. They should not silently weaken the obligation to make the plan look successful. A revision to acceptance criteria is itself a decision with authority and rationale.

This solution space remains useful even if the scheduler stays simple. A faithful record of failures, assumptions, and incomplete evidence could improve human-directed work before autonomous planning becomes dependable.

6.5 Experiments that would change the design

E9 is the overall test of the system. Start it with the smallest complete workflow supported by the current scenario rung, and extend it as later rungs become relevant. E1–E8 investigate particular risks and explain results from that workflow. Their numbering does not require eight successful subsystem experiments before an end-to-end trial. Each experiment should preserve its inputs, evaluator, failures, and raw measurements. Agree proposed thresholds before inspecting results.

E1. Can a second implementation explain an accepted change?

Use an existing independent verifier where it supports the contract. Build a small read-only verifier only for missing behavior, independently from the integrator's decision implementation. Give it the accepted state, evidence records, relevant policy snapshots, and trust roots. Remove access to the original controller and bus. Ask it to reproduce the reason for acceptance or report the exact missing fact.

Evaluate historical authorization separately from present acceptability. Supply the historical registry or checkpoint needed to establish what was authorized at decision time. A later revocation may change what can be accepted now without erasing the reason the earlier decision was made.

Include ordinary changes, policy amendments, stale evidence, revoked keys, and a re-parented landing. Test unknown predicate versions and unavailable artifacts. A verifier should distinguish an invalid record from a record it lacks enough information to evaluate.

The key result is agreement on well-defined cases and explicit disagreement where the contract is underspecified. If independent verification requires undocumented controller behavior, the protocol boundary is incomplete. This experiment should precede claims of backend independence.

E2. Is evidence reuse sound for the admitted computation model?

Create a corpus of changes with independently specified expected invalidations. Include source edits, check-definition edits, compiler changes, generated inputs, directory additions, policy changes, and changes to external observations. Include two independently authored edits that combine badly despite a clean textual merge.

For deterministic checks, compare reuse decisions against execution of the approved check on the candidate landing. Instrument the executor so that identical success markers from two different commands do not count as identical computations. Repeat with whole-tree and finer dependency models.

Any reused result that disagrees with the specified deterministic computation is a failure of that reuse rule. A zero-failure run is useful evidence for the tested cases, not proof for all programs. Report conservatism as well: how much safe reuse was missed, and what declaration effort would recover it? For effectful checks, specify acceptable observation age and scope instead of pretending a rerun sees the identical world.

E3. Where does the latency advantage come from?

Compare four configurations: a serialized check-and-merge loop; a speculative queue; the current valley transfer approach; and a hybrid using speculation plus reuse. Run the same checks with the same hardware budget and acceptance policy. Hold cache warmth constant within comparisons, then repeat with cold caches.

Vary arrival rate, check duration, dependency overlap, failure rate, and the fraction of effectful checks. Record time from completed authorship to acceptance, time spent waiting for required evidence, serialized decision time, total compute, invalidations, and starvation. Report distributions and worst waits, not only an average.

Use a common starting point before verification begins and include author-side execution. Also report the narrower submission-to-acceptance interval. Moving computation before submission can improve that interval without reducing total completion time; the measurements should reveal both effects.

The result should identify operating regions. The transfer design is especially attractive if it saves computation and waiting on real independent edits. If gains disappear once trust and caching are equalized, speed alone does not justify the additional protocol. Policy evaluation and Nix evaluation costs must appear in the measurement even if no test is intentionally run at commit time.

E4. What survives interruption at each boundary?

Use a disposable repository and effect target. Interrupt the integrator between preparation, target update, evidence publication, request deletion, and notification. Interrupt an actuator before and after the remote system accepts an operation. Restart from retained state.

The test oracle should require an unambiguous accepted source state and a recoverable explanation. For effects, it should require either one known logical result or an explicit unresolved result with a recovery procedure. Merely exiting successfully on restart is insufficient.

Then remove the bus, delete a ref, lose a cache object, and restore an old registry snapshot. Record which guarantees survive each loss. This creates a failure model based on demonstrations and reveals which storage changes are necessary before later automation depends on the record.

E5. Does the graph improve unfamiliar work?

Use the four knowledge conditions from chapter 5: ordinary docs; current graph; graph with additional justifications; and a bounded derived briefing. Hold model, harness, tools, and budget constant. Use tasks requiring changes the source material does not already solve.

Add a matched-information comparison: present the same facts and rationale as ordinary prose and as typed nodes. This separates improvements in content from improvements in representation. Task authors and evaluators should avoid wording held-out tasks around cues unique to the graph.

Blind a sample of human evaluations to the knowledge condition. Score conceptual errors, justified abstention, working results, and later corrections. Include the cost of creating and maintaining knowledge. Evaluate several independent runs per task and keep a held-out task set while improving the graph.

This is the strongest test of the graph's distinctive purpose. If it improves recall but not changes, investigate missing explanations and misleading authority cues. If ordinary docs perform equally well, prefer the simpler representation for that class of work.

E6. Can outcome automation avoid claiming false success?

Specify a small executable model before building a sophisticated scheduler. Include AND requirements, OR alternatives, abandoned prerequisites, cancelled roots,

expired evidence, and recurring obligations. Give an agent permission to propose decompositions but a separate evaluator authority to accept attainment.

Construct cases where every child is closed while the parent remains false. Include repeated revisions that create new identifiers without making progress. Verify that cancellation withdraws the cancelled root's demand from descendants and that late results from an expired attempt cannot authorize an effect. Include a prerequisite shared by two live roots: cancelling one root must preserve work and authority still justified by the other.

The failure criterion is an unsupported completion claim or an effect outside current authority. Efficiency comes later. Compare against a manual ready-work query to establish what the automation actually adds.

E7. Does witness sampling buy the intended assurance?

Start with simulation using chapter 3's explicit probability model. Vary audit probability, detection probability, witness backlog, error clustering, and compromised shared dependencies. Separate random mistakes from a signer that waits until it has earned a favorable score before making one false claim.

Then run controlled checks through independently configured witnesses. Measure detection delay and the number and consequence of accepted actions before detection. Expiry of authority and invalidation of historical evidence should be separate outputs.

The result should be an assurance statement for a workload and trust model, not a universal trust score. If high-impact failures cannot tolerate the residual exposure, require stronger evidence before those effects. A later successful revocation is not a successful prevention experiment.

E8. Can the operator remain informed without reviewing everything?

Run a bounded trial across several weeks of representative work. Compare pre-integration review, targeted review, sampled completed-change review, and scheduled digests. Group notifications by the underlying decision rather than by each blocked agent.

Randomize comparable changes where appropriate or counterbalance trial periods to limit confounding by workload changes and operator learning. A trial lasting several weeks can estimate routine burden; it cannot by itself establish an assurance level for rare severe defects.

Measure interruptions, time spent reconstructing context, defects found, detection delay, and the operator's ability to explain current system behavior. Sample uneventful changes as well as failures; otherwise the trial cannot discover what unattended work silently changed.

The useful result is a policy that reduces review burden while preserving understanding and an acceptable consequence profile. If nobody can explain the system after a quiet week of successful automation, notification volume was the wrong success metric.

E9. Does the complete workflow justify the composition?

Define a representative piece of work at the active scenario rung. For S2/S3, start with an intended change and project context, give the work to a fresh author, establish the required evidence, and integrate the result. Check both the resulting behavior and the retained explanation for acceptance. As S4/S6 become relevant, extend the same workflow through an authorized effect, an observed problem, recovery, and an update to project knowledge. A general scheduler is unnecessary for the first trial.

Run this workflow on the valley configuration and the strongest practical existing configuration. Match the intended result, trust assumptions, and resource budget. Where feasible, also compare the same components with and without the proposed shared constraints. That comparison helps identify the value of the integration decisions separately from the quality of the selected tools.

Measure successful outcomes, total elapsed time, operator intervention, reconstruction of missing context, escaped defects, and recovery after interruption. Subsystem speed

matters when it improves these results. Count the work of maintaining adapters and reconciling records as part of the cost.

Then perform the same operating tasks: add a project, rotate a key, introduce a check, recover a lost host, diagnose a failed integration, and change an executor. Count operator steps, required credentials, separate upgrade procedures, and time spent locating authoritative state.

Record integration code and incident handling, not just component sizes. Repeat selected tasks after a period without operating the system, because comprehensibility after absence is part of ownership. If the composition is worse on routine work, identify whether the capabilities required by the project compensate for that cost. A positive result may come entirely from configuration and clear contracts between existing components. There should be a concrete reason to retain every custom component.

6.6 Measurement discipline

The report recommends a prospective experiment register containing a question, hypothesis, baseline, dataset or workload, fixed evaluator, primary measurements, and decision rule. Version the register before collecting results. Exploratory observations can remain useful, but should be labeled as exploratory rather than used to retroactively redefine success.

Separate the following quantities:

Quantity	Why it needs its own measurement
Source accepted per unit time	Measures the integrator; can rise while useful work stagnates
Outcomes accepted under unchanged criteria	Measures delivered intent; still depends on evaluator adequacy
Time to recover project understanding	Measures the knowledge record's practical value
Human time and interruptions	Captures supervision shifted out of the apparent fast path
Compute and repeated attempts	Prevents a retry-heavy system from appearing free
Later rework and escaped defects	Captures costs deferred beyond initial acceptance
Lost or unresolved historical facts	Measures whether the transparency promise survives failure

Use multiple projects when practical, including the-valley itself and a less tailored consumer. Self-hosting is a valuable integration test but a biased effectiveness test: the project's own conventions and authors are unusually adapted to the system. Negative results on another repository may reveal a portability assumption that successful self-development conceals.

6.7 An opinionated end-to-end system

The project's success criterion is that a useful workflow meets clear requirements at an acceptable cost. Individual tools can be familiar, and a suitable existing configuration can supply most of the system. The work particular to the valley may be selecting the constraints, connecting the tools, and making the combined behavior dependable. That is sufficient reason to build it.

"More than the sum of its parts" should have an observable meaning. Each tool may work correctly in isolation while the operator still has to reconstruct intent, translate identities, reconcile policies, or discover lost obligations between them. A successful composition removes some of that work without losing the guarantees it supplied. The important gain occurs at the handoffs.

Constraints that could create the joint value

The following are proposed constraints on the complete system. They develop the requirements in [chapter 1](#); they are not new project decisions. They supplement the adopted premise constraints, which a component selection must not implicitly waive. Each constrains the behavior of several components while leaving their implementation open.

A requested result keeps its meaning throughout the work. A proposal, check, accepted change, external action, and completion claim can identify the request they serve. The acceptance conditions that applied remain available, together with any authorized amendments. An existing work

tracker, repository, executor, and evaluator can provide the parts. Their shared obligation is to prevent a sequence of successful tasks from silently replacing the result that was requested.

Verification has an identified definition and an explicit reuse rule. Local and integration checks agree on what is being tested. A consumer can decide whether supplied evidence applies to its candidate state under the required trust policy. Uncertainty triggers new evidence or a recorded block. Build systems, caches, and attestation tools can supply the mechanisms. Their shared contract prevents a familiar check name or green status from acquiring different meanings at each boundary.

Accepted changes and their reasons survive together. After a supported failure, recovery retains the facts needed to explain the accepted state. Indexes and notifications may be disposable when those facts can reconstruct them. This constraint does not require one physical database or log. It does require an explicit acknowledgment and recovery contract across the stores involved. The tradeoff is that stronger durability may add coordination or delay.

Authority is checked where the action occurs. Permission to propose, integrate, deploy, and amend an acceptance criterion remains distinguishable. Delayed or replayed work does not gain authority merely by having been queued. An external action has a retained obligation and an observed result, or an explicit unresolved outcome.

Existing identity and workflow systems can enforce these rules; the composition must align their scopes, revocation behavior, and recovery paths.

Project understanding belongs to the project. Accepted design explanations, important reasons, open questions, and relevant outcomes remain readable and recoverable with the project. An agent briefing or interface view identifies the source state it summarizes. Generated text becomes authoritative only through the project's acceptance process. Documents, repositories, search, and knowledge tools can supply the parts. The joint benefit is continuity when the author, interface, or host changes, with an ongoing cost for keeping explanations useful.

Replacement and recovery are ordinary operations. A supported backup or export restores a usable project, including the evidence and outstanding work needed to continue it. Replacing a component preserves the agreed behavior at its boundaries. This requires maintained contracts and recovery procedures; merely owning the bytes is insufficient. The benefit must be weighed against the work of maintaining adapters and testing replacements.

These constraints deliberately restrict the available combinations. A tool may be excellent at its local task and still require an expensive adapter to preserve project authority or durable reasons. Another tool with fewer local features may make the complete workflow simpler. Choosing between them requires a concrete fit assessment, not a preference for novelty, small executables, or a particular shape of user interface.

Adopt, configure, adapt, then build the missing part

For each responsibility, record the required behavior and its acceptance check. Identify which existing components or configurations could supply it. Evaluate those candidates against the shared constraints, including failure, recovery, and ongoing operator work. An existing solution that meets the requirement is the default choice.

If configuration is insufficient, identify the smallest missing behavior. An adapter, upstream change, or narrow custom component may supply it. The justification should name the requirement, the candidate examined, the demonstrated gap, and the cost of closing it. A claim that a tool is "platform-shaped" is not a sufficient rejection. A larger existing component may reduce the total system's operating burden.

Adapters deserve the same scrutiny as other custom code. A short bridge that translates authority or acceptance decisions can carry the system's most consequential rules. Its size does not make those rules trivial. Conversely, a simple export transformation need not become a new universal protocol.

When a constraint excludes every practical solution, revisit its purpose. The choice may be to accept the cost, narrow the supported workflow, or revise the constraint. Record that decision explicitly. A constraint earns its place through the end-to-end property it protects.

A complete journey is the decisive test

An agent receives an outcome and reconstructs the relevant design. It changes the project and supplies the required evidence. The system accepts the change under explicit authority and performs an authorized consequence. The primary host then fails. A replacement host restores the project and resolves unfinished work. A fresh author can explain what happened and safely make the next change.

That journey combines familiar capabilities into a demanding whole. Evaluate it incrementally through the scenario ladder, using E9 as the main test and E1-E8 to investigate its failure modes. Compare with the strongest practical existing configuration under the same trust and resource assumptions. A gain in completed work, recovery, understanding, or operator effort can justify the composition. No component has to outperform every alternative in isolation.

The report therefore recommends clarifying the active requirements, choosing a small set of shared constraints, and assembling the smallest complete workflow from established solutions. Build custom mechanisms when a demonstrated gap calls for them. Expand scheduling and federation when a later scenario requires them. Novelty may emerge from this work; it is not an acceptance criterion.

7. Reading guide and glossary

The most useful way to study this material is to move between a primary source, a concrete failure case, and the corresponding part of the-valley. The sequence below is designed to develop judgment about mechanisms rather than familiarity with names. The source registers provide direct links and state which portions were inspected for this report.

7.1 The shortest route through the foundations

For a first sustained pass, prioritize these twelve readings. They span the main intellectual dependencies without requiring every product manual first.

Reading	Source register	Question to carry into the-valley
Parnas, <i>On the Criteria To Be Used in Decomposing Systems into Modules</i>	S01	Which decisions can a component change without forcing other components to change?
Saltzer, Reed, and Clark, <i>End-to-End Arguments in System Design</i>	S02	Which desired result can only the application establish?

Reading	Source register	Question to carry into the-valley
Dolstra, <i>The Purely Functional Software Deployment Model</i> , selected foundations	<u>V01</u>	What makes an execution input explicit, and what remains an assumption?
Mokhov, Mitchell, and Peyton Jones, <i>Build Systems à la Carte</i>	<u>V02</u>	Which part chooses what to do, and which part decides whether a result remains valid?
Kung and Robinson, <i>On Optimistic Methods for Concurrency Control</i>	<u>V11</u>	What must be checked immediately before publishing speculative work?
Torres-Arias et al., <i>in-toto</i>	<u>V15</u>	How can independently performed steps carry a verifiable relationship?
Lamport, <i>Time, Clocks, and the Ordering of Events in a Distributed System</i>	<u>D01</u>	What does an order of events establish about causality?
Helland, <i>Life beyond Distributed Transactions</i>	<u>D13</u>	Who resolves uncertainty after an external action and a local crash?
Naur, <i>Programming as Theory Building</i>	<u>A01</u>	What understanding must a new contributor acquire beyond reading the code?

Reading	Source register	Question to carry into the-valley
van Lamsweerde, <i>Goal-Oriented Requirements Engineering: A Guided Tour</i>	A07	Why should the proposed subgoals be sufficient for the parent goal?
Bainbridge, <i>Ironies of Automation</i>	A25	What must a human retain in order to supervise exceptions competently?
Kapoor et al., <i>AI Agents That Matter</i>	A22	Would an apparent gain survive equal budgets, stronger baselines, and held-out evaluation?

For each reading, write four short notes: the problem it addresses, its assumptions, what it establishes, and one thing it does not establish. Then find a local design sentence that becomes more precise after reading it. This exercise is more valuable than importing the paper's vocabulary into the project unchanged.

7.2 Eight deeper study sessions

Session 1: Ownership and composition

Read Parnas and the end-to-end paper, then compare Radicle, Fossil, SourceHut, and Tangled in [chapter 2](#). Consult S03–S09 for the original system descriptions.

Draw the authoritative stores for one project under each architecture. Mark what a normal clone preserves, what requires another export, and who can decide the accepted branch. Then remove the primary host from the drawing. Identify which functions continue and which require recovery.

The test of understanding is the ability to explain why storage ownership, identity independence, availability, and decision authority can vary separately. Do not rank systems by a single "decentralized" score.

Then choose one end-to-end requirement and assign its responsibilities to existing tools. Construct a case in which every tool passes its own checks while the complete workflow fails. Identify the missing agreement between them. This gives a concrete reason for a shared constraint, an adapter, or a different selection of tools. Use section 6.7 to assess whether that cost is justified.

Session 2: Computation identity and reuse

Read the Nix foundations, *Build Systems à la Carte*, and the selective-testing work in V27. Use the Nix manual and Bazel documents in V04–V08 to connect the abstractions to actual execution controls.

Choose three real checks: a prose formatter, a compiler test, and a network-dependent observation. For each, list all the information required to decide whether an old result still applies. Include the executable definition and facts about absent or newly introduced inputs.

Construct one conservative invalidation and one unsound reuse rule. Explain why the former wastes work while the latter can accept a result for the wrong computation. This prepares the ground for E2 and E3 in chapter 6.

Session 3: Claims, trusted builders, and witnesses

Read in-toto, the in-toto Statement format, DSSE, and SLSA's actual build requirements. Compare the Sigstore security model, TUF, and transparency guarantees in V15–V22. Read Thompson's compiler paper to challenge an overly simple definition of independent verification.

Follow one hypothetical signed check through the system. Ask separately whether its bytes are authentic, its check definition is approved, its result can be reproduced, its signer is authorized, and its test is adequate. Produce an example in which four answers are favorable and the fifth prevents acceptance.

Then use chapter 3's sampling equations with several audit probabilities. State the independence and detection assumptions aloud. Explain the difference between reducing the chance of an undetected pattern and ensuring that one particular claim will be checked.

Session 4: Integration under contention

Read Kung and Robinson, Crystal's collaboration-conflict paper, and the Zuul gating guide. Compare the GitHub and GitLab mechanisms in chapter 2. Keep the 2026 queue-governance preprint as an additional research lead rather than the foundation of the argument.

Work through three changes by hand. Make two textually disjoint changes interact through a shared behavioral assumption. Make a third change alter the check definition. For each possible order, identify the combined tree, reusable evidence, invalidated evidence, and serial decision work.

The goal is to explain why correct ordering, complete computation dependencies, and adequate checks solve different problems. A scheduler cannot discover every omitted semantic dependency merely by choosing a better queue order.

Session 5: History and effect recovery

Read Lamport, the event-sourcing discussion, the controller and Temporal contracts, Helland, and Sagas in D01-D13. Inspect the actual JetStream retention and acknowledgement semantics alongside those readings.

Draw a failure timeline for "accept source, request deployment, remote deployment succeeds, local process crashes." At each interval, write what durable fact exists and what a restarted process can know. Design separate procedures for re-enumerating current state, recovering the decision history, and resolving the uncertain external result.

A strong answer includes a case where the right recovery state is explicitly unknown. Replacing uncertainty with an automatic retry is a policy choice, not an explanation of what happened.

Session 6: Authority and recursive governance

Read complete mediation, Macaroons, SPIFFE/SPIRE, TUF's update rules, and the WebAuthn distinctions in D20–D25. Compare them with the local registry, adoption decision, and self-transparency draft.

Draw the trust boundary around a running agent. Mark every credential and effect path it can reach. Then compare that diagram with its recorded delegation. Any extra path is authority the record alone does not constrain.

Work through a normal policy amendment, a key compromise, an expired key during failed convergence, and a host restore from an old image. Identify who can restore authority and what survives as evidence of the exceptional action. Explain the difference between authorization then and authorization now.

Session 7: Understanding, rationale, and memory

Read Naur, gIBIS, Doyle's truth-maintenance work, and selected agent-memory papers in A01–A17. Add Dung's argumentation model, A28, when considering objections and conflicting reasons. Read these as different problem statements, rather than as interchangeable support for "use a knowledge graph."

Choose a current design decision and write the smallest explanation that would let an unfamiliar engineer defend it, identify a condition that would invalidate it, and propose a compatible extension. Compare that explanation with a chronological conversation transcript and a retrieved summary.

Use the theory-reconstruction experiment to ask what was lost. If the reader gives the right answer, change the situation and see whether the understanding transfers. A memorized phrase should not earn the same score as a justified new decision.

Session 8: Outcomes, supervision, and evaluation

Read goal-oriented requirements, HTN planning, scheduling, the automation papers, and the evaluation work in A07–A10 and A18–A27. Pair the earlier METR study with the later methodology update. Treat benchmark claims as conditional on their tasks, harnesses, and observation periods.

Design an outcome with an incomplete decomposition, an abandoned blocker, two valid alternatives, and a prerequisite shared by two roots. Decide who may revise the acceptance criterion. Add a budget and a cancellation halfway through a run.

Write a completion report that can say "not attained" even when substantial useful work has been done. Then specify the comparison that would show whether automation improved the outcome rather than merely increasing activity.

7.3 Questions for judging a new paper or tool

Use the same questions when extending this report:

- What exact problem is solved, and which local requirement needs that solution?
- Which assumptions make the guarantee possible? Who controls the dependencies and evaluator?
- Is the evidence a proof, a benchmark, a field study, a project description, or a proposal?
- What is the strongest baseline? Are trust, budgets, caches, and human work held comparable?
- Which failure cases are excluded? What happens to existing results after revocation or recovery?
- Does the evaluation measure the desired outcome or a proxy that can improve independently?
- Could the mechanism be adopted as one component? What durable state or protocol would become tied to it?
- What observation would make its adoption a mistake for the current pilot?

7.4 Working glossary

These definitions state how the report uses the terms. They are reading aids, not additions to the project's schema. Follow the relevant chapter for assumptions and primary sources.

Acceptance criterion

Meaning in this report: A stated procedure or condition used to decide whether requested work counts as achieved

Action cache

Meaning in this report: A mapping from an identified computation to result metadata

Attestation

Meaning in this report: An attributable statement about an identified subject; its predicate says what is claimed

Attainment

Meaning in this report: The requested condition is accepted as holding in a specified scope and environment

Authority

Meaning in this report: Permission to make a particular decision or perform an action at an enforcement boundary

Canonical state

Meaning in this report: The project state accepted under the relevant governance rules

Causality

Meaning in this report: A relationship of influence or dependence; its exact meaning differs between execution order and incident explanation

Complete mediation

Meaning in this report: Checking authority across every relevant access path within a declared boundary

Content address

Meaning in this report: An identifier derived from the content it names

Critical path

Meaning in this report: A duration-dependent chain constraining completion under a specified scheduling model

CRDT

Meaning in this report: A data type whose defined concurrent operations support convergence under specified assumptions

Delegation

Meaning in this report: A grant of authority from one actor to another, with a scope that must be enforced

Determinism

Meaning in this report: The same specified computation and inputs yield the same result

Effect

Meaning in this report: A change outside a computation's returned value, such as updating a service or sending a message

Effect attempt

Meaning in this report: One execution attempt toward a logical effect; retries need not represent new intended effects

Evidence transfer

Meaning in this report: Accepting a prior result as applicable to the candidate state under a stated rule

Hermeticity

Meaning in this report: Restricting a computation's observations to its specified environment and dependencies

Idempotency

Meaning in this report: Repetition does not repeat the substantive state change the application seeks to perform

Inclusion proof

Meaning in this report: Evidence that an entry belongs to a particular committed log view

Input closure

Meaning in this report: The dependencies required by a computation, under a specified dependency model

Invalidation

Meaning in this report: A reason that earlier evidence or a derived conclusion can no longer be used as before

Materialized view

Meaning in this report: Stored results derived from authoritative facts for convenient access

Optimistic concurrency control

Meaning in this report: Doing work speculatively and validating its assumptions before publishing its effect

Oracle

Meaning in this report: The procedure used to judge correctness or success in an evaluation

Outcome

Meaning in this report: A condition someone wants to hold; distinct from the attempt intended to produce it

Provenance

Meaning in this report: Recorded relationships among inputs, activities, outputs, and responsible actors

Readiness

Meaning in this report: Satisfaction of the conditions that permit an attempt to begin

Reconciliation

Meaning in this report: Repeatedly comparing durable desired state with observations and acting on remaining differences

Reproducibility

Meaning in this report: Ability to recreate specified artifacts from identified source, environment, and instructions

Revocation

Meaning in this report: Withdrawal of authority or acceptability under a policy; it does not erase past external effects

Saga

Meaning in this report: A sequence of committed steps with compensating actions for relevant failures

Sound reuse

Meaning in this report: Reuse that preserves the result of the specified approved computation under its assumptions

Speculative gating

Meaning in this report: Checking possible combined future states before their final integration order is completed

Theory reconstruction

Meaning in this report: Acquiring enough project understanding to explain and modify it appropriately

Transactional outbox

Meaning in this report: Recording a state change and the obligation to publish it within one atomic boundary

Transparency

Meaning in this report: Making a defined class of actions or claims externally inspectable under an explicit retention and trust model

Trusted computing base

Meaning in this report: The components and assumptions whose failure can invalidate a stated guarantee

Witness

Meaning in this report: An independent participant that checks or observes a specified claim; independence must be defined

The most important distinctions to retain are authenticity versus truth, provenance versus cause, readiness versus attainment, current state versus event history, and replay versus repeating an external effect. Many attractive designs become easier to assess once those words stop doing each other's work.

Sources S: architecture and existing systems

These entries support chapters [1](#) and [2](#). All external sources were accessed on 2026-09-20. "Selected sections" means that the cited mechanism was read in the source; it does not mean that the complete manual or paper was reviewed. Documentation establishes a project's stated behavior, not independently measured reliability. No product in this register was deployed or benchmarked for this report.

S01 — Information hiding

D. L. Parnas. **On the Criteria To Be Used in Decomposing Systems into Modules.** *Communications of the ACM* 15(12), 1053-1058, 1972. [DOI](#); [paper transcription](#).

Read: abstract, worked decomposition discussion, criteria, and implementation tradeoffs. The paper provides a criterion for replaceable components: hide decisions likely to change. It is conceptual design analysis, not evidence that microservices reduce operating cost. Apply it to the-valley's evidence, transport, executor, and storage boundaries.

S02 — End-to-end responsibility

J. H. Saltzer, D. P. Reed, and D. D. Clark. **End-to-End Arguments in System Design**. *ACM Transactions on Computer Systems* 2(4), 277-288, 1984. [Author-hosted paper](#).

Read: introduction, file-transfer argument, performance discussion, and delivery guarantees. The paper explains why application-level knowledge can remain necessary despite reliable lower layers. It supports an analogy for the-valley, not a theorem about its architecture. Use it to question whether bus delivery, signatures, or check success establish the desired outcome.

S03 — Radicle

Radicle contributors. **Radicle Protocol Guide**. Living Heartwood protocol guide. [Source](#).

Read: repository identity, signed refs, canonical branches, storage, collaborative objects, and concurrency. It is close prior art for authenticated collaboration objects replicated with Git. The guide is a project description, not an independent durability evaluation. Compare its acceptance and object models before inventing parallel social-record protocols.

S04 — Fossil

Fossil contributors. **Fossil Versus Git**. Living project documentation. [Source](#).

Read: introduction and sections on integrated facilities and self-contained deployment. It supplies a counterexample to treating many separate programs as the only route to minimalism. Its comparative claims are explicitly project-authored and favorable to Fossil. Use the architecture as a baseline; measure operating cost independently.

S05 — Forgejo

Forgejo contributors. **Forgejo CLI**, v15.0 documentation. [Source](#).

Read: command overview, dump, dump-repo, and restore-repo descriptions. The export model illustrates the distinction between owning a forge and representing every durable artifact in Git. Backup command documentation alone does not establish a consistent backup or successful restore. This is the conventional self-hosted comparison for S1.

S06 — SourceHut

Drew DeVault. **Announcing the SourceHut project hub**. 2020; page displays April 29, URL April 30. [Source](#).

Read: full announcement. It describes composing independently useful development services under a project view. The announcement establishes the design intent, not a present performance comparison. Its relevance is separating concern ownership from the coherence of the user interface.

S07 — b4 and email contribution

b4 contributors. **am, shazam: retrieving and applying patches**. Living end-user documentation. [Source](#).

Read: retrieval flow, review trailers, attestation checks, and selected options. It demonstrates structured contribution through a transport independent of target-repository write access. Patch authentication is not execution attestation. Borrow the separation of proposal, attribution, and acceptance rather than assuming the email interface itself is required.

S08 — Tangled's component model

The Tangled Contributors. **Tangled docs**, overview. Living documentation. [Source](#).

Read: overview of knots, AT Protocol, and the appview. It is useful comparative architecture for separating hosting from a collaboration view. The page's displayed update date does not pin all linked documentation to that date. No federation or recovery behavior was independently tested.

S09 — Tangled's execution model

The Tangled Contributors. **Spindles**. Living documentation. [Source](#).

Read: pipeline format, triggers, engines, and caching sections. Nixery and microVM execution make it a nearby design, while its workflow format differs from the-valley's runtime constraint. Nix-related execution alone should not be interpreted as a portable integration-evidence protocol.

S10 — Gerrit's change model

Gerrit contributors. **Changes**. Living documentation, page displayed v3.14.2-892-g1ff12a23c6 when accessed. [Source](#).

Read: change identity, patch sets, related changes, and submit strategies. It distinguishes a logical review unit from the commits that revise it. It does not establish portability of every review artifact. Use it to separate proposal identity, content identity, and integration-attempt identity.

S11 — Gerrit's governed configuration

Gerrit contributors. **Submit Requirements**. Same living documentation family as S10. [Source](#).

Read: configuration, testing requirements, and policy-lockout recovery discussion. It shows that reviewing the rules that govern review already has practical precedent. Its administrative recovery paths are specific to Gerrit; the-valley still needs its own explicit trust and recovery model.

S12 — Zuul

Zuul contributors. **Project Gating**. latest documentation at access date. [Source](#).

Read: parallel speculative testing, window behavior, cross-project queues, and dependencies. It is the strong comparison for concurrent checks with ordered integration. Performance depends on failure and dependency patterns; this report measured none. It prevents an unfair comparison against only a serial CI gate.

S13 — GitHub merge queues

GitHub. **Managing a merge queue**. Living product documentation. [Source](#).

Read: merge groups, CI triggers, concurrency settings, and queue-reordering examples. This establishes that speculative combined-state validation is available in conventional hosting. Availability and plan restrictions may change; the report makes no procurement recommendation or benchmark claim.

S14 — GitLab merge trains

GitLab. **Merge trains**. Living product documentation. [Source](#).

Read: combined-state pipeline model, parallel execution, failure removal, and redundant-pipeline cancellation. It provides another established comparison for concurrency without unsafe independent merges. Its behavior does not imply the-valley's evidence transfer is unnecessary; the mechanisms avoid different kinds of repeated work.

S15 — Dagger

Dagger contributors. **Introduction**. Documentation displayed 1.0-beta at access date. [Source](#).

Read: complete introductory page. It states the portable execution-DAG and local/CI reuse model. This is a vendor introduction, not a measured performance result or a detailed security contract. Its role here is to identify a credible baseline for sharing the development and verification runtime.

S16 — Bazel caching

Bazel contributors. **Remote Caching**. Living documentation. [Source](#).

Read: action-cache/content-store distinction, execution flow, and security considerations. It clarifies the identities needed for computational reuse. A cache hit still depends on the action model and trust in cache writers. The-valley adds an acceptance-policy question to that computational question.

S17 — Bazel hermeticity

Bazel contributors. **Hermeticity**. Living documentation. [Source](#).

Read: full substantive page. It describes isolation, declared tool dependencies, and common sources of nondeterministic or host-dependent behavior. A documented goal is not proof that an arbitrary project meets it. Use these failure classes in executor conformance experiments.

S18 — Jujutsu

Jujutsu contributors. **Operation log**. Living latest documentation. [Source](#).

Read: full substantive page. Repository-operation history is distinct from source-commit history and supports inspection and recovery. Its local operational purpose does not establish shared audit authority or indefinite retention. It informs the-valley's history model without requiring a VCS migration.

S19 — Pijul

Pijul contributors. **Theory**. Living manual. [Source](#).

Read: graph representation, change dependencies, conflicts, and CRDT discussion. It offers a different model of changes and their composition. Its merge properties concern the representation of edits, not arbitrary program correctness. Keep that boundary when applying it to verification reuse.

S20 — Proactive collaboration conflicts

Yuriy Brun, Reid Holmes, Michael D. Ernst, and David Notkin. **Proactive Detection of Collaboration Conflicts**. ESEC/FSE, 168-178, 2011. [Author-hosted paper](#).

Read: introduction, speculative-analysis explanation, relationship taxonomy, and selected study methods/results. It distinguishes text, build, and test conflicts and presents Crystal. Historical repositories and available tests limit transport of measured rates to today's agents. Its mechanism motivates earlier interaction warnings, alongside admission control.

S21 — BulkPR-Bench

Zetong Xiong et al. **BulkPR-Bench: Benchmarking Queue-Level Governance of Interacting Pull Requests**. arXiv:2608.02685v1, August 3, 2026. [Versioned text](#).

Read: abstract, task framing, selected results, and discussion/limitations. The useful distinction is between partial relation-level success and whole-queue safety. Constructed pools, fixed scaffolding, and limited repeats constrain generalization. Treat as a preprint, irrespective of the manuscript's future conference label. It is a candidate evaluation lead, not evidence of production reliability.

S22 — What review accomplishes

Alberto Bacchelli and Christian Bird. **Expectations, Outcomes, and Challenges of Modern Code Review.** ICSE, 712-721, 2013. [Research publication page.](#)

Read: author-institution abstract and bibliographic record only; an attempted author-hosted PDF was unavailable through the browser. The cited findings are limited to that abstract. It reports benefits including understanding and knowledge transfer in studied Microsoft teams. The report uses it to identify functions to preserve, not to quantify a universal effect.

S23 — Concurrent agent contributions

George Xu, Arjun Subramanian, and Nithilan Karthik. **AI Agent Pull Requests on GitHub: Frequency, Structure, and Merge Conflict Rates.** arXiv:2607.04697v2, July 7, 2026. [Versioned text.](#)

Read: abstract, study framing, discussion, and threats to validity. It studies co-activity and textual merge replay. Observation is limited to the selected public dataset and earlier collection period; time costs and semantic failures are not measured. Its more sweeping conclusions should not be imported as findings. It motivates measuring the actual pilot's conflict distribution.

Sources V: verification and integration

These sources support Verification that can travel with a change. IDs are local to this report. All external resources were accessed on **2026-09-20**. Publication dates identify papers; documentation entries are living documents unless a version is specified. “Read” records the portions examined, not a claim that every cited work was read cover to cover. The annotations distinguish a source's contribution from this report's assessment of its relevance.

The search followed three connected lines: sound reuse of computation, authentication of claims about computation, and validation at an integration boundary. Academic papers supplied models and limits; official documentation supplied operational contracts. No citation count, product marketing claim, or search-result summary alone was treated as proof of a system's security.

V01 — Nix's deployment model

The Purely Functional Software Deployment Model — Eelco Dolstra, PhD thesis, Utrecht University, 2006.

Read: contents and targeted passages on trust, content addressing, and purity, especially §6.2-6.3 and §7.1.3-7.1.5.

Contribution: makes functional deployment and dependency identity the basis for reliable reuse. **Limitation:**

a foundational model and implementation account, not proof of every current Nix build's determinism. **Relevance:** the-valley's equality argument should retain the purity assumptions that accompany the original model.

V02 — Build-system decomposition

Build Systems à la Carte — Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones, *Proceedings of the ACM on Programming Languages*, ICFP, 2018.

Read: task and build abstractions, dynamic dependencies, and scheduler/rebuilder sections, including concrete systems. **Contribution:** separates execution order from deciding whether results need replacement. **Limitation:** its computational model is not an adversarial attestation protocol. **Relevance:** supplies a clearer decomposition for integration scheduling, evidence validation, and caching.

V03 — Dynamic dependency tracking

An Experimental Analysis of Self-Adjusting Computation — Umut A. Acar, Guy E. Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan, *ACM TOPLAS* 32(1), article 3, 2009.

Read: abstract, introduction, and overview of dynamic dependence graphs and change propagation. **Contribution:** combines dependency tracking and memoization to reuse parts of changing computations. **Limitation:** selected computations and an instrumented model; no arbitrary-shell

completeness guarantee. **Relevance:** exposes the tradeoff between dependency precision, instrumentation cost, and reuse.

V04 — Nix's actual sandbox and cache controls

Nix 2.28.8 Reference Manual: nix.conf — Nix contributors, versioned documentation.

Read: configuration precedence, require-sigs, sandbox, sandbox-fallback, and sandbox-paths. **Contribution:** specifies concrete isolation and substitution controls, including exceptions. **Limitation:** documentation does not establish how a particular host is configured or whether it is trustworthy. **Relevance:** runner identity must include or constrain the execution boundary; invoking Nix alone does not attest those settings.

V05 — Evaluation can trigger builds

Nix 2.28.8 Reference Manual: Import From Derivation — Nix contributors, versioned documentation.

Read: complete explanatory text, example, and diagrams. **Contribution:** describes evaluation pausing to realize a store object and the switch that disables this. **Limitation:** establishes a possible behavior, not that the-valley's current checks exercise it. **Relevance:** the integrator's "builds nothing" contract needs an enforced runner restriction and a regression case.

V06 — What a Nix content address means

Nix 2.28.8 Reference Manual: Content-Addressing Store Objects — Nix contributors, versioned documentation.

Read: complete page, including references, self-references, and addressing methods. **Contribution:** specifies content identity and distinguishes content-addressed from input-addressed objects. **Limitation:** a store-object identity is not an execution proof. **Relevance:** prevents overinterpreting a digest of derivation reference names as a measurement of all runtime input bytes.

V07 — Practical hermeticity

Hermeticity — Bazel contributors, living documentation.

Read: definition, benefits, non-hermeticity examples, and troubleshooting. **Contribution:** identifies concrete environmental dependencies that undermine reuse. **Limitation:** engineering guidance, not a formal guarantee for every Bazel action. **Relevance:** a useful checklist for constructing adversarial dependency tests, especially around host tools, timestamps, and source-tree writes.

V08 — Action results versus stored bytes

Remote Caching — Bazel contributors, living documentation.

Read: overview of actions and cache storage, plus cache backend and access-control passages. **Contribution:** separates the action-result mapping from content-addressed output storage. **Limitation:** shared caching relies on suitable reproducibility and access assumptions. **Relevance:** a digest can authenticate output bytes while leaving the claimed connection to a computation dependent on another authority.

V09 — Reproducibility's scope

Definitions — Reproducible Builds project, living documentation.

Read: full definition and explanations. **Contribution:** defines reproducibility in terms of specified source, environment, instructions, and artifacts. **Limitation:** the definition itself provides neither a measurement nor a guarantee. **Relevance:** separates artifact agreement from incidental logs, execution truth, and functional correctness.

V10 — Historical Nix rebuildability

Reproducibility of Build Environments through Space and Time — Julien Malka, Stefano Zacchiroli, and Théo Zimmermann, ICSE-NIER, 2024; DOI 10.1145/3639476.3639767.

Read: abstract, introduction, definitions, and targeted methods/results/failure passages. **Contribution:** measures historical environment reconstruction and successful

rebuilding. **Limitation:** reuses cached dependencies, leaves bit-for-bit artifact reproducibility for later work, and gives preliminary failure classifications. **Relevance:** supports practical optimism about functional deployment without establishing full source-closure rebuildability or deterministic outputs.

V11 — Optimistic validation

On Optimistic Methods for Concurrency Control — H. T. Kung and John T. Robinson, *ACM TODS* 6(2), 1981; DOI 10.1145/319566.319567.

Read: read/write phases, serial-equivalence validation, and passages on moving work outside the critical section.

Contribution: validates speculative transactions before publishing their writes. **Limitation:** assumes transactions preserve the specified integrity conditions. **Relevance:** provides the integration analogy and its often omitted precondition: checks must actually establish the properties being preserved.

V12 — Conflicts beyond text

Proactive Detection of Collaboration Conflicts — Yuriy Brun, Reid Holmes, Michael D. Ernst, and David Notkin, *ESEC/FSE*, 2011.

Read: abstract and introductory study/Crystal discussion; dataset not reanalyzed. Contribution and limitations: S20.

V13 — Speculative project gating

Project Gating — Zuul contributors, living documentation.

Read: speculative testing, failed predecessors, windows, and cross-project dependencies. Contribution and limitations: S12.

V14 — A hosted merge-queue baseline

Managing a merge queue — GitHub, living documentation.

Read: overview, required-check behavior, and operation description. **Contribution:** validates candidates with the target branch and preceding queued work. **Limitation:** documented product behavior is not a workload-independent performance result. **Relevance:** comparisons with hosted CI should include contemporary merge queues rather than assume every forge serially reruns isolated branches.

V15 — End-to-end workflow integrity

in-toto: Providing farm-to-table guarantees for bits and bytes — Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos, USENIX Security, 2019.

Read: introduction, definitions, threat model, security goals, and project-owner-compromise passage. **Contribution:** binds authorized supply-chain steps and artifact flows through signed metadata. **Limitation:** compromised policy

authority or unsuitable authorized checks remain fundamental limits. **Relevance:** close prior art for composing evidence across independently operated tools.

V16 — Typed claims about immutable subjects

Statement layer specification, v1 — in-toto contributors, living specification.

Read: schema and subject/predicate field semantics.

Contribution: binds a typed predicate to artifact digests.

Limitation: predicate meaning and acceptance policy remain external to the generic statement layer. **Relevance:** the pure/effectful distinction can be expressed interoperably rather than treated as an objection to typed supply-chain attestations.

V17 — SLSA's actual requirements

SLSA v1.2: Build requirements — SLSA community/OpenSSF, approved versioned specification.

Read: requirements table, provenance generation, and isolation. **Contribution:** defines concrete Build-level obligations. **Limitation:** conformity requires deployment assessment. **Relevance:** local signatures and sampling do not justify the repository's approximate Level 3 claim.

V18 — Signing without confusing payload types

DSSE Protocol — DSSE contributors, living specification.

Read: pre-authentication encoding, signing, verification, and multi-signature procedures. **Contribution:** binds payload type and exact payload bytes in a signature envelope.

Limitation: says nothing about the truth of the payload.

Relevance: clarifies what canonical serialization solves and what an interoperability adapter must preserve.

V19 — Identity and transparency services

Security Model — Sigstore contributors, living documentation.

Read: identity, trust-root, Fulcio, certificate-transparency, and short-lived-certificate explanations. **Contribution:** connects identity-based signing with an auditable history.

Limitation: identity and transparency do not establish computation correctness; monitoring remains necessary.

Relevance: an optional signing ecosystem, with trust and operational choices separate from the-valley's evidence semantics.

V20 — Updating trust safely

The Update Framework Specification — TUF contributors, living specification.

Read: roles and delegation, client update/expiration passages, and key-rotation procedures. **Contribution:** addresses secure delivery of trusted metadata amid replay, staleness, and partial compromise. **Limitation:** cannot make an authorized artifact correct or recover automatically from every root compromise. **Relevance:** authoritative revocation distribution needs more than appending a negative statement to a log.

V21 — Transparency's explicit limits

RFC 9162: Certificate Transparency Version 2.0 — Ben Laurie, Eran Messeri, and Rob Stradling, IETF Experimental RFC, 2021.

Read: overview and §11 security considerations, especially detection and misbehaving logs. **Contribution:** distinguishes logged evidence, monitoring, and inconsistent log views. **Limitation:** a certificate protocol; direct reuse requires adaptation, and some gossip mechanisms are outside its scope. **Relevance:** inclusion is not correctness, completeness of observation, or agreement on one history.

V22 — A transparency implementation component

Tessera — transparency.dev contributors, living repository documentation.

Read: README lifecycle sections on sequencing, integration, and checkpoint publication. **Contribution:** supplies a tile-based log library with distinct durability/publication stages and witness-policy support. **Limitation:** does not choose application acceptance or outage policy. **Relevance:** the-valley must specify which completed stage authorizes an action and how incomplete publication is recovered.

V23 — Evidence, appraisal, and freshness

RFC 9334: Remote ATtestation procedureS (RATS) Architecture — Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan, IETF Informational RFC, 2023.

Read: role definitions, conceptual messages, trust relationships, and freshness passages. **Contribution:** separates evidence production from appraisal and reliance. **Limitation:** architecture rather than a universal hardware or protocol guarantee. **Relevance:** clarifies the additional boundary needed for measured execution and the policy nature of freshness.

V24 — Common-mode toolchain compromise

Reflections on Trusting Trust — Ken Thompson, *Communications of the ACM* 27(8), 1984; DOI 10.1145/358198.358210.

Read: compiler-backdoor construction and concluding discussion. **Contribution:** demonstrates a compromise that reproduces through a compiler without remaining in inspected source. **Limitation:** a construction illustrating possibility, not a prevalence measurement. **Relevance:** independent witnesses sharing one compromised toolchain do not provide independence against that compromise.

V25 — The meaning of a passing test

The Oracle Problem in Software Testing: A Survey — Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo, *IEEE Transactions on Software Engineering* 41(5), 2015.

Read: abstract, introduction, and metamorphic-relations discussion. **Contribution:** organizes ways of deciding whether observed behavior is correct. **Limitation:** does not supply an automatic universal oracle. **Relevance:** reliable evidence of execution must be distinguished from adequate evidence of requirement satisfaction.

V26 — Nondeterministic tests

An Empirical Analysis of Flaky Tests — Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov, FSE, 2014.

Read: abstract and introduction describing the study and failure implications. **Contribution:** examines 201 likely flaky-test-fixing commits across 51 projects. **Limitation:** historical sample; this report does not extrapolate its prevalence to the-valley. **Relevance:** repeated-run disagreement can expose unstable tests rather than dishonest signers, requiring diagnosis before reputation changes.

V27 — Safe selection is relative to an existing test suite

A Safe, Efficient Regression Test Selection Technique — Gregg Rothermel and Mary Jean Harrold, *ACM TOSEM* 6(2), 1997; DOI 10.1145/248233.248262.

Read: abstract and analytical evaluation of safety, precision, efficiency, and generality. **Contribution:** provides conditional safety guarantees for selecting regression tests. **Limitation:** controlled-regression assumptions and the original suite's coverage remain essential. **Relevance:** evidence reuse needs a no-missed-retest argument; it cannot inherit a no-missed-bug guarantee.

Local implementation evidence and unresolved work

The chapter inspected attestation execution⁷⁹, Nix invocation and dependency collection⁸⁰, digest construction⁸¹, signature verification⁸², evidence gathering⁸³, policy composition⁸⁴, verdict rules⁸⁵, ref publication⁸⁶, transfer statements⁸⁷, and the attestation schema⁸⁸. The reading was static code analysis, not penetration testing or a formal verification of those programs.

The implementation findings are deliberately narrower than a complete audit. Same-tree computation binding, effectful-command binding, future timestamps, and the landing/evidence crash boundary are proposed regression or fault-injection cases. No exploit success rate or deployment-wide exposure is claimed. The source set establishes strong precedents but does not establish novelty of the-valley's particular combination; that would require a more systematic survey of research prototypes and industrial integration systems.

Sources D: events, trust, and durability

This register supports Events, trust, and durability. All links were checked on 2026-09-20. “Sections” means the named portions were examined; it does not imply a full reading of the paper, specification, proofs, implementation, or linked references. None of the systems below was deployed or benchmarked for this report. Source annotations identify both what is useful and what the source does not establish for the-valley.

D01 — Logical causality

Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM 21(7), 558-565.

Read: sections on partial ordering, logical clocks, and total ordering, including the clock-condition distinction. Establishes the difference between causal precedence and a chosen total order. Essential for interpreting “one history.” It does not supply a model of incident responsibility or establish completeness of observed events. Read before designing causal references across valleys.

D02 — Event sourcing and replay

Martin Fowler. 2005. Event Sourcing. Author's architecture essay, explicitly marked draft.

Read: definition, reconstruction discussion, external updates, external queries, and code changes. Useful for distinguishing replayable domain state from repeating real-world effects. This is a primary design account, not a formal correctness result or current product specification. Its external-system discussion is particularly relevant to the actuator proposal.

D03 — Ref history and expiration

Git project. git-reflog. Living command documentation, accessed 2026-09-20.

Read: description and expiration options. Documents the operational log of ref updates and its retention controls. Useful when assessing whether repository state can reconstruct earlier ref transitions. A reflog's existence on one host is not evidence that the project's backup or synchronization path retains it. The report does not recommend changing global reflog policy without an explicit history contract.

D04 — Ref transactions

Git project. git-update-ref. Living command documentation, accessed 2026-09-20.

Read: expected old values, --stdin, transaction commands, and concurrent-reader warning. Supplies the relevant existing primitive for coordinating target, evidence, and journal refs. Its caveat about readers observing a subset matters to the proposed landing protocol. Further implementation-specific fault testing is required; this documentation alone does not establish arbitrary power-loss guarantees for every ref backend.

D05 — Stream storage contracts

NATS project. Streams. Official documentation repository, accessed 2026-09-20.

Read: publication acknowledgement, retention, discard policy, and configuration table. Documents the choices behind a “durable stream.” The official repository was archived on 2026-08-24; the historical documentation is linked because former documentation URLs redirected during research. Check these guarantees against the deployed server and CLI versions before implementation. No deployment-specific performance claim is inferred.

D06 — Deduplication and acknowledgements

NATS project. JetStream Model Deep Dive. Official archived documentation, accessed 2026-09-20.

Read: message deduplication, acknowledgement modes, exactly-once semantics, and starting positions. Explains the actual scope of the transport mechanisms. Particularly useful for resisting the inference that acknowledging a message also commits a remote effect. This describes the documented mechanism and default duplicate window, not every newer server feature or configuration.

D07 — Consumer recovery state

NATS project. Consumers. Official archived documentation, accessed 2026-09-20.

Read: consumer model, delivery guarantees, redelivery, and push/pull overview. Establishes that consumers maintain delivery and acknowledgement state. Relevant to deciding what replay restores and what a restarted subscriber must remember. It does not establish that application processing is idempotent or that an external action and acknowledgement are atomic.

D08 — Reconciliation

Kubernetes project. Controllers. Living documentation, accessed 2026-09-20.

Read: controller pattern, API-server control, direct external control, and desired/current-state discussion. Provides the operational precedent for the implemented polling integrator and proposed standing-demand controller. This is

a conceptual description rather than a proof that any particular controller converges. It supports borrowing the pattern without adopting the Kubernetes platform.

D09 — Durable workflow history

Temporal project. Events and Event History. Living documentation, accessed 2026-09-20.

Read: event and activity-event definitions, persisted history, history limits, and recovery discussion. Explains what Temporal retains to recover an execution. Useful as a comparison for any custom operation journal. The chapter deliberately avoids relying on exact current limits as a design constant. It does not evaluate Temporal's operational cost in this project's deployment.

D10 — Activity failures and cancellation

Temporal project. Activity Execution. Living documentation, accessed 2026-09-20.

Read: lifecycle, timeouts, retries, cancellation, and external asynchronous completion discussion. Makes the boundary between durable orchestration and fallible activity execution explicit. Relevant to deployment and agent attempts. It does not promise exactly-once behavior in an arbitrary external service or that cancellation reverses an effect already accepted there.

D11 — Transactional outbox

Microsoft / Azure Samples. 2026. Azure Cosmos DB design pattern: Transactional Outbox. Published 2026-07-06; accessed 2026-09-20.

Read: pattern explanation, transaction boundary, relay, and duplicate-consumer discussion. Shows the local atomicity requirement behind reliable publication. The concrete implementation is Cosmos DB specific; the report transfers the pattern, not its product guarantees, to a possible Git journal. Example code and its crash demonstration were not executed.

D12 — Compensating long operations

Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. ACM SIGMOD, 249-259.

Read: abstract and introductory long-transaction and reservation examples. Establishes the motivation for committed subtransactions with compensation. Useful for deployments and coordinated releases that cannot remain one atomic transaction. This chapter uses the conceptual distinction; it does not claim to have reviewed the full recovery algorithm or proved that a proposed deployment compensation preserves an invariant.

D13 — Independent transaction scopes

Pat Helland. 2007. Life beyond Distributed Transactions: an Apostate's Opinion. CIDR.

Read: message-versus-method discussion and §5 on retries, idempotence, and substantive behavior. A strong engineering account of obligations that reappear when transactional scope is local. Relevant even at small scale because a local process and remote actuator already cross a transaction boundary. Its large-scale framing is not evidence that the-valley needs large-scale infrastructure.

D14 — A provenance vocabulary

Luc Moreau and Paolo Missier, editors; W3C Provenance Working Group. 2013. PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.

Read: overview, attribution, association, delegation, influence, and bundles. Useful for separating production claims and authority. Representation does not establish truth or enforce permissions. Also registered as A04.

D15 — Operational correlation

OpenTelemetry project. Tracing API. Living specification, accessed 2026-09-20.

Read: span context, identifiers, sampled flags, recording, and links. Useful for correlating execution detail with retained project decisions, including operations with multiple predecessors. A tracing API is not a durable audit-store contract. The report does not equate possession of a trace ID with authenticated provenance or assume all spans are recorded.

D16 — Ownership and local operation

Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-First Software: You Own Your Data, in spite of the Cloud. Onward! DOI: 10.1145/3359591.3359737.

Read: motivation, seven-ideals framing, and selected storage/synchronization comparison sections. A close conceptual predecessor to repo-carried project knowledge and provider independence. It is a design and research agenda, not evidence that CRDTs solve every governance conflict. The report does not claim a complete review of the prototypes or their usability results.

D17 — Convergence guarantees

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free Replicated Data Types. SSS 2011, 386-400.

Read: introduction and §2 system model, strong eventual consistency, and state-based convergence conditions. Clarifies which assumptions support automatic merging. Relevant to collaborative metadata and federation. The cited model assumes non-Byzantine processes; convergence alone does not establish authorization or arbitrary application invariants. Graph constructions and omitted companion proofs were not independently checked.

D18 — Git-backed decentralized collaboration

Radicle project. Radicle Protocol Guide. Living protocol guide, accessed 2026-09-20.

Read: identity, delegates, signed references, collaborative objects, and concurrency. Relevant to portable governance; compatibility with the-valley remains untested. Also registered as S03.

D19 — Integrated project memory

Fossil project. Fossil Versus Git. Living project documentation, accessed 2026-09-20.

Read: features, packaging, and export limitations. Relevant to operational simplicity; project-authored comparisons are not independent benchmarks. Also registered as S04.

D20 — Enforcement boundaries

Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems: Basic Principles. Proceedings of the IEEE 63(9), 1278-1308. Author-hosted transcription.

Read: design principles, especially complete mediation, least privilege, and separation of privilege. The foundational test for whether the registry and actuator constrain real access paths. The paper is much broader than the selected excerpt. It does not supply a ready-made delegated-agent protocol or remove the need to define a trusted computing base.

D21 — Attenuated delegation

Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. 2014. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. NDSS.

Read: introductory construction and examples, caveat verification, holder-of-key discussion, and revocation/versioning. Highly relevant to monotonic delegation. The bearer-token default differs from the registry's stated credential policy. Formal security arguments, implementation details, and performance results were not independently audited. The recommendation concerns the semantics before the token format.

D22 — Workload authentication

SPIFFE project. SPIFFE Overview. Living documentation, accessed 2026-09-20.

Read: standards overview, short-lived identity documents, and Workload API description. Identifies an existing interface for workload authentication across heterogeneous infrastructure. It is not an authorization policy for projects or delegated agent tasks. The ecosystem table was not used as a product-selection ranking, and no individual implementation's feature coverage was verified.

D23 — Identity issuance machinery

SPIFFE / SPIRE project. SPIRE Concepts. Living documentation, accessed 2026-09-20.

Read: server/agent architecture, registration entries, node attestation, and workload attestation. Supplies a concrete comparison for the deferred certificate-issuance service. Potentially compatible with registry-derived policy, but that integration is a report proposal. Security depends on the chosen attestors, host boundary, and issuance policy; those have not been selected or tested here.

D24 — Trust succession

The Update Framework project. Specification, version 1.0.36 (2026-08-05).

Read: §§5.2–5.3, trusted roots and updates. Relevant to succession; initial trust remains separate. Also registered as V20.

D25 — Human authentication properties

W3C Web Authentication Working Group. 2026. Web Authentication: An API for accessing Public Key Credentials, Level 3. W3C Recommendation, 25 August 2026.

Read: scope, single/multi-device terminology, backup eligibility, authenticator flags, and assertion-binding structure. Corrects overly broad equations between passkeys, non-exportability, presence, and approval. The standard is large; this is a targeted reading. No approval envelope, authenticator policy, or browser implementation was validated for the-valley.

D26 — What a clone preserves

Git project. git-clone. Living command documentation, accessed 2026-09-20.

Read: ordinary, bare, and mirror cloning options, especially custom-ref coverage. Important for backup acceptance tests because attestations and requests live outside ordinary branch names. This documents command behavior rather than the repository's actual backup completeness. Neither a successful clone nor a configured mirror demonstrates a verified disaster recovery.

D27 — Replication and agreement

Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC extended paper.

Read: leader election, log replication, commitment/safety discussion, and membership-change explanation. Provides the right scope for claims about agreed replicated state. It is not a prescription to add Raft to this project. The report did not re-prove safety or use the paper as evidence for protection against malicious administrators or backup deletion.

D28 — Actual cause and model assumptions

Joseph Y. Halpern and Judea Pearl. 2005. Causes and Explanations: A Structural-Model Approach. Part I: Causes. British Journal for the Philosophy of Science 56(4), 843-887; author manuscript dated 24 October 2005.

Read: abstract, introduction, and structural-model discussion, especially variable selection and counterfactual interpretation. Useful for distinguishing causal explanation from provenance. This is one influential formal account, not a claim that causation has a single settled definition. Later examples, proofs, and competing revisions were outside this chapter's reading.

Repository evidence

The primary local evidence was the requirements⁸⁹, architecture⁹⁰, roadmap⁹¹, open questions⁹², and self-transparency draft⁹³. The implementation reading covered event and identity schemas, ref replay, publication, the integrator's landing and evidence paths, and registry rendering and installation. The chapter links those files at the corresponding findings.

The signal contracts⁹⁴, identity registry⁹⁵, valley repository⁹⁶, agent provenance⁹⁷, delegation⁹⁸, actuator⁹⁹, and production-DAG¹⁰⁰ nodes informed the interpretation of intended behavior. An idea node is evidence of a proposal, not evidence that its mechanism exists.

Sources A: knowledge, agents, and outcomes

This register supports Chapter 5. All links were checked on 2026-09-20. “Selected sections” means the named portions were read; it does not mean the entire paper or its proofs were reviewed. Abstract-only entries support correspondingly narrow descriptions. Living documentation establishes what a project documents, not an independently verified performance claim. Limitations and proposed applications below are this report's assessments unless attributed.

A01 — Programming as Theory Building

Peter Naur, 1985. *Microprocessing and Microprogramming*. Paper, in a later reprint; university-hosted scan.

Read: selected sections of the reprint: introduction, modification, and program life, death, and revival.

Contribution: a sharp account of understanding that exceeds code and documentation. **Limit:** conceptual argument and historical examples, not evidence about LLM-based reconstruction. **Relevance:** the strongest challenge to the graph's theory-revival ambition. Read before designing its evaluation; success should mean useful new work, not reproducing an allegedly complete mental state from text.

A02 — gIBIS: A Hypertext Tool for Exploratory Policy Discussion

Jeff Conklin and Michael L. Begeman, 1988. *ACM CSCW*, 140-152. [DOI](#); [author-uploaded record and abstract](#).

Read: abstract only; full PDF retrieval was unsuccessful.

Contribution: collaborative typed hypertext supporting design deliberation through IBIS. **Limit:** no detailed usability or empirical claim is derived from this limited reading. **Relevance:** establishes that structured rationale capture substantially predates agent memory. The precursor is Kunz and Rittel's 1970 *Issues as Elements of Information Systems* ([institutional record](#)); that precursor's bibliographic record was checked, but its scanned body was not successfully read.

A03 — A Truth Maintenance System

Jon Doyle, 1979. MIT AI Memo 521; journal version in *Artificial Intelligence*. [Institutional record and abstract](#).

Read: abstract only. **Contribution:** maintaining reasons for beliefs and revising assumptions when they conflict with discoveries. **Limit:** this review does not establish the details or complexity of the TMS algorithms. **Relevance:** motivates separating logical support from task precedence. A design's invalidated premise should trigger reconsideration, not be interpreted as a completed work dependency.

A04 — PROV-DM: The PROV Data Model

Luc Moreau and Paolo Missier, editors, 2013. W3C Recommendation, 30 April 2013. [Specification](#).

Read: introduction and overview. **Contribution:** an interoperable provenance vocabulary. **Limit:** representation alone does not establish truth. **Relevance:** compare run and evidence records with existing concepts before defining new ones.

A05 — git-bug

git-bug contributors. Living official repository documentation; accessed 2026-09-20. [Repository and README](#).

Read: README overview, workflows, and development/feature pointers. **Contribution:** a distributed tracker embedded in Git, usable offline and through multiple interfaces and bridges. **Limit:** no installation, bridge round trip, concurrency test, or performance measurement was performed for this report. **Relevance:** demonstrates an alternative to checked-out Markdown nodes. The-valley's authorization and atomic-publication requirements would need separate comparison with git-bug's storage and synchronization protocol.

A06 — Beads

Beads contributors, associated with Steve Yegge's agent-tooling work. Living official documentation; accessed 2026-09-20. [Repository](#) and [README](#).

Read: README features, commands, storage modes, synchronization, and Git-free usage. **Contribution:** dependency-aware work, ready queries, atomic claiming, and persistent memory over Dolt. **Limit:** fast-moving documentation; no independent concurrency or durability test here. Marketing phrases such as conflict elimination are not adopted as findings. **Relevance:** the nearest tool-level comparison for the outcome dispatch interface. Its separate database history requires analysis of publication semantics, not a presumption that files are inherently safer.

A07 — Goal-Oriented Requirements Engineering: A Guided Tour

Axel van Lamsweerde, 2001. *IEEE International Symposium on Requirements Engineering*. [Author-hosted paper](#).

Read: introduction and sections 2–4 on goals, goal links, and specification. **Contribution:** AND/OR refinement, responsibilities, environmental assumptions, and temporal goal classes, including KAOS. **Limit:** a survey and methodological account; it does not demonstrate the cost of formalizing the-valley's workload. **Relevance:** clarifies what

an outcome means, whether children suffice, and why “attain once” differs from “maintain continuously.” Start here before expanding edge types.

A08 — HTN Planning: Complexity and Expressivity

Kutluhan Erol, James Hendler, and Dana S. Nau, 1994. *AAAI*. [Publisher-hosted paper](#).

Read: abstract, introduction, and initial planning overview; proofs not reviewed. **Contribution:** formal distinctions and complexity results for hierarchical task decomposition. **Limit:** the results depend on precise formal restrictions and cannot be transferred directly to small informal task graphs. **Relevance:** shows why choosing a decomposition is a planning problem distinct from dispatching known ready tasks. Its vocabulary helps specify which recursion and ordering restrictions an initial outcome engine accepts.

A09 — Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing

Haluk Topcuoglu, Salim Hariri, and Min-You Wu, 2002. *IEEE Transactions on Parallel and Distributed Systems*. [Paper](#); [DOI](#).

Read: abstract and introductory algorithm description; detailed experiments not reviewed. **Contribution:** HEFT and CPOP heuristics for mapping known task graphs to heterogeneous processors. **Limit:** estimated costs and a pre-existing graph differ from uncertain agent decomposition. **Relevance:** a concrete baseline for discussions of critical-path scheduling. Topology, execution costs, resource assignment, and communication all matter; root priority alone is insufficient.

A10 — A Proof for the Queuing Formula: $L = \lambda W$

John D. C. Little, 1961. *Operations Research* 9(3), 383–387.
[Publisher record and abstract.](#)

Read: abstract and theorem conditions; proof not reviewed. **Contribution:** connects average population, arrival rate, and residence time under stated conditions. **Limit:** not a universal causal rule for an unstable or changing workflow. **Relevance:** encourages measuring queues and end-to-end delay instead of counting concurrent agents as productivity. Separate integration and human-decision queues can reveal bottlenecks hidden by a single completion-rate metric.

A11 — MemGPT: Towards LLMs as Operating Systems

Charles Packer et al., 2023; revised 2024. arXiv:2310.08560.
[Version 2 full text.](#)

Read: introduction, architecture, and selected conversational/document experiments in sections 2–3. **Contribution:** explicit working and external memory with model-directed memory operations. **Limit:** task-specific evaluations and dependence on retrieval and tool-use ability; virtual memory is an analogy rather than a correctness guarantee. **Relevance:** illustrates how disposable agent context can be derived from durable records. The system still needs independent rules for the authority and promotion of stored claims.

A12 — Generative Agents: Interactive Simulacra of Human Behavior

Joon Sung Park et al., 2023. *ACM UIST*; arXiv:2304.03442. [Version 2 full text.](#)

Read: abstract, introduction, and selected end-to-end evaluation in section 7. **Contribution:** a combined experience-memory, reflection, planning, and retrieval architecture, evaluated in a simulated community. **Limit:** believability and short simulated social interactions do not establish reliable engineering outcomes. **Relevance:** architectural inspiration for memory loops, with an important warning to match evaluation criteria to the intended use. A coherent story about behavior is not the same result as verified task success.

A13 — Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn et al., 2023. *NeurIPS*; arXiv:2303.11366. [Version 4 full text](#).

Read: abstract, introduction, selected experiment/ablation passages, and limitations. **Contribution:** textual feedback from attempts retained for subsequent trials without weight updates. **Limit:** feedback quality, retry budgets, test specification, and local optimization affect the result.

Relevance: informs incident-to-memory and retry loops. Store an agent's reflective explanation as a hypothesis until observation supports it, and compare with simpler retry baselines under the same total budget.

A14 — Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu et al., 2023 preprint; 2024 *TACL*. arXiv:2307.03172. [Version 3 full text](#).

Read: introduction and selected methods/results in sections 2-4. **Contribution:** controlled position tests reveal uneven use of long contexts in the evaluated models. **Limit:** measured magnitudes belong to those models and tasks, not every later model. **Relevance:** provides a robust evaluation pattern for generated project briefings. Vary placement, irrelevant material, and context budget while keeping needed facts fixed.

A15 — LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu, 2024 preprint; 2025 *ICLR*. [Version 1 full text](#).

Read: introduction, memory-ability definitions, and selected evaluator/commercial-system details. **Contribution:** evaluates temporal change, cross-session information, updates, and abstention as well as recall. **Limit:** constructed conversational histories differ from changing source-code projects. **Relevance:** supplies dimensions for testing current-versus-historical project knowledge and justified uncertainty. A single aggregate retrieval score would conceal these failure modes.

A16 — LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues

Di Wu et al., May 2026. Preprint, arXiv:2605.12493. [Version 1 full text](#).

Read: abstract, introduction, benchmark comparison, and context-gathering formulation. **Contribution:** environment-specific memory questions about state, workflows, pitfalls, and premise awareness; bounded retrieved evidence is evaluated through a fixed reader. **Limit:** recent preprint;

web-environment question answering leaves engineering transfer unproven. **Relevance:** a promising way to isolate briefing quality from downstream model ability. Treat as an experimental design to adapt, not a ready-made measure of the-valley's theory reconstruction.

A17 — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis et al., 2020. *NeurIPS*. arXiv:2005.11401. [Paper record and abstract](#).

Read: abstract only. **Contribution:** combines learned document retrieval with generation for knowledge-intensive language tasks. **Limit:** this reading supports the broad mechanism, not a claim that its architecture is best for current repository search. **Relevance:** foundational retrieval prior art. the-valley additionally needs version, authority, and evidence semantics that retrieval relevance alone does not supply.

A18 — SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

John Yang et al., 2024. *NeurIPS*. arXiv:2405.15793. [Version 3 full text](#).

Read: introduction, interface-design principles, and selected ablation descriptions. **Contribution:** treats repository tools and their feedback as design variables for coding agents. **Limit:** specific harnesses, models, and benchmark versions constrain the empirical result. **Relevance:** supports testing small graph query and evidence-navigation tools. Comparisons of model ability must hold the tool interface constant or explicitly report its contribution.

A19 — SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

Carlos E. Jimenez et al., 2023 preprint; 2024 *ICLR*.
arXiv:2310.06770. [Version 3 full text](#).

Read: abstract, discussion/limitations, task fields, and Appendix A evaluation procedure. **Contribution:** executable repository issue-resolution tasks assessed with designated failing and regression tests. **Limit:** original Python scope and incomplete test oracles; the paper itself notes qualities that execution-based evaluation misses. **Relevance:** a useful patching baseline, but the-valley needs additional measures of architectural fit, completion semantics, and later maintenance costs.

A20 — SWE-bench Goes Live!

Linghao Zhang et al., 2025. arXiv:2505.23419. [Version 1 full text](#).

Read: abstract, introduction, and limitations appendix.
Contribution: a live-updatable task collection and automated execution-environment construction. **Limit:** the reviewed version focuses on Python and reports no repeated full experiments; fresh tasks are not proof of zero exposure.
Relevance: motivates prospective task collection and recorded environment snapshots. Keep task freshness, repository diversity, and oracle completeness as separate evaluation properties.

A21 — Proving Test Set Contamination in Black Box Language Models

Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto, 2023.
arXiv:2310.17623. Version 1; latest paper record.

Read: abstract and problem statement; proofs not reviewed.
Contribution: a statistical test using canonical versus shuffled benchmark order likelihoods. **Limit:** requires the method's assumptions and likelihood access; non-detection is not a general certificate of clean data. **Relevance:** disciplines claims about leakage. the-valley should document task exposure and retain prospective holdouts instead of labeling a benchmark uncontaminated without a defined argument.

A22 — AI Agents That Matter

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan, 2024. arXiv:2407.01502.
[Version 1 full text.](#)

Read: introduction, cost-controlled evaluation discussion, and simple-baseline results. **Contribution:** joint accuracy/cost evaluation, holdout design, and reproducibility criticism. **Limit:** its empirical comparisons concern particular benchmarks and cannot rank every current agent design. **Relevance:** a methodological priority for testing repeated dispatch and multi-step memory systems. Compare total spend and operator work, and include simple retries or escalation before attributing gains to architectural sophistication.

A23 — Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity

Joel Becker, Nate Rush, Beth Barnes, and David Rein, 2025. METR research report. [Paper.](#)

Read: abstract and introductory study design/results. **Contribution:** a randomized field comparison on experienced maintainers' own tasks, including a mismatch between perceived and measured speedup. **Limit:** specific participants, tasks, period, and tools; no universal productivity effect follows. **Relevance:** actual completion

time and later rework matter more than confidence or generated output. Read with A24 before applying the result to current agent workflows.

A24 — We are Changing our Developer Productivity Experiment Design

Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud, 24 February 2026. METR. Research update.

Read: findings, selection effects, time-measurement problems, and proposed design changes. **Contribution:** explains why later participant/task selection and concurrent agent use undermine a simple continuation of the earlier estimate. **Limit:** an update diagnosing measurement problems, not a definitive current uplift estimate.

Relevance: the-valley should record task-selection and abandonment effects. Parallel-agent elapsed time and human effort require separate accounting.

A25 — Ironies of Automation

Lisanne Bainbridge, 1983. *Automatica*. Paper.

Read: introduction and selected monitoring, intervention, and operator-skill passages. **Contribution:** explains why residual human supervision can become harder as routine operation is automated. **Limit:** industrial automation analysis, not a direct experiment on software agents.

Relevance: frames the risk that reviewers lose context

while remaining responsible for rare failures. Attention and retained understanding should be outcomes of the workflow evaluation.

A26 — A Model for Types and Levels of Human Interaction with Automation

Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens, 2000. *IEEE Transactions on Systems, Man, and Cybernetics, Part A*. DOI; [accessible paper transcription](#).

Read: abstract, introduction, and initial types/levels framework. **Contribution:** differentiates automation across information, analysis, decision, and action functions. **Limit:** a general framework does not select appropriate boundaries for this repository. **Relevance:** specify what a human retains control over rather than choosing a single autonomy level for the whole pipeline.

A27 — Principles of Mixed-Initiative User Interfaces

Eric Horvitz, 1999. *ACM CHI*. [Author-organization-hosted paper](#).

Read: principles and initial LookOut decision-making discussion. **Contribution:** reasons about uncertainty, attention, interruption costs, clarification, and reversible interaction with automation. **Limit:** illustrated through calendaring assistance, with no direct result on agent-code

review. **Relevance:** informs decision-request routing, escalation, and the value of asking at the right time. Measure whether a request resolves uncertainty efficiently, rather than counting delivery as successful human participation.

A28 — On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games

Phan Minh Dung, 1995. *Artificial Intelligence* 77, 321–357. Paper.

Read: introduction and section 2 definitions of attack, acceptability, and extension semantics. **Contribution:** a formal account of which arguments can be accepted together given their attacks. **Limit:** abstract argumentation does not validate factual premises or extract sound arguments from prose. **Relevance:** separates disagreement from work precedence. A useful graph may contain cyclic argument relationships while requiring acyclic execution dependencies; one graph does not imply one relation or one acceptance rule.

Repository source notes

1. Repository source: README.md ↵
2. Repository source: design/requirements.md ↵
3. Repository source: design/user-scenarios.md ↵
4. Repository source: .the-valley/ideas/ida-eac723e-outcome-dag.md ↵
5. Repository source: nix/valley-host.nix ↵
6. Repository source: schema/valley.cue ↵
7. Repository source: .the-valley/README.md ↵
8. Repository source: schema/node.cue ↵
9. Repository source: nix/knowledge-lint.py ↵
10. Repository source: attest/ ↵
11. Repository source: schema/attestation.cue ↵
12. Repository source: attest/conformance/ ↵
13. Repository source: integrator/main.go ↵
14. Repository source: integrator/verdict/verdict.go ↵
15. Repository source: design/integration.md ↵
16. Repository source: schema/identity.cue ↵
17. Repository source: identity/ ↵
18. Repository source: .the-valley/decisions/dcr-b87f6e8-identity-is-a-governed-registry.md ↵
19. Repository source: schema/events.cue ↵
20. Repository source: bin/valley ↵
21. Repository source: integrator/bus.go ↵
22. Repository source: design/verification.md ↵

- 23. Repository source: [design/roadmap.md ↵](#)
- 24. Repository source: [.the-valley/ideas/ida-3145b7a-demand-pressure.md ↵](#)
- 25. Repository source: [.the-valley/ideas/ida-b48bded-production-dags-and-events.md ↵](#)
- 26. Repository source: [design/self-transparency.md ↵](#)
- 27. Repository source: [.the-valley/ideas/ida-f83d7ba-theory-revival-accelerator.md ↵](#)
- 28. Repository source: [.the-valley/ideas/ida-99b87d9-theory-rebuild-as-evaluation.md ↵](#)
- 29. Repository source: [attest/run.go ↵](#)
- 30. Repository source: [attest/verify.go ↵](#)
- 31. Repository source: [integrator/refs.go ↵](#)
- 32. Repository source: [integrator/verdict/verdict.go ↵](#)
- 33. Repository source: [design/roadmap.md#phase-6--trust-backstop ↵](#)
- 34. Repository source: [attest/check.go ↵](#)
- 35. Repository source: [attest/digest.go ↵](#)
- 36. Repository source: [design/verification.md#the-mechanism-stack ↵](#)
- 37. Repository source: [schema/attestation.cue ↵](#)
- 38. Repository source: [.the-valley/bugs/bd-eaefe82-check-definitions-come-from-the-branch.md ↵](#)
- 39. Repository source: [integrator/evidence.go ↵](#)
- 40. Repository source: [integrator/verdict/verdict.go ↵](#)
- 41. Repository source: [integrator/refs.go ↵](#)
- 42. Repository source: [design/requirements.md ↵](#)
- 43. Repository source: [design/architecture.md ↵](#)

- 44. Repository source: `nix/valley-host.nix` [↵](#)
- 45. Repository source: `bin/valley` [↵](#)
- 46. Repository source: `integrator/refs.go` [↵](#)
- 47. Repository source: `identity/compile.go` [↵](#)
- 48. Repository source: `schema/events.cue` [↵](#)
- 49. Repository source: `bin/valley` [↵](#)
- 50. Repository source: `integrator/refs.go` [↵](#)
- 51. Repository source: `integrator/transfer.go` [↵](#)
- 52. Repository source: `.the-valley/decisions/dcr-62ecc36-signal-contracts.md` [↵](#)
- 53. Repository source: `integrator/main.go` [↵](#)
- 54. Repository source: `.the-valley/ideas/ida-b48bded-production-dags-and-events.md` [↵](#)
- 55. Repository source: `.the-valley/ideas/ida-f1b39e8-outbound-effects-pass-through-an-actuator.md` [↵](#)
- 56. Repository source: `design/requirements.md#durability` [↵](#)
- 57. Repository source: `.the-valley/decisions/dcr-b87f6e8-identity-is-a-governed-registry.md` [↵](#)
- 58. Repository source: `.the-valley/ideas/ida-a8243d2-agent-runs-act-under-delegated-authority.md` [↵](#)
- 59. Repository source: `identity/render.go` [↵](#)
- 60. Repository source: `.the-valley/ideas/ida-45178f6-agent-identity-is-provenance.md` [↵](#)
- 61. Repository source: `design/self-transparency.md` [↵](#)
- 62. Repository source: `.the-valley/README.md` [↵](#)
- 63. Repository source: `schema/node.cue` [↵](#)

- 64. Repository source: [nix/knowledge-lint.py ↵](#)
- 65. Repository source: [.the-valley/ideas/ida-eac723e-outcome-dag.md ↵](#)
- 66. Repository source: [.the-valley/ideas/ida-3145b7a-demand-pressure.md ↵](#)
- 67. Repository source: [.the-valley/ideas/ida-b48bded-production-dags-and-events.md ↵](#)
- 68. Repository source: [.the-valley/bugs/bd-5cb2034-unbounded-loops-under-demand-pressure.md ↵](#)
- 69. Repository source: [.the-valley/bugs/bd-c324184-occ-starvation-under-contention.md ↵](#)
- 70. Repository source: [.the-valley/bugs/bd-c4d069b-git-churn-from-task-state.md ↵](#)
- 71. Repository source: [.the-valley/ideas/ida-f83d7ba-theory-revival-accelerator.md ↵](#)
- 72. Repository source: [.the-valley/ideas/ida-99b87d9-theory-rebuild-as-evaluation.md ↵](#)
- 73. Repository source: [.the-valley/ideas/ida-aea57f0-prior-art-beads.md ↵](#)
- 74. Repository source: [.the-valley/decisions/dcr-62ecc36-signal-contracts.md ↵](#)
- 75. Repository source: [.the-valley/ideas/ida-f172c8e-project-knowledge-not-agent-instructions.md ↵](#)
- 76. Repository source: [.the-valley/ideas/ida-b48bded-production-dags-and-events.md ↵](#)
- 77. Repository source: [design/architecture.md ↵](#)
- 78. Repository source: [.the-valley/ideas/ida-45178f6-agent-identity-is-provenance.md ↵](#)
- 79. Repository source: [attest/run.go ↵](#)

- 80. Repository source: attest/check.go ↵
- 81. Repository source: attest/digest.go ↵
- 82. Repository source: attest/verify.go ↵
- 83. Repository source: integrator/evidence.go ↵
- 84. Repository source: integrator/policy.go ↵
- 85. Repository source: integrator/verdict/verdict.go ↵
- 86. Repository source: integrator/refs.go ↵
- 87. Repository source: integrator/transfer.go ↵
- 88. Repository source: schema/attestation.cue ↵
- 89. Repository source: design/requirements.md ↵
- 90. Repository source: design/architecture.md ↵
- 91. Repository source: design/roadmap.md ↵
- 92. Repository source: design/openquestions.md ↵
- 93. Repository source: design/self-transparency.md ↵
- 94. Repository source: .the-valley/decisions/dcr-62ecc36-signal-contracts.md ↵
- 95. Repository source: .the-valley/decisions/dcr-b87f6e8-identity-is-a-governed-registry.md ↵
- 96. Repository source: .the-valley/decisions/dcr-2965320-valley-is-its-repository.md ↵
- 97. Repository source: .the-valley/ideas/ida-45178f6-agent-identity-is-provenance.md ↵
- 98. Repository source: .the-valley/ideas/ida-a8243d2-agent-runs-act-under-delegated-authority.md ↵
- 99. Repository source: .the-valley/ideas/ida-f1b39e8-outbound-effects-pass-through-an-actuator.md ↵

100. Repository source: `.the-valley/ideas/ida-b48bded-production-dags-and-events.md` ↩